

INTERVIEW QUESTION BANK

Session 3 of 5

DATABASES · REDIS · KAFKA · DEVOPS

Candidate: **Gokulakonda Gautham**

Target Roles: **Backend / Platform / AI / Staff Engineer**

Target Companies: **FAANG · Indian Unicorns · Product Companies**

Categories: **07 · 08 · 09 · 10**

Tier Key:

[E] Easy [M] Mid [A] Advanced [P] Principal

Generated for exhaustive FAANG / unicorn interview preparation. All questions reference Gautham's actual project experience.

CATEGORY 07

DATABASES & SQL

PostgreSQL internals · MongoDB · Query Optimization · Schema Design · Sharding

24 questions · [E]=6 [M]=11 [A]=5 [P]=2

TOPICS COVERED IN THIS CATEGORY:

- PostgreSQL MVCC, VACUUM, HOT updates, WAL, checkpointing, buffer pool
- Query planner: EXPLAIN ANALYZE, seq scan vs index scan vs bitmap scan
- Index types: B-tree, Hash, GIN, GiST, BRIN, partial indexes, covering indexes, index-only scans
- Transaction isolation levels (READ COMMITTED → SERIALIZABLE) and anomalies
- Locking: row vs table vs advisory, deadlock detection
- Connection pooling with PgBouncer
- Partitioning: range, list, hash — and sharding strategies
- Normalization 1NF–BCNF with examples and denormalization tradeoffs
- SQL: window functions, CTEs, lateral joins, JSON/JSONB operators
- MongoDB: document model, aggregation pipeline, indexes, change streams, transactions
- N+1 problem and query optimization techniques
- Multi-tenant patterns, Row Level Security

[E] EASY Category 07 > PostgreSQL > MVCC Basics

Q: What is MVCC in PostgreSQL and why does it exist?

ANSWER:

MVCC (Multi-Version Concurrency Control) is PostgreSQL's strategy for allowing concurrent reads and writes without locking readers against writers. Instead of locking a row when it's read or written, PostgreSQL maintains multiple versions of each row simultaneously.

When a transaction updates a row, PostgreSQL does NOT overwrite the existing row. Instead it:

- Marks the old row version as expired (sets xmax = current transaction ID)
- Inserts a new row version (with xmin = current transaction ID)

Each row tuple carries two system columns: xmin (the transaction that created it) and xmax (the transaction that deleted/updated it). When a query runs, it uses a "snapshot" of active transactions to determine which row version is visible to it.

Why MVCC exists: Traditional locking (as in MySQL MyISAM) causes readers to block writers and writers to block readers. MVCC eliminates this — a SELECT never blocks an INSERT/UPDATE and vice versa. This is critical for high-concurrency OLTP systems like Deepta AI's onboarding platform, where student reads and admin writes happen simultaneously without degrading throughput.

The cost of MVCC: Dead tuples accumulate (old row versions no longer visible to any transaction).

PostgreSQL's VACUUM process reclaims this space. Without VACUUM, tables bloat and query plans degrade.

IMPLEMENTATION CHALLENGE:

In your Deepta AI PostgreSQL setup, write a script that demonstrates MVCC visibility in action:

1. Open two psql sessions (session A and session B)
2. Session A: BEGIN; UPDATE students SET status='enrolled' WHERE id=1; (do NOT commit yet)
3. Session B: SELECT status FROM students WHERE id=1; — what do you see and why?
4. Session A: COMMIT;
5. Session B: SELECT status FROM students WHERE id=1; — what changes?

Extend this: run EXPLAIN ANALYZE on a full table scan and observe how dead tuples affect cost estimates. Then run VACUUM ANALYZE and re-run to see the difference.

FOLLOW-UP PROBES:

- How does PostgreSQL's snapshot isolation differ from MySQL's InnoDB implementation of MVCC?
- What happens to MVCC snapshots in long-running transactions? How does this cause table bloat?
- What is transaction ID wraparound and how does PostgreSQL prevent it?
- Your Deepta AI platform has a reporting job that runs for 4 hours — what MVCC-related issue does this create and how do you fix it?

[E] EASY Category 07 > PostgreSQL > Index Types

Q: What are the different index types in PostgreSQL and when do you use each one?**ANSWER:**

PostgreSQL supports several index types, each optimised for different query patterns:

B-tree (default): A balanced tree structure supporting equality (=), range (<, >, BETWEEN), LIKE 'prefix%', IS NULL, and ORDER BY. This covers ~95% of use cases. Used in Deepta AI for student ID lookups, date range queries on application timestamps.

Hash: Stores a hash of the indexed value. Supports ONLY equality (=) comparisons, not ranges. Slightly faster than B-tree for pure equality lookups but not WAL-logged before PostgreSQL 10 (making them unsafe on crash). Rarely preferred over B-tree.

GIN (Generalised Inverted Index): Stores a mapping from element values to the rows containing them. Best for: full-text search (tsvector/tsquery), JSONB containment (@>, ?), arrays (@>, &&). E.g., indexing JSONB application metadata in Deepta AI for field existence checks.

GiST (Generalised Search Tree): A framework for custom index strategies. Used for geometric data, range types (tsrange, daterange), PostGIS geometry. Good for "does this range overlap that range" queries.

BRIN (Block Range Index): Stores min/max values per range of physical pages. Tiny index size but only useful when column values correlate strongly with physical storage order (e.g., an auto-incrementing created_at timestamp). Excellent for time-series append-only tables.

SP-GiST: Space-partitioned GiST. For non-balanced structures like quad trees, k-d trees. Niche use for geometric or IP-range lookups.

Partial Index: Any index type filtered by a WHERE clause. E.g., CREATE INDEX ON applications(student_id) WHERE status='pending' — dramatically reduces index size and improves performance when queries always filter on that condition.

Covering Index (INCLUDE): Store extra columns in the index leaf nodes (CREATE INDEX ON applications(student_id) INCLUDE (status, university_id)) to enable index-only scans without a heap fetch.

IMPLEMENTATION CHALLENGE:

For the Deepta AI student application table with columns: (id UUID, student_id UUID, university_id UUID, status TEXT, metadata JSONB, created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ):

1. Design indexes for these query patterns: (a) find all pending applications for a university, (b) search metadata for applications where source='google_ads', (c) range query for applications created in the last 30 days, (d) full-text search on essay_text stored in metadata JSONB.
2. For each, justify the index type.
3. Write EXPLAIN ANALYZE for each query before and after the index.
4. Identify which indexes could be combined as covering indexes.

FOLLOW-UP PROBES:

- What is an index-only scan and what conditions must be met for PostgreSQL to use one?
- When would a BRIN index outperform a B-tree index on a timestamp column?
- You added a GIN index on a JSONB column but queries are still doing seq scans. What are the possible reasons?
- If you have a partial index on status='pending' but a query says WHERE status='pending' OR status='processing', does PostgreSQL use the partial index? Why or why not?

[E] EASY Category 07 > PostgreSQL > Transaction Isolation

Q: Name the four SQL transaction isolation levels and the anomalies each prevents.

ANSWER:

The SQL standard defines four isolation levels. PostgreSQL implements them using MVCC (not locking), which affects how they behave in practice:

READ UNCOMMITTED: In theory allows dirty reads (reading uncommitted changes from another transaction). PostgreSQL does NOT actually implement this — it silently upgrades to READ COMMITTED. So dirty reads are impossible in PostgreSQL.

READ COMMITTED (PostgreSQL default): Each statement within a transaction sees a fresh snapshot of committed data. Prevents dirty reads. Does NOT prevent non-repeatable reads (re-reading a row may return different results if another transaction committed between the two reads) or phantom reads (a range query may return different rows).

REPEATABLE READ: The transaction takes a snapshot at the START of the first statement and uses it for all subsequent reads. Prevents dirty reads and non-repeatable reads. In PostgreSQL, also prevents phantom reads (stronger than the SQL standard requires). Does NOT prevent serialization anomalies (write skew, lost updates in certain patterns).

SERIALIZABLE (SSI): PostgreSQL's Serializable Snapshot Isolation uses predicate locking to detect and abort transactions that would cause a non-serializable schedule. Prevents all anomalies: dirty read, non-repeatable read, phantom read, write skew. Highest correctness but incurs overhead from predicate tracking.

Common anomalies:

- Dirty read: Reading another transaction's uncommitted changes (impossible in PostgreSQL)
- Non-repeatable read: Same SELECT returns different rows within same transaction because another transaction committed between reads
- Phantom read: A range query returns different rows because rows were inserted/deleted by a concurrent transaction
- Write skew: Two transactions read overlapping data, each decides to update based on what they read, but the combined result violates a constraint (classic: two doctors both see they're not the last on-call and both check out)

IMPLEMENTATION CHALLENGE:

Reproduce each anomaly in PostgreSQL using two concurrent psql sessions:

1. Non-repeatable read: Use READ COMMITTED isolation. Session A begins, reads student count. Session B inserts and commits. Session A re-reads — count differs. Show the query transcript.
2. Write skew simulation: Create a table with constraint "at least one admin must exist." Session A and B both see two admins, both decide to delete their own admin record — show how REPEATABLE READ allows both to proceed and violate the constraint.
3. Serializable prevention: Repeat write skew scenario with SET TRANSACTION ISOLATION LEVEL SERIALIZABLE — observe which transaction gets aborted with error code 40001.

FOLLOW-UP PROBES:

- PostgreSQL's REPEATABLE READ prevents phantom reads — this is stronger than the SQL standard requires. How does MVCC achieve this without range locks?
- What is write skew and why does REPEATABLE READ not prevent it while SERIALIZABLE does?
- At Deepta AI, you process student fee payments — which isolation level would you use and why?
- What is the performance overhead of SERIALIZABLE isolation and how does PostgreSQL's SSI differ from traditional two-phase locking?

[E] EASY Category 07 > PostgreSQL > WAL Basics

Q: What is the Write-Ahead Log (WAL) in PostgreSQL and why is it fundamental to durability?

ANSWER:

The WAL (Write-Ahead Log) is PostgreSQL's mechanism for ensuring durability (the D in ACID). The core principle: before any change is written to the actual data files on disk, a record of that change is first written

to the WAL log. If PostgreSQL crashes mid-operation, it replays the WAL on restart to recover to a consistent state.

How WAL works:

1. A transaction modifies rows in the shared buffer pool (in-memory)
 2. A WAL record describing the change is written to the WAL buffer
 3. When a transaction commits, the WAL buffer is flushed to disk (fsync)
 4. Only later (at checkpoint time) are the dirty data pages in the buffer pool written to the actual heap files
- Why "write-ahead": The WAL is written BEFORE the data page — hence "write-ahead." This means you never need to flush the entire modified data page to disk on every commit, only the WAL record (which is sequential and much smaller), giving far better write throughput.

WAL enables:

- Crash recovery: Replay WAL records after a crash to reconstruct the committed state
- Point-in-time recovery (PITR): Archive WAL segments to S3/GCS and restore to any point in time
- Streaming replication: Ship WAL records to standbys in real time
- Logical replication: Decode WAL records to replicate to heterogeneous systems

WAL in your GCP GKE setup at Deepti AI: Cloud SQL PostgreSQL automatically handles WAL archiving to GCS for PITR. Understanding WAL is critical when setting `wal_level` (minimal, replica, logical) to support your replication requirements.

IMPLEMENTATION CHALLENGE:

Simulate WAL behavior in a local PostgreSQL instance:

1. Enable WAL archiving: set `archive_mode=on`, `archive_command='cp %p /tmp/wal_archive/%f'`
2. Create a table and insert 10,000 rows
3. Observe the WAL segment files created in `pg_wal/`
4. Simulate a crash: `pg_ctl stop -m immediate`
5. Restart and observe recovery from WAL in the logs
6. Use `pg_waldump` to inspect the WAL records for your INSERT operations and describe what each record contains.

Bonus: measure the difference in write throughput with `synchronous_commit=on` vs `synchronous_commit=off` — what durability guarantee do you sacrifice?

FOLLOW-UP PROBES:

- What is a checkpoint in PostgreSQL and how does it relate to WAL? What happens if checkpoints are too infrequent?
- Explain the difference between `wal_level=replica` and `wal_level=logical`. When do you need logical?
- What is `synchronous_commit=off` and when would you deliberately use it?
- Your Deepti AI database has massive write spikes during university application deadlines. How would you tune WAL and checkpoint settings to handle this?

[E] EASY Category 07 > PostgreSQL > Normalization

Q: Explain normalization forms 1NF through BCNF with concrete examples. When do you intentionally denormalize?

ANSWER:

Normalization is a systematic approach to designing relational schemas to reduce data redundancy and prevent update anomalies.

1NF (First Normal Form): Each column contains atomic values (no repeating groups, no arrays as comma-separated strings). A table has a defined primary key.

Bad: `student_courses = (student_id, "CS101,CS102,CS103")`. Good: separate `student_courses` table with `(student_id, course_id)` rows.

2NF (Second Normal Form): Must be in 1NF, and every non-key attribute must depend on the WHOLE primary key (no partial dependencies). Relevant when the primary key is composite.

Bad: (student_id, course_id, student_name, course_name). student_name depends only on student_id (partial dependency). Good: split into students(student_id, student_name) and enrollments(student_id, course_id) and courses(course_id, course_name).

3NF (Third Normal Form): Must be in 2NF, and no non-key attribute depends on another non-key attribute (no transitive dependencies).

Bad: students(student_id, department_id, department_name). department_name depends on department_id, not student_id. Good: separate departments(department_id, department_name).

BCNF (Boyce-Codd Normal Form): A stricter version of 3NF. For every functional dependency $A \rightarrow B$, A must be a superkey. Handles anomalies that 3NF misses when there are overlapping candidate keys.

4NF and 5NF address multi-valued dependencies and join dependencies — rarely applied in practice.

When to denormalize intentionally:

- Read-heavy analytics: Pre-joining tables into wide "fact" tables eliminates expensive JOINS at query time
- Materialized views: PostgreSQL can maintain a denormalized aggregate that's refreshed periodically
- Caching frequently-read computed fields (e.g., storing total_applications_count on university row instead of COUNT(*) join)
- Event sourcing / audit tables: Append-only denormalized snapshots for history

In Deepta AI's multi-tenant student platform, the core OLTP tables should be normalized (3NF/BCNF) to prevent anomalies, while reporting queries can use materialized views or read replicas with denormalized projections.

IMPLEMENTATION CHALLENGE:

Given Deepta AI's student application domain, design a normalized schema (at minimum 3NF) for:

- Students (profile, contact info, nationality)
- Universities (name, country, departments)
- Applications (student applies to a department at a university, with status, documents, timestamps)
- Lead sources (Google Ads, Facebook, IVR — each with campaign metadata)
- Users (university admins, Deepta AI staff)

Identify all functional dependencies. Show how violations of each NF arise if you collapse these into fewer tables. Then design one intentional denormalization: a materialized view for "application_summary_per_university" that university dashboards query.

FOLLOW-UP PROBES:

- Your students table stores country_code and country_name. Is this a 2NF or 3NF violation? Explain.
- BCNF can sometimes prevent certain lossless-join decompositions. Give an example where decomposing to BCNF forces you to use a more complex join strategy.
- When would you choose not to normalize past 3NF? Give a data-warehouse specific example.
- How does PostgreSQL's JSONB column challenge traditional normalization and when is it acceptable to store a blob of attributes as JSON in an otherwise-normalized schema?

[E] EASY Category 07 > SQL > Window Functions

Q: Explain SQL window functions. What is the difference between RANK, DENSE_RANK, and ROW_NUMBER?

ANSWER:

Window functions perform calculations across a set of rows related to the current row, without collapsing rows (unlike GROUP BY). The window is defined by OVER(...).

Anatomy of a window function:

```
function_name(...) OVER (
  PARTITION BY col1      -- groups (like GROUP BY, but rows not collapsed)
  ORDER BY col2 DESC     -- ordering within the partition
  ROWS/RANGE BETWEEN ... -- frame specification
)
```


Key ranking functions:

ROW_NUMBER(): Assigns a unique sequential integer to each row within the partition. Never produces ties — if two rows are equal in ORDER BY, they still get different numbers (arbitrary order for ties).

RANK(): Assigns the same rank to ties, then SKIPS the next rank(s). E.g., ranks 1, 1, 3, 4 (no rank 2 after two tied firsts).

DENSE_RANK(): Assigns the same rank to ties but does NOT skip ranks. E.g., 1, 1, 2, 3.

Practical window functions:

- **LAG(col, n):** Value from n rows before current in partition (e.g., previous month's revenue)
- **LEAD(col, n):** Value from n rows ahead
- **SUM/AVG/COUNT OVER():** Running totals, moving averages
- **NTILE(n):** Divides rows into n buckets (e.g., quartiles)
- **FIRST_VALUE / LAST_VALUE:** First/last in window frame
- **PERCENT_RANK, CUME_DIST:** Relative rank percentiles

In Deepta AI context: window functions are invaluable for "show the latest application per student" (ROW_NUMBER partitioned by student_id ordered by created_at DESC, then filter WHERE rn=1), or "running count of applications per university by day."

IMPLEMENTATION CHALLENGE:

Write SQL window function queries on the Deepta AI applications table (columns: id, student_id, university_id, status, created_at, counselor_id, lead_source):

1. For each university, find the most recent application (use ROW_NUMBER).
 2. Rank students by number of applications submitted (use RANK and DENSE_RANK, explain the difference in output).
 3. Calculate a 7-day rolling average of applications per day across all universities.
 4. For each counselor, show each application and the previous application they handled (use LAG).
 5. Partition applications into quartiles by submission time to identify seasonal patterns (use NTILE).
- For each query, include EXPLAIN ANALYZE output and explain what index would speed it up.

FOLLOW-UP PROBES:

- What is the difference between ROWS BETWEEN and RANGE BETWEEN in window frame specification? When does it matter?
- Can you use a window function in a WHERE clause? What is the correct approach to filter on window function results?
- How does PostgreSQL execute window functions — does it require a separate sort for each window function in the same query?
- Your query uses ROW_NUMBER() OVER (PARTITION BY student_id ORDER BY created_at DESC) and the table has 10M rows. What index design gives the best performance?

[M] MID Category 07 > PostgreSQL > VACUUM & Bloat

Q: Explain how PostgreSQL VACUUM works, what table bloat is, and how to detect and fix it.

ANSWER:

VACUUM is PostgreSQL's house-cleaning process that reclaims space from dead tuples created by MVCC. Every UPDATE and DELETE leaves behind dead row versions that remain physically in the table until VACUUM runs.

How VACUUM works:

1. Scans heap pages to identify dead tuples (where xmax is visible to all current and future transactions)
2. Marks those tuple slots as reusable (doesn't actually shrink the file, just marks space as free for reuse)
3. Updates the Free Space Map (FSM) so future INSERTs can use those slots
4. Updates the Visibility Map (VM) to mark pages where all tuples are visible to all transactions (enables index-only scans and skip-scanning)

VACUUM FULL: Rewrites the entire table into a new file, actually returning disk space to the OS. But it acquires an ACCESS EXCLUSIVE lock, blocking all reads and writes — use sparingly, prefer `pg_repack` in production.

autovacuum: PostgreSQL runs VACUUM automatically in the background based on:

- `autovacuum_vacuum_threshold` (default 50 dead tuples)
- `autovacuum_vacuum_scale_factor` (default 0.2 = 20% of table)

Fires when `dead_tuples > threshold + scale_factor * total_rows`.

Table bloat symptoms and detection:

- Table size grows despite no data increase (`pgstattuple` extension)
- Slow sequential scans (more pages to read)
- Query planner chooses seq scan even with good indexes (stale `pg_class.reltuples`)
- Query: `SELECT relname, n_dead_tup, n_live_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC`

HOT updates (Heap Only Tuple): If an updated column is not indexed, PostgreSQL can use a HOT update — the new tuple goes on the same page, with a chain pointer from the old to new. The old index entry is not updated. This is much cheaper than a regular update (no index update) and the space is reclaimed by "pruning" without a full VACUUM pass.

In Deepta AI's platform, application status transitions (`pending` → `under_review` → `accepted`) cause frequent updates. If status is indexed, these are full updates (no HOT). Monitor `n_dead_tup` for the applications table vigilantly.

IMPLEMENTATION CHALLENGE:

On a test PostgreSQL database with a `student_applications` table (1M rows):

1. Write a script that updates 50,000 rows repeatedly (simulating status changes) and monitors table size with `SELECT pg_size_pretty(pg_total_relation_size('student_applications'))`
2. After each batch of updates, check `n_dead_tup` in `pg_stat_user_tables`
3. Disable autovacuum on the table: `ALTER TABLE student_applications SET (autovacuum_enabled=false)`
4. Observe bloat growth using: `SELECT * FROM pgstattuple('student_applications')`
5. Run `VACUUM ANALYZE` and observe space reclamation
6. Re-enable autovacuum and tune `autovacuum_vacuum_scale_factor=0.01` (1%) to run more aggressively — appropriate for high-churn tables
7. Compare query times before and after VACUUM for a range scan on `created_at`

FOLLOW-UP PROBES:

- What is transaction ID wraparound and how does PostgreSQL's aggressive autovacuum prevent it? What happens if it's not prevented?
- Why does VACUUM not return disk space to the OS but VACUUM FULL does? What is the production-safe alternative to VACUUM FULL?
- How does the Visibility Map enable index-only scans and how does VACUUM maintain it?
- Your Deepta AI applications table has autovacuum running but table bloat is still growing. The table receives 10,000 updates/second during peak hours. What do you change?

[M] MID Category 07 > PostgreSQL > Query Planner & EXPLAIN

Q: Walk through what EXPLAIN ANALYZE output tells you. What is the difference between Seq Scan, Index Scan, and Bitmap Index Scan?

ANSWER:

EXPLAIN shows the query plan — what PostgreSQL's planner decided to do and why. EXPLAIN ANALYZE actually executes the query and shows both the planned and actual statistics.

Reading EXPLAIN ANALYZE:

Each node shows:

- Operation type (Seq Scan, Index Scan, Hash Join, etc.)
- `cost=X..Y`: X = startup cost (before first row), Y = total cost (in arbitrary planner units)

- rows=N: estimated rows returned
- width=W: estimated bytes per row
- Actual time=a..b: actual time in milliseconds (requires ANALYZE)
- actual rows=N: actual rows returned
- loops=N: how many times this node was executed (important for nested loops)

Critical discrepancy: if estimated rows << actual rows, your table statistics are stale — run ANALYZE.

Seq Scan (Sequential Scan): Reads every page of the table in order. PostgreSQL chooses this when:

- No suitable index exists
- The query returns a large fraction of rows (index overhead exceeds seq scan cost)
- Table statistics suggest high selectivity is wrong

PostgreSQL will use parallel seq scans (Parallel Seq Scan) for large tables if `max_parallel_workers_per_gather > 0`.

Index Scan: Uses the B-tree (or other index) to look up specific heap pages. Returns rows in index order. Has random I/O to the heap for each match. Good for high-selectivity queries (returning few rows). Bad if many rows match (too much random I/O).

Index Only Scan: Like Index Scan but doesn't need to visit the heap at all — all needed columns are in the index (covering index + visibility map shows page is all-visible). The fastest read path for appropriate queries.

Bitmap Index Scan + Bitmap Heap Scan: PostgreSQL builds a bitmap of heap pages that contain matching rows (using the index), then scans those heap pages in physical order (reducing random I/O). Used when an Index Scan would produce too much random I/O but a Seq Scan would be wasteful. Also used when combining multiple indexes (BitmapAnd, BitmapOr).

Join strategies:

- Nested Loop: Good for small outer set with indexed inner lookups
- Hash Join: Build hash table on smaller relation, probe with larger — good for large equijoins
- Merge Join: Efficient when both sides are already sorted

For Deepti AI: use EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) to see buffer hit/miss rates, which reveals whether queries are reading from cache or disk.

IMPLEMENTATION CHALLENGE:

Take the Deepti AI query: "Find all pending applications for universities in India, showing student name and counselor email, ordered by created_at DESC, LIMIT 100."

1. Write the SQL query with appropriate JOINS.
2. Run EXPLAIN ANALYZE before any indexes — note the scan types.
3. Add a composite index on (university_id, status, created_at DESC) and re-run EXPLAIN ANALYZE.
4. Add covering index INCLUDE (student_id, counselor_id) and check if you get an Index Only Scan.
5. Force a Bitmap Index Scan scenario by changing LIMIT 100 to no limit — observe the plan change.
6. Introduce a statistics mismatch: DELETE 90% of rows, don't run ANALYZE, run EXPLAIN ANALYZE — show how the planner makes a poor decision. Fix with ANALYZE.

FOLLOW-UP PROBES:

- What does cost=0.00..4234.00 rows=1000 width=32 actually mean? What units is the cost in?
- Why might PostgreSQL choose a Seq Scan over an Index Scan even when an index exists and the query is highly selective?
- What is a generic query plan vs a custom query plan in PostgreSQL? When does PostgreSQL use prepared statement plan caching and what is 'plan instability'?
- Your EXPLAIN ANALYZE shows actual rows=500000 but estimated rows=100. What does this mean and what are all the possible causes?

[M] MID Category 07 > PostgreSQL > Locking

Q: Explain PostgreSQL's lock hierarchy: row-level, table-level, and advisory locks. How does deadlock detection work?

ANSWER:

PostgreSQL has a multi-level locking system:

Table-level locks (in order of increasing conflict):

1. ACCESS SHARE: Acquired by SELECT. Conflicts only with ACCESS EXCLUSIVE.
2. ROW SHARE: Acquired by SELECT FOR UPDATE/SHARE. Conflicts with EXCLUSIVE and ACCESS EXCLUSIVE.
3. ROW EXCLUSIVE: Acquired by INSERT, UPDATE, DELETE. Conflicts with SHARE and above.
4. SHARE UPDATE EXCLUSIVE: Acquired by VACUUM, CREATE INDEX CONCURRENTLY, ANALYZE, ALTER TABLE (some). Conflicts with itself and above.
5. SHARE: Acquired by CREATE INDEX (non-concurrent). Conflicts with ROW EXCLUSIVE.
6. SHARE ROW EXCLUSIVE: Conflicts with SHARE and above.
7. EXCLUSIVE: Conflicts with everything except ACCESS SHARE.
8. ACCESS EXCLUSIVE: The heaviest lock. Acquired by DROP TABLE, TRUNCATE, ALTER TABLE (most), LOCK TABLE. Conflicts with ALL other locks, including simple SELECTs.

Row-level locks: PostgreSQL uses tuple-level locks for concurrent DML. When a transaction updates a row, it locks that specific tuple (not the page, not the table). Other transactions can update different rows in the same table simultaneously.

- FOR UPDATE: Exclusive row lock. Prevents other FOR UPDATE, FOR SHARE, UPDATE, DELETE.
- FOR SHARE: Shared row lock. Prevents FOR UPDATE/DELETE but allows other FOR SHARE.
- FOR NO KEY UPDATE: Like FOR UPDATE but allows FOR KEY SHARE.
- FOR KEY SHARE: Weakest. Allows everything except FOR UPDATE.

Advisory locks: Application-managed locks, not tied to database objects. Your application can call `pg_advisory_lock(key)` to serialize operations on arbitrary resources (e.g., "only one worker processes university_id=42 at a time"). You must explicitly release them or they're released on session end.

Deadlock detection: PostgreSQL uses a wait-for graph. When a transaction waits for a lock, it registers in the lock manager. A background process periodically checks for cycles in the wait-for graph. When a cycle is detected, PostgreSQL aborts the youngest transaction (by transaction ID) with error code 40P01. The aborted transaction receives a deadlock error and must be retried.

Prevention strategies:

- Always acquire multiple locks in the same order across all transactions
- Keep transactions short and focused
- Use NOWAIT or lock_timeout to fail fast instead of waiting indefinitely

IMPLEMENTATION CHALLENGE:

In Deepta AI's application processing system:

1. Design a scenario where two concurrent processes (Process A: "approve application X and debit university quota," Process B: "check university quota and update application X") deadlock.
2. Write the actual SQL transactions that cause the deadlock and reproduce it.
3. Observe PostgreSQL's deadlock error message and identify which transaction was aborted.
4. Fix the deadlock by reordering lock acquisition.
5. Implement an advisory lock solution: `pg_try_advisory_xact_lock(university_id)` to ensure only one worker processes a university's application queue at a time. Handle the case where the lock is not available (skip and retry).
6. Use `pg_locks` and `pg_stat_activity` to monitor lock contention in real time.

FOLLOW-UP PROBES:

- What is lock_timeout vs statement_timeout vs idle_in_transaction_session_timeout and when do you use each?
- Why does CREATE INDEX (non-concurrent) block all writes but CREATE INDEX CONCURRENTLY doesn't? What is the trade-off?
- What is the difference between SELECT FOR UPDATE and SELECT FOR UPDATE SKIP LOCKED? When is SKIP LOCKED the right pattern?

- Your ALTER TABLE ADD COLUMN statement has been waiting for an ACCESS EXCLUSIVE lock for 10 minutes in production. What is likely holding the lock and how do you safely resolve this without killing the ALTER?

[M] MID Category 07 > PostgreSQL > Connection Pooling

Q: Why does PostgreSQL struggle with high connection counts? Explain PgBouncer's pooling modes and how you'd configure it for a multi-tenant platform.

ANSWER:

PostgreSQL's connection model: Each client connection spawns a dedicated OS process (fork) with its own memory allocation (~5-10MB per connection for shared_buffers overhead, stack space, etc.). This is fundamentally different from thread-per-connection models.

At 500+ concurrent connections, PostgreSQL faces:

- Memory pressure (500 * 10MB = 5GB just for connection overhead)
- Context switching overhead between OS processes
- Lock table contention (pg_locks management)
- Scheduler burden

Most PostgreSQL instances start degrading significantly beyond 200-300 connections.

PgBouncer is a lightweight connection pooler that sits between your application and PostgreSQL:

Session pooling: A client connection maps to a server connection for the duration of the session. The server connection is returned to the pool when the client disconnects. Provides the least multiplexing. Safe for all PostgreSQL features (SET, prepared statements, LISTEN).

Transaction pooling: A server connection is held only for the duration of a transaction. Between transactions, the client connection is served from the pool, and the server connection returns to the pool. Provides maximum multiplexing (100 clients can share 10 server connections if their transactions are short).

LIMITATION: Cannot use session-level features (SET role, advisory locks, LISTEN/NOTIFY, prepared statements with protocol-level binding).

Statement pooling: Server connection returned after each statement. Extremely aggressive multiplexing. Only safe for simple autocommit workloads. Almost never used.

Configuration for Deepta AI's multi-tenant platform:

- Use transaction mode (the common production choice)
- max_client_conn = 5000 (application connections)
- default_pool_size = 25 (server connections per database/user pair)
- For multi-tenant: configure per-user or per-database pools based on university size
- pgbouncer.ini: pool_mode=transaction, server_idle_timeout=600
- Enable auth_type=scram-sha-256

On GCP: Cloud SQL with PgBouncer sidecar or use Cloud SQL's built-in connection pooling. Alternatively, pgpool-II for more advanced features (load balancing across replicas).

IMPLEMENTATION CHALLENGE:

For the Deepta AI platform, you have 50 Golang microservices, each maintaining a connection pool of max 10 connections, deployed across 20 pods (GKE), targeting a single PostgreSQL instance. That's potentially $50 \times 10 \times 20 = 10,000$ connections.

1. Design the PgBouncer deployment: How many PgBouncer instances? Where do they sit (sidecar vs dedicated service)?
2. Write the pgbouncer.ini configuration for transaction mode with 2,000 max server connections.
3. In your Go services, you use database/sql — what pool settings do you configure (SetMaxOpenConns, SetMaxIdleConns, SetConnMaxLifetime)?
4. What happens to Deepta AI's Kafka consumer goroutines that call PostgreSQL within Kafka message processing? Is transaction pooling safe here?
5. Implement a health check endpoint that verifies PgBouncer connectivity and reports pool utilization.

FOLLOW-UP PROBES:

- PgBouncer transaction mode doesn't support prepared statements at the protocol level. How do you handle this in a Go application that uses pgx with prepared statement caching?
- How would you handle database authentication rotation (password changes) in PgBouncer without dropping all connections?
- What is the difference between PgBouncer and Pgpool-II? When would you choose one over the other?
- Your Deepta AI application sees 'connection refused' errors from PgBouncer during peak load. Walk through your diagnosis and the metrics you'd check.

[M] MID Category 07 > PostgreSQL > Partitioning

Q: Explain PostgreSQL's declarative table partitioning — range, list, and hash. When does partitioning help vs hurt?

ANSWER:

PostgreSQL 10+ has declarative partitioning: you define a parent table and PostgreSQL manages child tables (partitions) transparently.

Range partitioning: Partitions by a range of values on a column, typically dates or integers.

```
CREATE TABLE applications (id UUID, created_at TIMESTAMPTZ, ...) PARTITION BY RANGE (created_at);
```

```
CREATE TABLE applications_2024 PARTITION OF applications FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

```
CREATE TABLE applications_2025 PARTITION OF applications FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

List partitioning: Partitions by discrete values.

```
CREATE TABLE applications_by_region PARTITION BY LIST (region);
```

```
CREATE TABLE applications_india PARTITION OF applications_by_region FOR VALUES IN ('IN');
```

```
CREATE TABLE applications_uk PARTITION OF applications_by_region FOR VALUES IN ('UK');
```

Hash partitioning: Distributes rows evenly by hashing a column. No natural "slice" semantics — used to spread load.

```
CREATE TABLE applications PARTITION BY HASH (student_id);
```

```
CREATE TABLE applications_0 PARTITION OF applications FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

When partitioning HELPS:

- Partition pruning: Queries with WHERE created_at BETWEEN '2025-01-01' AND '2025-03-31' only scan the 2025 partition, not the entire table
- Efficient archival: DROP TABLE applications_2022 — instant, no VACUUM needed (dropping a partition)
- Index maintenance: Each partition has its own indexes, making REINDEX faster
- Vacuum efficiency: VACUUM runs per partition, reducing lock scope
- Parallel query: Each partition can be scanned in parallel

When partitioning HURTS:

- Cross-partition queries (WHERE across multiple partitions) may be slower than a single unpartitioned table scan (planning overhead)
- Partition-wise joins require both sides to be partitioned by the same key
- Unique constraints must include the partition key (a global unique index across partitions is not directly supported)
- Too many partitions (>100+) increases planning time
- Foreign keys TO partitioned tables work but are slower

For Deepta AI: partition applications by created_at (range, monthly or yearly) for time-based queries and archival, and potentially sub-partition by region for geographic queries.

IMPLEMENTATION CHALLENGE:

Design and implement partitioning for Deepta AI's application_events table (audit log of all status changes):

- Schema: (id BIGSERIAL, application_id UUID, event_type TEXT, old_status TEXT, new_status TEXT, created_at TIMESTAMPTZ, payload JSONB)

- Expected volume: 5M events/month, kept for 3 years

- Common queries: last 30 days of events per application, count events by type per month

1. Create a range-partitioned table by month (monthly partitions).
2. Write a script to auto-create the next month's partition in advance.
3. Write a cron job query to DROP the partition older than 36 months.
4. Demonstrate partition pruning by running EXPLAIN on a date-filtered query.
5. Add a composite index (application_id, created_at) on each partition.
6. Test: what happens if you INSERT a row with a created_at that doesn't match any partition (no default partition)?

FOLLOW-UP PROBES:

- PostgreSQL's declarative partitioning does not support global unique indexes. How do you enforce a uniqueness constraint on a non-partition-key column across all partitions?
- What is partition pruning and what are the conditions that prevent it (why might PostgreSQL not prune even when it seems it should)?
- Compare PostgreSQL table partitioning vs manual sharding to separate databases. When does each approach reach its limits?
- You're partitioning a table by student_id (hash, 8 partitions) but your university admin queries filter by university_id (not the partition key). How does this affect partition pruning and what's your strategy?

[M] MID Category 07 > SQL > CTEs and Lateral Joins

Q: What are CTEs (Common Table Expressions) and LATERAL joins? When does each help?

ANSWER:

Common Table Expressions (CTEs): Named subqueries defined at the top of a SQL statement using WITH. They improve readability and can be referenced multiple times.

Basic CTE:

```
WITH pending_apps AS (
  SELECT * FROM applications WHERE status='pending'
)
SELECT * FROM pending_apps pa JOIN universities u ON pa.university_id = u.id;
```

Recursive CTE: Used for hierarchical/graph traversal:

```
WITH RECURSIVE org_hierarchy AS (
  SELECT id, name, parent_id, 0 AS depth FROM departments WHERE parent_id IS NULL
  UNION ALL
  SELECT d.id, d.name, d.parent_id, oh.depth+1 FROM departments d JOIN org_hierarchy oh ON
  d.parent_id = oh.id
)
SELECT * FROM org_hierarchy;
```

CTE materialization (PostgreSQL 12+ change): Before PostgreSQL 12, CTEs were always "optimization fences" — materialized as a subquery, preventing the planner from pushing predicates into them. Since PostgreSQL 12, CTEs are inlined by default unless they're recursive or have side effects. Use WITH ... AS MATERIALIZED to force old behavior (good when the CTE is used multiple times and shouldn't be re-executed).

LATERAL joins: Allow a subquery on the right side of a JOIN to reference columns from the left side — like a correlated subquery but as a join.

```
SELECT u.name, recent.created_at
FROM universities u
CROSS JOIN LATERAL (
```

```
SELECT created_at FROM applications WHERE university_id = u.id ORDER BY created_at DESC LIMIT 1) recent;
```

Without LATERAL, you'd need a complex window function or correlated subquery. LATERAL is especially powerful with:

- `jsonb_to_recordset()` to expand JSONB arrays inline
- `generate_series()` correlated to row values
- `LIMIT N` per group patterns (top-N per partition, more efficient than `ROW_NUMBER` in some cases)

In Deepta AI context: Use CTEs for complex multi-step reporting queries (calculate lead conversion rate per source). Use LATERAL for "get the 3 most recent applications per student" in a single scan.

IMPLEMENTATION CHALLENGE:

Write the following SQL queries for Deepta AI using CTEs and LATERAL joins:

1. Using a recursive CTE: Given a university department hierarchy (departments table with `id`, `name`, `parent_id`), retrieve the full path from root to leaf for each department.
2. Using LATERAL: For each lead source (`google_ads`, `facebook`, `ivr`), retrieve the top 3 most recent converted applications (`status='accepted'`), including student name and university name. Do NOT use `ROW_NUMBER` (use LATERAL instead).
3. Using a CTE as an optimization fence: A complex multi-step query calculates the "conversion funnel" — count of applications at each status stage, per month, per lead source. Use multiple CTEs chained together. Ensure the intermediate CTE is materialized to avoid re-computation.
4. Compare the `EXPLAIN ANALYZE` output of the LATERAL join vs the equivalent `ROW_NUMBER` window function approach for query #2.

FOLLOW-UP PROBES:

- What changed about CTE behavior between PostgreSQL 11 and 12? What does `WITH ... AS MATERIALIZED` do and when do you force it?
- Can you use LATERAL with an `UPDATE` or `DELETE` statement? What is the use case?
- How does PostgreSQL handle LATERAL joins with set-returning functions like `jsonb_to_recordset`? Give a concrete example.
- Your recursive CTE has a cycle in the data (department A is parent of B, B is parent of A — data corruption). What happens and how do you protect against it?

[M] MID Category 07 > PostgreSQL > JSON Operators

Q: Explain PostgreSQL's JSONB operators and when to use JSONB vs a normalized relational schema.

ANSWER:

PostgreSQL has two JSON types: `json` (stored as text, parsed on each access) and `jsonb` (stored in a decomposed binary format, supports indexing, faster for most operations). Always use `jsonb` in practice.

Key JSONB operators:

- `>` : Returns JSON object field (returns JSON). Example: `metadata->'source'` returns `"google_ads"` (as JSON text with quotes)
- `>>` : Returns text value. Example: `metadata->>'source'` returns `google_ads` (without quotes)
- `#>` : Access nested path using array. Example: `metadata#>{'contact,email'}`
- `#>>` : Access nested path, return as text
- `@>` : Contains. Example: `metadata @> '{"source":"google_ads"}'` — used with GIN index
- `<@` : Is contained by
- `?` : Key exists. Example: `metadata ? 'utm_campaign'`
- `?|` : Any key exists. Example: `metadata ?| array['source','medium']`
- `?&` : All keys exist
- `||` : Concatenate two JSONB objects
- `:` : Delete key. Example: `metadata - 'internal_note'`

- `jsonb_set(target, path, new_value)`: Update nested value
- `jsonb_build_object('key', value, ...)`: Build JSONB programmatically
- `jsonb_each(metadata)`: Expand to key-value rows
- `jsonb_array_elements(arr)`: Expand a JSONB array to rows

Indexing JSONB:

- GIN index on entire column: `CREATE INDEX ON applications USING gin(metadata)` — enables `@>`, `?>`, `?|`, `?&` operators
- GIN with `jsonb_path_ops` (smaller index, only `@>` supported): `CREATE INDEX ON applications USING gin(metadata jsonb_path_ops)`
- B-tree on extracted value: `CREATE INDEX ON applications((metadata->>'source'))` — enables `=` and `ORDER BY` on a specific field
- Partial index: `CREATE INDEX ON applications((metadata->>'source')) WHERE metadata ? 'source'`

When JSONB vs normalized:

Use JSONB when:

- Schema is truly variable across rows (different lead sources have different fields)
- Rapid iteration — adding a new metadata field doesn't require `ALTER TABLE`
- EAV (entity-attribute-value) patterns that would be painful to normalize
- Semi-structured data from third-party webhooks (Google Ads, Facebook payloads vary)

Use normalized when:

- The field is queried frequently with equality/range filters (better B-tree performance)
- The field participates in foreign key constraints
- Data integrity via `CHECK` constraints is needed
- The field is joined on frequently

In Deepta AI: webhook payloads from Google Ads/Facebook are ideal JSONB candidates — their structure varies by campaign type. Core student/university attributes should be normalized columns.

IMPLEMENTATION CHALLENGE:

Design the `lead_events` table for Deepta AI that captures webhooks from Google Ads, Facebook, and IVR services — each with different payload schemas.

1. Schema: (id UUID, source TEXT, received_at TIMESTAMPTZ, payload JSONB)
2. Create appropriate GIN indexes.
3. Write queries using JSONB operators for:
 - a. Find all leads where `source='google_ads'` AND payload contains a `'campaign_id'`
 - b. Extract `campaign_name` from nested payload `->>'ad'>'campaign'>>'name'` for all Google Ads leads
 - c. Find leads where the payload has ANY of the keys: `'utm_source'`, `'utm_medium'`, `'gclid'`
 - d. Aggregate lead count by lead source per day using `payload->>'timestamp'`
4. Use `jsonb_set` to update a specific field in a payload without rewriting the entire JSONB.
5. Use `jsonb_array_elements` to normalize a JSONB array of tags into rows for a `COUNT` query.

FOLLOW-UP PROBES:

- What is the difference between GIN with default ops (`jsonb_ops`) vs `jsonb_path_ops`? Which index is smaller and why?
- Does the `@>` operator use the GIN index for nested paths like `metadata @> '{"contact": {"email": "x@y.com"}}'`? Explain.
- What is `JSONPath` in PostgreSQL 12+ and how does `jsonb_path_query` differ from the `->` operator approach?
- Your Deepta AI webhook payloads are growing to 50KB each with deeply nested arrays. You need to query specific array elements frequently. At what point does JSONB become a liability vs proper table normalization?

Q: Explain MongoDB's document model, aggregation pipeline, and when you'd choose it over PostgreSQL.**ANSWER:**

MongoDB is a document database that stores data as BSON (Binary JSON) documents in collections. Documents can have nested objects and arrays, and schema can vary across documents in the same collection.

Document model advantages:

- Embedding vs referencing: Related data can be embedded in a single document (denormalized) or referenced by ID (similar to foreign keys but without enforcement). Embed when data is always read together and belongs to one entity. Reference when data is shared across many documents or can grow unboundedly.
- Schema flexibility: Fields can differ per document. Version your application code to handle schema evolution.
- Array operators: \$push, \$pull, \$elemMatch, \$unwind — natural fit for multi-valued attributes.

MongoDB indexes:

- Single field, compound, multikey (on array fields), text, geospatial, hashed (for sharding)
- TTL index: DELETE documents automatically after a field's datetime + expiry — perfect for session stores or temporary data
- Wildcard index: Index all fields in a subdocument — useful for dynamic metadata patterns

Aggregation pipeline: A sequence of stage operators that transform a collection's documents:

- \$match: Filter (like WHERE)
- \$group: Aggregate with accumulators (\$sum, \$avg, \$count, \$push, \$first)
- \$project: Shape output (like SELECT)
- \$lookup: LEFT JOIN with another collection
- \$unwind: Deconstruct an array into separate documents
- \$sort, \$limit, \$skip: Ordering and pagination
- \$addFields: Add computed fields
- \$facet: Run multiple aggregation pipelines simultaneously
- \$bucket: Group into ranges (like CASE WHEN)

Change Streams: MongoDB 3.6+ supports change streams — a long-lived cursor that watches a collection for inserts/updates/deletes and emits change events. Useful for: real-time notifications, CDC (change data capture), maintaining derived caches. Backed by the oplog (replication log), similar to PostgreSQL's WAL.

When MongoDB over PostgreSQL:

- Highly variable document schemas (different data per record type)
- Hierarchical/embedded data that's always read as a unit
- Write-heavy workloads with horizontal scaling needs (sharding built-in)
- Rapid prototyping (schema flexibility)

When PostgreSQL over MongoDB:

- Complex relational queries with multiple JOINS
- ACID transactions across multiple collections (MongoDB has multi-document transactions but they're heavier)
- Strong consistency requirements
- Rich SQL ecosystem (BI tools, reporting)

IMPLEMENTATION CHALLENGE:

At Deepta AI, the Vigilant AI investment copilot project stores financial event data with highly variable schemas (earnings calls have different fields than regulatory filings). Design the MongoDB schema:

1. Collection: financial_events with fields: company_id, event_type (earning/filing/news), timestamp, source_url, payload (variable per type).
2. Write an aggregation pipeline that: (a) filters events from last 90 days, (b) groups by company_id and event_type to count occurrences, (c) adds a field is_high_impact based on payload.sentiment_score > 0.7, (d) looks up company metadata from a companies collection, (e) sorts by event count descending.
3. Create a TTL index that auto-deletes events older than 1 year.

4. Implement a change stream listener in Node.js that watches for new events with event_type='earnings' and triggers a Kafka message.
5. Write a query using \$elemMatch to find events where any tag in an array field matches 'AI' or 'semiconductor'.

FOLLOW-UP PROBES:

- What is the MongoDB oplog and how do Change Streams use it? What happens if your change stream listener is down for 30 minutes?
- Explain the \$lookup stage. What are its performance characteristics for large collections and how do you optimize it?
- When does embedding documents vs referencing by ID cause problems? What is the MongoDB rule of thumb for the 'unbounded array' anti-pattern?
- Your MongoDB collection has 100M documents and you run an aggregation with \$group. It OOMs. What are your options?

[M] MID Category 07 > Databases > N+1 Problem

Q: What is the N+1 query problem? How do you detect and fix it in a PostgreSQL-backed Go service?

ANSWER:

The N+1 problem: You execute 1 query to get N records, then execute N additional queries to fetch related data for each record — a total of N+1 queries when a single query could have retrieved everything. Classic example: Fetching 100 student applications, then for each application separately querying the student's profile:

Query 1: `SELECT * FROM applications LIMIT 100;` -- 1 query

For each of 100 rows: `SELECT * FROM students WHERE id = $1;` -- 100 queries

Total: 101 queries instead of 1 JOIN.

Detection:

- Logging: In Go, use pgx's query logging or a middleware that counts queries per request — alert if a single HTTP request executes >10 queries
- APM tools: Datadog APM, New Relic, or OpenTelemetry traces show individual DB spans; N+1 appears as hundreds of identical short-duration spans
- EXPLAIN: Identify if the same query is called repeatedly with different parameter values
- Database slow query log: Even fast individual queries become a pattern at scale

Solutions:

1. JOIN or subquery: Fetch students and applications in a single JOIN query. Most direct fix.
2. Batch loading (DataLoader pattern): Group all needed student IDs, then issue a single IN query: `SELECT * FROM students WHERE id = ANY($1)`. This is the N+1 → 1+1 pattern.
3. Eager loading: In your data access layer, pre-load related entities when loading the parent. ORMs like GORM in Go support `Preload()`.
4. GraphQL DataLoader: If you have a GraphQL API, use DataLoader to batch per-field resolver queries across the request.
5. Denormalization: Store the frequently-needed related field on the parent table directly (e.g., store `student_name` on `applications` — updated via trigger or application logic).

In Deepta AI's Go microservices: since you use raw SQL or a thin query builder, implement explicit batch loading. When fetching application lists, collect all `student_ids`, then `SELECT * FROM students WHERE id = ANY($1::uuid[])`.

IMPLEMENTATION CHALLENGE:

In Deepta AI's Go application service, you have an endpoint `GET /universities/:id/applications` that returns 50 applications with student name and lead source details.

1. Write the naive implementation that causes N+1 (Go + pgx pseudocode).
2. Measure with benchmarking: `httptrace` + pgx query counting to confirm 51 queries.
3. Fix with a single JOIN query and map the result to the response struct.

4. Alternatively, implement the "IN-list batch" pattern: first fetch applications, collect unique student_ids, then batch-fetch with WHERE id = ANY(\$1::uuid[]).
5. Implement a generic batchLoader[K comparable, V any] in Go that: accepts a list of keys, de-duplicates them, calls a loader function with the deduplicated keys, returns a map of key → value. Handle partial failures.
6. What changes if the list of applications can be 10,000 rows (exceeding IN clause limits)? Chunk the requests.

FOLLOW-UP PROBES:

- GraphQL APIs are particularly prone to N+1 because each field resolver is independent. Explain the DataLoader batching mechanism and how it solves this.
- Your Go service uses GORM. How does Preload() avoid N+1 and what is its SQL output for a has-many relationship?
- N+1 can also occur with Redis: fetching N cached keys individually vs MGET. Explain how MGET in your Deepta AI Redis session cache prevents this pattern.
- You fixed the N+1 with a JOIN but now the response time is worse than before because the JOIN is a hash join on a large table. What do you do?

[M] MID Category 07 > Databases > Query Optimization

Q: Walk through a systematic approach to optimizing a slow PostgreSQL query. What are the most common culprits?

ANSWER:

Systematic query optimization process:

Step 1 — Identify the slow query: Use pg_stat_statements (must be in shared_preload_libraries). Query: SELECT query, calls, total_exec_time/calls AS avg_ms, rows/calls AS avg_rows FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20. Or check the slow query log (log_min_duration_statement).
 Step 2 — EXPLAIN ANALYZE: Run EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) <your query>. Look for:

- Seq Scan on large tables (missing index or planner choosing seq scan due to stale stats)
- Nested Loop on large row counts (should be Hash Join)
- Actual rows >> Estimated rows (stale statistics — run ANALYZE)
- High Buffers hit count on disk (shared_buffers too small or data not cached)

Step 3 — Common culprits and fixes:

1. Missing index: Most common. Add index on WHERE, JOIN ON, ORDER BY columns.
2. Stale statistics: Run ANALYZE. Check pg_stat_user_tables.last_analyze.
3. Function in WHERE clause: WHERE date_trunc('day', created_at) = '2025-01-01' prevents index use. Rewrite as WHERE created_at >= '2025-01-01' AND created_at < '2025-01-02'.
4. Implicit type cast: WHERE id = 123 when id is a TEXT column causes a seq scan. Cast explicitly or match types.
5. OR conditions: WHERE status='pending' OR status='accepted' — use IN() or UNION or GIN index.
6. LIKE '%prefix%': Leading wildcard prevents B-tree index. Use full-text search (GIN) or pg_trgm (trigram) index.
7. COUNT(*) vs COUNT(col): COUNT(*) counts rows; COUNT(col) skips NULLs. For large tables, use an approximate: SELECT reltuples FROM pg_class WHERE relname='table'.
8. Correlated subqueries: Replace with JOINS or window functions.
9. ORDER BY + LIMIT on large result: Add index (sort_col) to avoid sort operation.
10. Index bloat: If an index has high bloat (pg_stat_user_indexes.idx_scan is 0 but table is large), the index may need REINDEX.

Step 4 — Query rewrite: Sometimes restructuring the SQL (CTEs, subqueries, join order hints via set enable_*=off) changes the plan.

Step 5 — Config tuning: work_mem (more memory for sorts/hashtables), effective_cache_size (planner estimate of OS cache), random_page_cost (lower for SSDs, reduces seq scan preference).

IMPLEMENTATION CHALLENGE:

You receive a production alert: this query runs in 8 seconds on the Deepta AI platform:

```
SELECT s.name, s.email, COUNT(a.id) as app_count, MAX(a.created_at) as last_app
FROM students s
LEFT JOIN applications a ON a.student_id = s.id
WHERE date_trunc('month', a.created_at) = date_trunc('month', NOW())
  AND s.country_code IN (SELECT code FROM target_countries WHERE active=true)
  AND a.status != 'draft'
GROUP BY s.id, s.name, s.email
HAVING COUNT(a.id) > 3
ORDER BY last_app DESC
LIMIT 50;
```

1. Identify every anti-pattern in this query (there are at least 4).
2. Rewrite the query to fix each one.
3. Design the indexes needed.
4. Show the EXPLAIN ANALYZE plan before and after your changes.
5. If this query still takes >500ms due to table size, what are the architectural alternatives (materialized view, read replica, pre-aggregation)?

FOLLOW-UP PROBES:

- What is the enable_seqscan, enable_indexscan planner knob and when would you toggle it in a session (and why this is not a production fix)?
- How does work_mem affect sort and hash join operations? What is the risk of setting it too high?
- What is an index-bloat problem and how do you detect and fix it without taking the table offline?
- You've exhausted query-level optimizations — the query is as fast as it can be but still takes 2 seconds. What are your next escalation paths?

[M] MID Category 07 > PostgreSQL > Sharding Strategies

Q: Explain the difference between PostgreSQL partitioning and application-level sharding. What are the tradeoffs at scale?

ANSWER:

Partitioning (single PostgreSQL instance, multiple physical tables): Splits one logical table into multiple physical child tables on the same Postgres instance. Partition pruning reduces scan scope. Great for archival, time-series data. Limited by single node's CPU, RAM, and disk. Max practical table sizes with partitioning: 1-10TB on well-tuned hardware.

Application-level sharding (multiple PostgreSQL instances): The application (or a shard coordinator) decides which database server handles a given row. Common shard keys: customer_id, tenant_id, geographic region.

Shard key selection — critical decisions:

- High cardinality (many unique values) to distribute load evenly
- Immutable — once assigned, a row shouldn't need to move shards
- Queried with equality (range queries across shards are expensive)
- Avoids hotspots — e.g., user_id is better than country_code if one country dominates

Sharding strategies:

- Range sharding: Shard 1 handles user IDs 1-1M, Shard 2 handles 1M-2M. Simple but risks hotspots on new user ranges.
- Hash sharding: shard = hash(user_id) % N. Even distribution but range queries span all shards.
- Directory-based (lookup table): A separate service maps entity ID to shard ID. Flexible but adds lookup latency.
- Consistent hashing: Used in distributed caches. Adding/removing shards only remaps K/N keys.

Cross-shard challenges:

- JOINS: Must be handled in application logic (fetch from shard A, join in memory with shard B data)
- Transactions: No native cross-shard ACID — need distributed transactions (2PC) or eventual consistency
- Aggregations: Must query all shards and merge (scatter-gather)
- Schema migrations: Must be applied to all shards in coordination

Tools: Citus (PostgreSQL extension for transparent sharding), AWS Aurora Sharding, Vitess (for MySQL). For Deepta AI's multi-tenant platform: `tenant_id` (`university_id`) is a natural shard key. Each university's data stays on one shard, making queries within a university fast. Cross-university queries (global reporting) require scatter-gather.

IMPLEMENTATION CHALLENGE:

Deepta AI is projecting 10,000 universities using the platform globally with ~500M student applications total. The current single PostgreSQL instance handles 50M rows but is approaching limits.

Design a sharding strategy:

1. Choose the shard key. Justify why `university_id` is better than `student_id` for the query patterns.
2. Design a 16-shard setup using hash sharding on `university_id % 16`.
3. Write the shard routing logic in Go: a `ShardRouter` struct that takes a `university_id` and returns a `*sql.DB` connection.
4. Handle the global reporting query: "count all applications by status across all universities." Write the scatter-gather implementation.
5. What happens when Deepta AI onboards a mega-university with 10M applications on one shard? Design a shard-split procedure.
6. How would Citus (PostgreSQL Citus extension) simplify this entire architecture?

FOLLOW-UP PROBES:

- What is a 'hot shard' and how do you detect and mitigate it without resharding everything?
- How do you handle database schema migrations when you have 16 shards, each needing the migration applied, while the application is live?
- What is the difference between sharding at the application layer vs using Citus vs using a distributed SQL database like CockroachDB or PlanetScale?
- Your shard key is `university_id`, but a regulatory requirement says you must be able to transfer a specific student's data to a different region (GDPR right to portability). How does this interact with your sharding strategy?

[A] ADVANCED Category 07 > PostgreSQL > MVCC Edge Cases

Q: Explain transaction ID wraparound in PostgreSQL. Why is it catastrophic and how does autovacuum's anti-wraparound mode prevent it?

ANSWER:

PostgreSQL uses 32-bit transaction IDs (XID). With $2^{32} \approx 4.2$ billion possible XIDs, and PostgreSQL treating XID comparison as a circular ring ($XID \pm 2^{31} = \text{the relative age}$), there's a fundamental limit: if a table hasn't been VACUUMed and its oldest XID is more than 2^{31} transactions in the past, PostgreSQL can no longer determine whether that XID is in the "past" or "future" — tuples appear to spontaneously disappear.

The wraparound hazard timeline:

- Current XID = 3 billion
- A table has tuples with `xmin` = 500 million (created 2.5 billion transactions ago)
- Approaching the 2^{31} boundary, PostgreSQL sees these tuples as "future" transactions — and hides them (they become invisible)
- Effectively, data loss and corruption

Prevention mechanism:

- `pg_class.relfrozenxid`: Tracks the oldest unfrozen XID in each table

- **VACUUM FREEZE:** Rather than just marking dead tuples, it replaces old XIDs with a special "FrozenTransactionId" (XID=2) which is always considered in the past — making it immune to wraparound
- **autovacuum anti-wraparound trigger:** When $\text{age}(\text{relfrozenxid}) > \text{vacuum_freeze_max_age}$ (default 200M transactions), autovacuum runs in "anti-wraparound mode" — it FREEZE-vacuums the table regardless of dead tuple count
- **Emergency:** When $\text{age}(\text{datfrozenxid})$ approaches $2^{31} - 10\text{M}$, PostgreSQL shuts down and refuses connections until VACUUM FREEZE is run manually — a catastrophic outage

Detection: `SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY age(datfrozenxid) DESC;`
If age approaches 1.5 billion, take action immediately.

Mitigation in Deepta AI's GCP deployment: Cloud SQL PostgreSQL handles anti-wraparound autovacuum automatically, but long-running transactions or autovacuum backlog can cause age to drift. Monitor $\text{age}(\text{datfrozenxid})$ in Prometheus with an alert at 500M.

IMPLEMENTATION CHALLENGE:

Build a PostgreSQL health monitoring dashboard for Deepta AI that tracks wraparound risk:

1. Query: `SELECT datname, age(datfrozenxid) as xid_age, 2147483647 - age(datfrozenxid) as xids_until_wraparound FROM pg_database ORDER BY xid_age DESC;`
2. Query tables at risk: `SELECT relname, age(relfrozenxid), last_autovacuum, last_autoanalyze FROM pg_class JOIN pg_stat_user_tables ON relid=relid WHERE age(relfrozenxid) > 100000000;`
3. Write a Cloud Monitoring alert (GCP) that fires when any table's relfrozenxid age > 500M.
4. Simulate high XID consumption: run a script that creates and commits 10M transactions as fast as possible — observe the XID age change rate.
5. Demonstrate that VACUUM FREEZE reduces the age to near-zero and explain what it actually does to the tuple headers.

FOLLOW-UP PROBES:

- What is multixact ID wraparound and how is it different from XID wraparound? When does it occur?
- A table has $\text{age}(\text{relfrozenxid}) = 1.8$ billion. Autovacuum is running but can't keep up because the table is continuously written to. What are your options?
- What does the error 'could not read block in file' after an XID wraparound actually look like at the application level?
- PostgreSQL 14 introduced 64-bit XIDs for the purpose of eliminating wraparound. Is this actually in production PostgreSQL yet, and what is the current state?

[A] **ADVANCED** Category 07 > PostgreSQL > Advanced Locking

Q: Explain SELECT FOR UPDATE SKIP LOCKED and how it enables scalable queue processing. What are the edge cases?

ANSWER:

`SELECT FOR UPDATE SKIP LOCKED` is PostgreSQL's mechanism for implementing work queues within a database table, eliminating the need for an external queue in some use cases.

How it works:

`SELECT id, payload FROM job_queue WHERE status='pending' ORDER BY created_at LIMIT 1 FOR UPDATE SKIP LOCKED;`

- **FOR UPDATE:** Acquires a row-level lock on the selected rows, preventing other transactions from also selecting them
- **SKIP LOCKED:** Instead of waiting for locked rows (default behavior) or failing (`NOWAIT`), it simply skips rows that are already locked by another transaction

This allows N workers to pull from the same queue table concurrently without blocking each other or causing deadlocks. Each worker gets a different row.

Pattern:

`BEGIN;`

```
SELECT id, payload FROM jobs WHERE status='queued' ORDER BY priority DESC, created_at ASC LIMIT 1 FOR UPDATE SKIP LOCKED;
```

- process the job...

```
UPDATE jobs SET status='complete', completed_at=NOW() WHERE id=$1;
COMMIT;
```

Edge cases and gotchas:

1. ORDER BY inconsistency: SKIP LOCKED happens AFTER the lock attempt. If the ordered first row is locked, the next available row is returned — which may not be the globally "next" row by order. High-priority jobs can be skipped if locked by a slower worker.
 2. Starvation: A row that is repeatedly locked by a long-running transaction can starve other workers — they keep skipping it.
 3. Transaction rollback: If a worker dies mid-processing, the lock is released and the job reverts to 'queued'. Needs a timeout mechanism (check updated_at and re-queue stale 'processing' jobs).
 4. Visibility: Workers only see committed rows (READ COMMITTED), so a row inserted by an uncommitted transaction is invisible — prevents phantom work.
 5. Performance at scale: On a table with millions of jobs, the scan to find available rows can be expensive. Add index on (status, priority, created_at) with a partial index WHERE status='queued'.
 6. No true FIFO: In the presence of multiple locked rows, the effective order is unpredictable.
- In Deepta AI context: This pattern is excellent for the student application processing queue — each application document needs async validation, and multiple Go goroutines can concurrently pick jobs without an external queue like Redis or Kafka for simpler workflows.

IMPLEMENTATION CHALLENGE:

Implement a robust PostgreSQL-backed job queue in Go for Deepta AI's document validation pipeline:

1. Schema: CREATE TABLE validation_jobs (id UUID DEFAULT gen_random_uuid(), application_id UUID, document_type TEXT, status TEXT DEFAULT 'queued', priority INT DEFAULT 5, payload JSONB, created_at TIMESTAMPTZ DEFAULT NOW(), locked_at TIMESTAMPTZ, completed_at TIMESTAMPTZ, error TEXT, attempts INT DEFAULT 0);
2. Create indexes for efficient dequeue.
3. Write Go worker pool code (5 concurrent workers) using pgx that: dequeues one job per worker using SKIP LOCKED, processes it (mock 100ms work), marks as complete or failed.
4. Handle stale jobs: a separate goroutine re-queues jobs where status='processing' AND locked_at < NOW() - INTERVAL '5 minutes'.
5. Implement dead letter: after 3 attempts, move to status='dead'.
6. Benchmark: at 10,000 jobs enqueued simultaneously, how long does processing take? At what worker count does PostgreSQL become the bottleneck vs CPU?

FOLLOW-UP PROBES:

- What happens to SELECT FOR UPDATE SKIP LOCKED ordering guarantees when you add LIMIT? Can a high-priority row be permanently skipped?
- How does SELECT FOR UPDATE SKIP LOCKED compare to using Kafka or Redis Streams for the same work queue pattern? When would you choose PostgreSQL?
- What is the advisory lock alternative to SKIP LOCKED and when might you prefer it?
- Your Deepta AI processing queue has 100,000 jobs backed up. Adding more workers causes increased lock contention instead of linear improvement. Diagnose the bottleneck and fix it.

[A] **ADVANCED** Category 07 > PostgreSQL > Replication & HA

Q: Explain PostgreSQL streaming replication and logical replication. How do you achieve zero-downtime failover?

ANSWER:

PostgreSQL Streaming Replication (Physical): The primary server ships WAL records to standby servers in real time. The standby applies the WAL stream to its own data directory, keeping an exact binary copy of the primary.

Setup:

- Primary: wal_level=replica, max_wal_senders=5, replication slot (optional)
- Standby: primary_conninfo in postgresql.conf, restore_command for WAL fetch
- Replication lag: Monitor pg_stat_replication.write_lag, flush_lag, replay_lag on primary

Synchronous vs asynchronous replication:

- Async (default): Primary commits after local WAL flush. Standby may lag. Risk of data loss on failover.
- Sync (synchronous_standby_names): Primary waits for at least one standby to confirm WAL receipt before committing. Zero data loss but added latency (round-trip to standby per commit).

Replication slots: Prevent the primary from removing WAL segments that the standby hasn't consumed yet. Dangerous if a standby disconnects — WAL accumulates unboundedly, potentially filling disk. Use replication_slot_wal_keep_size limit.

Logical replication (wal_level=logical): Decodes WAL into logical change events (INSERT/UPDATE/DELETE on specific tables) and ships them. Allows:

- Replication across different PostgreSQL versions
- Partial replication (only specific tables)
- Replication to non-PostgreSQL systems (Debezium, Kafka Connect)
- Zero-downtime major version upgrades

Zero-downtime failover:

1. Patroni: Most common HA solution. Uses etcd/ZooKeeper/Consul for distributed consensus on primary election. Automatic failover in 10-30 seconds. Works with PgBouncer (pause connections during failover, replay).
2. AWS RDS Multi-AZ: Automated. DNS TTL change on failover (60-120 seconds). Cloud SQL (GCP) equivalent.
3. Steps: (a) Confirm primary is dead (not just network partition — fencing is critical), (b) Promote standby with pg_promote(), (c) Update DNS/PgBouncer target, (d) Former primary comes back — must not become "split-brain" primary — Patroni handles fencing via etcd.

In Deepta AI's GCP GKE setup: Cloud SQL PostgreSQL HA mode uses synchronous replication to a standby in a different zone. Automatic failover in ~60 seconds. For sub-10-second RTO, run Patroni on your own GKE pods.

IMPLEMENTATION CHALLENGE:

Design a high-availability PostgreSQL setup for Deepta AI on GCP:

1. Architecture: 1 primary + 2 standbys (one sync, one async) across 3 GCP zones in asia-south1.
2. Patroni configuration in GKE: StatefulSet with 3 replicas, etcd as DCS, PgBouncer sidecar.
3. Write the Patroni YAML configuration for the primary promotion endpoint and health checks.
4. Simulate a primary failure: kill the primary pod. Measure time-to-failover using application-side latency metrics.
5. Implement a connection string strategy in Go: use the PgBouncer virtual IP, which Patroni updates on failover.
6. Handle the "split-brain" scenario: two pods both believe they are primary. What is the fencing mechanism?
7. Set up logical replication from PostgreSQL to BigQuery via Debezium for Deepta AI's analytics pipeline.

FOLLOW-UP PROBES:

- What is the difference between a replication slot and a logical replication publication/subscription? Can you have both simultaneously?
- Explain the sequence of events during a Patroni-managed failover in detail, from failure detection to new primary accepting writes.
- What is a replication lag of 5 minutes on an async standby? If the primary fails, how much data do you lose? How do you calculate the RPO?
- Your Cloud SQL instance has a standby in a different zone. A zone-level outage occurs. Walk through the recovery procedure and the expected RTO/RPO.

[A] ADVANCED Category 07 > SQL > Advanced Optimization Techniques

Q: What is a partial index and an expression index? Give examples where each provides dramatic speedups.

ANSWER:

Partial Index: An index built only on rows satisfying a WHERE condition. Dramatically smaller than a full index if only a subset of rows are queried.

Example: Applications table has 100M rows, 99% have status='completed' but queries almost always filter status='pending' (1M rows):

```
CREATE INDEX idx_pending_applications ON applications(created_at DESC) WHERE status='pending';
```

This index contains only 1M entries vs 100M for a full index. Queries with WHERE status='pending' use this tiny index. Storage savings: 99%.

Rule: The WHERE clause of the partial index must EXACTLY match (or be a superset of) the query's predicate for the planner to choose it. WHERE status='pending' does NOT help a query with WHERE status='pending' OR status='processing' (PostgreSQL can't determine the index covers both cases — use a separate index or a combined expression).

Expression Index (Functional Index): An index built on the result of an expression or function, not just a column value.

Problem: WHERE LOWER(email) = LOWER(\$1) — PostgreSQL can't use a regular index on email because the function LOWER() changes the value. Solution:

```
CREATE INDEX idx_student_email_lower ON students(LOWER(email));
```

Now WHERE LOWER(email) = 'john@example.com' uses the index.

Other common expression indexes:

- Date truncation: CREATE INDEX ON events(date_trunc('day', created_at)) — now WHERE date_trunc('day', created_at) = '2025-01-01' is indexable
- JSONB extraction: CREATE INDEX ON applications((payload->>'source')) — allows WHERE payload->>'source' = 'google_ads' to use an index
- Computed column: CREATE INDEX ON students((first_name || ' ' || last_name)) for full-name searches
- UUID casting: CREATE INDEX ON applications((id::text)) if queries cast the UUID to text

Combined: partial + expression:

```
CREATE INDEX idx_pending_source ON applications((payload->>'source')) WHERE status='pending';
```

This is a tiny, fast index for the common "find pending Google Ads applications" query.

Gotcha: Expression indexes are maintained on every write — complex expressions add write overhead. Profile the index creation time and write amplification before adding them to hot write paths.

IMPLEMENTATION CHALLENGE:

For Deepti AI's applications table (100M rows: 98% completed, 2% active):

1. Create a partial index that makes this query use an Index Only Scan: SELECT id, created_at FROM applications WHERE status IN ('pending','under_review') AND university_id=\$1 ORDER BY created_at DESC LIMIT 20.
2. Create an expression index to make this query fast: WHERE LOWER(student_email) = LOWER(\$1).
3. Create a JSONB expression index for: WHERE payload->>'utm_source' = 'google' AND status='pending'.
4. Use EXPLAIN ANALYZE to confirm each index is used.
5. Measure index sizes with: SELECT pg_size_pretty(pg_relation_size(indexrelid)) FROM pg_stat_user_indexes WHERE relname='applications'.
6. Add the combined partial+expression index from the JSONB case. Compare index size to a non-partial equivalent.

FOLLOW-UP PROBES:

- When PostgreSQL says 'Index Scan using idx_foo' in EXPLAIN ANALYZE, how do you determine if it's actually a partial index being used?
- Your expression index on LOWER(email) is defined but EXPLAIN ANALYZE shows a Seq Scan for WHERE LOWER(email)=\$1. What are all the possible reasons?

- What is an immutable function and why does PostgreSQL require the function used in an expression index to be immutable?
- You have a partial index WHERE status='pending'. Business logic changes and 'pending' is renamed to 'queued'. What happens to your index and what's the migration strategy?

[A] ADVANCED Category 07 > PostgreSQL > Multi-tenant Patterns

Q: What are the three main multi-tenancy models in PostgreSQL? Compare their isolation, performance, and operational complexity.

ANSWER:

Multi-tenancy in PostgreSQL can be implemented at three levels of isolation:

1. Separate databases per tenant:

Each university gets its own PostgreSQL database (or even separate instance).

- Isolation: Maximum. Complete schema and data isolation. Easy to drop a tenant.
- Performance: No cross-tenant interference. Each tenant can be tuned independently.
- Downside: Connection overhead (PgBouncer needed per database). Schema migrations must be applied to each database. Hard to run cross-tenant analytics. Not viable beyond a few hundred tenants.
- Use when: Strict regulatory isolation required (HIPAA, SOC2 per-customer reports), very large tenants (enterprise).

2. Separate schemas per tenant (schema-based multi-tenancy):

All tenants share one database. Each tenant has its own PostgreSQL schema (namespace):

university_42.applications, university_43.applications.

- Isolation: Good. Schema-level separation. search_path controls which schema is active for a session.
- Performance: Shared connection pool, shared buffer pool. Cross-schema queries possible (for global reporting).
- Downside: Schema migrations still require applying to each schema (Flyway/Liquibase with per-schema scripts). Can have 1000+ schemas — pg_class bloat, slow INFORMATION_SCHEMA queries. Harder to move a tenant.
- Use when: Mid-scale SaaS (< 10,000 tenants), where schema-level isolation is an explicit product feature.

3. Shared schema with tenant_id column (row-level multi-tenancy):

All tenants share the same tables, distinguished by a tenant_id (university_id) column.

- Isolation: Weakest. Cross-tenant data leakage possible via bugs. Mitigated by Row Level Security (RLS).
- Performance: Best resource utilization. One connection pool, one buffer pool, one set of indexes.
- Schema migrations: Applied once.
- Downside: RLS adds a performance overhead (especially on complex queries). tenant_id must be in every index (composite indexes). Large tenants can cause hotspots.
- Use when: High tenant count (10,000+), small-to-medium tenant sizes, SaaS at scale (Deepta AI's model).

Row Level Security (RLS): PostgreSQL's mechanism for enforcing row-level access control at the database level:

```
CREATE POLICY tenant_isolation ON applications USING (university_id =
current_setting('app.current_university_id')::uuid);
```

```
ALTER TABLE applications ENABLE ROW LEVEL SECURITY;
```

The application sets current_setting at the start of each request. Bypassed by superuser (use SET ROLE to a non-superuser role for tenant queries).

Deepta AI's platform: Shared schema with university_id (row-level model) is the right choice for a platform serving thousands of universities. Implement RLS for defense-in-depth and ensure every query plan includes university_id in the leading index column for partition pruning / index selectivity.

IMPLEMENTATION CHALLENGE:

Implement the shared-schema row-level multi-tenancy pattern for Deepta AI:

1. Create the applications table with `university_id` as the first column of all composite indexes.
2. Enable RLS: write a policy that restricts each database role (one role per tier: `api_user`, `reporting_user`, `admin`) to only see their university's rows.
3. In Go, set the current tenant at the start of each request: `conn.Exec(ctx, "SET app.current_university_id = $1", universityID)`
4. Handle the PgBouncer transaction mode issue: SET is session-scoped but transaction mode returns connections to the pool. Use SET LOCAL (transaction-scoped) instead.
5. Write a cross-tenant aggregation query that bypasses RLS (using a SECURITY DEFINER function) for the admin dashboard.
6. Benchmark: Compare query time for applications where `university_id` is the leading index column vs trailing column, on a 10M row table with 1000 universities.

FOLLOW-UP PROBES:

- How does PostgreSQL Row Level Security interact with connection pooling (PgBouncer transaction mode)? What is the correct pattern for setting the tenant context?
- What is a SECURITY DEFINER function and why is it dangerous in a multi-tenant context? How do you use it safely?
- Your largest university tenant has 10x more data than average, causing slow queries for their users. How do you handle a 'large tenant' problem in a shared-schema model without disrupting other tenants?
- You need to comply with GDPR: 'delete all data for university X.' In a shared-schema model, write the deletion procedure ensuring completeness across all related tables.

[P] PRINCIPAL Category 07 > PostgreSQL > Principal Design

Q: Design a PostgreSQL-backed system for Deepta AI that handles 50,000 concurrent student application submissions during peak university deadline hours. The system must be ACID-compliant, support multi-tenancy, and survive a primary database failure with <30s RTO.

ANSWER:

This is a system design problem requiring you to combine every database engineering concept discussed. Here is a reference architecture:

WRITE PATH DESIGN:

1. Application submission is a complex operation:

- Validate student identity (idempotency check)
- Insert application record
- Insert document metadata records
- Trigger lead source attribution
- Send Kafka event for async processing

This must be atomic.

2. Write amplification mitigation:

- Use a Kafka queue to absorb the spike. Student submits → ACK immediately → Kafka consumer writes to PostgreSQL.
- This decouples the 50k/s burst from PostgreSQL's write capacity (~5k TPS on commodity hardware).
- Use idempotent producer + Kafka consumer with manual offset commit after successful DB write.

3. PostgreSQL write optimizations:

- Turn off unnecessary triggers during bulk write
- Batch inserts: COPY instead of individual INSERTs (100x throughput improvement)
- Disable autovacuum during the bulk write window, run manually after

- Use `synchronous_commit=local` (flush to primary WAL, don't wait for standby) during burst — trades some durability for throughput

MULTI-TENANCY:

- Shared schema with `university_id` as shard key
- RLS enabled for application-level queries
- Each university's data is co-located; no cross-partition joins for common queries
- Hash-partition applications table on `university_id` (8 partitions) for parallel autovacuum

HA ARCHITECTURE (<30s RTO):

- Patroni on GKE: 1 primary + 1 synchronous standby (different GCP zone)
- PgBouncer with pause/resume on failover
- etcd (3-node) as DCS for Patroni leader election
- Application uses DNS-based connection string; Patroni updates DNS on failover
- Synchronous replication: `synchronous_commit=remote_write` → at most one in-flight WAL record lost

TUNING FOR 50K CONCURRENT:

- Connection: PgBouncer transaction mode, 50 server connections, 10,000 client connections
- `shared_buffers=25%` of RAM (e.g., 64GB RAM → 16GB `shared_buffers`)
- `effective_cache_size=75%` RAM
- `work_mem=4MB` (low — 50k concurrent transactions × 4MB = 200GB if all do sorts)
- `max_connections=200` (all go through PgBouncer)
- `wal_compression=on` — reduces WAL write size by 40-60%
- Partitioning: Range-partition `application_events` by month for efficient purging

MONITORING CRITICAL METRICS:

- `pg_stat_statements`: top queries by total_time
- `pg_stat_replication.replay_lag`: standby lag
- `pg_stat_bgwriter.buffers_checkpoint`: if high, increase `shared_buffers` or `checkpoint_completion_target`
- `n_dead_tup` in `pg_stat_user_tables`: autovacuum health
- PgBouncer SHOW POOLS: client wait time, server idle count

IMPLEMENTATION CHALLENGE:

Design the full infrastructure-as-code deployment for this system:

1. Write Terraform for GCP: 2 GKE node pools (primary DB nodes: n2-standard-32 with local SSD, application nodes: n2-standard-8), GCP Cloud Armor for DDoS protection.
2. Write Kubernetes manifests: Patroni StatefulSet (3 replicas), PgBouncer Deployment (2 replicas), etcd StatefulSet (3 replicas).
3. Write the Patroni configuration YAML that sets: `synchronous_commit=remote_write`, `wal_level=logical`, `shared_buffers=16GB`, `checkpoint_completion_target=0.9`.
4. Implement the application-side Kafka consumer in Go: reads `application_submitted` events, batches 100 events, COPY-inserts to PostgreSQL within a transaction, commits Kafka offset.
5. Write a load test using k6: 50,000 virtual users simultaneously submitting applications. Target: <500ms p99 latency for the API response (Kafka ACK), <5s p99 latency for the DB write confirmation via polling.
6. Define SLOs: uptime 99.95%, RTO <30s, RPO <5s (synchronous standby).

FOLLOW-UP PROBES:

- During the 50k peak, autovacuum kicks in on the applications table and causes query latency spikes. What is the production-safe way to handle this conflict?
- Your synchronous standby is in a different GCP zone with 5ms network latency. What is the added per-commit latency with `synchronous_commit=remote_write` vs `remote_apply` vs on?
- The Patroni leader election takes 18 seconds but your SLA is 30 seconds RTO. A bug in the application takes 25 seconds to reconnect after PgBouncer reports the new primary. You're cutting it close — what do you change?

- Regulatory requirement: all student PII must be encrypted at rest AND queryable (for exact-match searches on name and email). How do you implement searchable encryption in PostgreSQL without breaking index performance?

[P] PRINCIPAL Category 07 > Databases > Distributed Design

Q: You are building the data layer for Vigilant, Deepta AI's multi-agent AI investment copilot, which needs to ingest 10M financial events/day, support sub-100ms queries from 6 specialized agents, and maintain a 3-year history. Design the complete data architecture.

ANSWER:

This requires a polyglot persistence architecture — no single database serves all access patterns optimally.

ACCESS PATTERN ANALYSIS:

The 6 Vigilant agents have different patterns:

- Data Ingestion Agent: Write-heavy, 10M events/day (~115 events/second average, bursts to 1,000+/s during market hours). Needs fast idempotent upserts. Low-latency guarantee not critical (background).
- Fundamental Analysis Agent: Reads structured time-series (prices, ratios, filings). Complex aggregations, date-range queries. Low cardinality (thousands of companies).
- Event Impact Agent: Reads recent events (last 30 days) by company, needs <20ms read latency for recent data.
- Portfolio Decision Agent: Reads current portfolio state, needs ACID consistency (no stale portfolio positions).
- Opportunity Discovery Agent: Full-text search on news/filings, vector similarity search (semantic similarity of financial events).
- Guardrail Agent: Reads compliance rules, rarely changes, needs extreme read availability.

POLYGLOT DATA STORES:

1. PostgreSQL (TimescaleDB extension) — Primary store for structured time-series:
 - Stock prices, ratios, earnings data as hypertables (automatic time-based partitioning)
 - Portfolio positions (ACID, small dataset)
 - Compliance rules (read-heavy, high consistency)
 - TimescaleDB continuous aggregates for pre-computed OHLCV at hourly/daily intervals
 - Compression policies for data > 1 month old (10x storage reduction)
2. MongoDB — Semi-structured event store:
 - Raw financial events (filings, earnings transcripts, news) with variable schemas
 - TTL index for auto-deletion after 3 years
 - Change streams feeding the Kafka pipeline
 - Aggregation pipeline for impact event queries
3. Redis — Hot data cache:
 - Current stock prices (update every second during market hours)
 - Portfolio state cache (invalidated on trade execution)
 - Agent results cache with 5-minute TTL
 - Sorted set for "top opportunities by score" — $O(\log N)$ updates, $O(1)$ retrieval
4. Qdrant/Pinecone (vector database) — Semantic search:
 - Embeddings of financial events for similarity search ("find events similar to 2008 Lehman failure")
 - Opportunity Discovery Agent queries this for pattern matching
5. Kafka — Event backbone:
 - All events flow through Kafka before being written to their respective stores
 - Topics: market.prices, financial.events, portfolio.changes, alerts.signals
 - Retention: 7 days (downstream stores handle long-term)

WRITE PATH:

Market data API → Kafka (market.prices) → Consumer batch-writes to TimescaleDB hypertable + updates Redis sorted set

READ PATH for Event Impact Agent (needs <20ms):

1. Check Redis cache (5ms if hit)
2. Miss → query MongoDB recent events index (10ms)
3. Store result in Redis with 60s TTL
4. Return to agent

CONSISTENCY MODEL:

- Portfolio positions: Strong consistency (PostgreSQL SERIALIZABLE, single writer)
- Market prices: Eventual consistency (Redis may be 1s stale — acceptable for decision making)
- Historical events: No strict consistency needed (append-only, immutable past)

IMPLEMENTATION CHALLENGE:

Implement the core data pipeline for Vigilant:

1. Kafka consumer in Go: Reads from market.prices topic, batches 1,000 prices, TimescaleDB COPY insert, then MSET all prices to Redis with 60s TTL. Measure end-to-end latency from Kafka message to Redis availability.
2. TimescaleDB hypertable: CREATE TABLE stock_prices (time TIMESTAMPTZ, ticker TEXT, open NUMERIC, high NUMERIC, low NUMERIC, close NUMERIC, volume BIGINT) with chunk_time_interval='1 day' and compression after 7 days.
3. Continuous aggregate: CREATE MATERIALIZED VIEW daily_ohlcv WITH (timescaledb.continuous) AS SELECT time_bucket('1 day', time), ticker, first(open,time), max(high), min(low), last(close,time), sum(volume) FROM stock_prices GROUP BY 1, 2.
4. Portfolio Decision Agent's ACID transaction: BEGIN SERIALIZABLE; check current position; calculate new position; INSERT into trades; UPDATE portfolio_positions; COMMIT. Handle serialization failures with retry.
5. Opportunity Discovery Agent: embedding pipeline — for each new financial event, call OpenAI embedding API, store vector in Qdrant with metadata (ticker, event_type, timestamp), query for top-5 similar historical events.

FOLLOW-UP PROBES:

- TimescaleDB's continuous aggregates have a 'refresh lag' — they may be slightly stale. For the Portfolio Decision Agent which needs real-time price data, what is your strategy?
- The Opportunity Discovery Agent's vector similarity search returns results in <100ms for 10M embeddings. At 100M embeddings (3-year history), it degrades to 800ms. What are your scaling strategies for the vector index?
- You need to provide point-in-time portfolio reconstruction ('what was the portfolio exactly at 14:35:22 on date X?'). How does your data architecture support this without full event replay?
- Regulatory audit requires you to prove the exact data state that the Portfolio Decision Agent had when it made a buy recommendation. How do you implement immutable audit trails across your polyglot persistence architecture?

CATEGORY 08

CACHING & REDIS

Data structures · Persistence · Cluster · Distributed Locking · Cache Patterns

15 questions · [E]=6 [M]=6 [A]=2 [P]=1

TOPICS COVERED IN THIS CATEGORY:

- Redis data structures internals: String (SDS), List (quicklist), Hash, Set, Sorted Set (skiplist+hashtable), HyperLogLog, Bloom Filter, Stream
- Redis persistence: RDB snapshots, AOF, RDB+AOF hybrid
- Eviction policies: LRU, LFU, allkeys vs volatile
- Redis Cluster: hash slots, gossip protocol, MOVED/ASK redirects, resharding
- Redis Sentinel: leader election, failover
- Distributed locking: Redlock algorithm and its controversies (Kleppmann critique)
- Lua scripting for atomicity: EVAL, EVALSHA, Redis Functions
- Pub/Sub vs Streams: when to use each vs Kafka
- Redis as a session store: JWT + Redis refresh token rotation
- Caching patterns: cache-aside, write-through, write-behind, read-through
- Cache stampede (thundering herd) and prevention mechanisms
- TTL strategies, hot/cold tiering, cache invalidation (event-driven, CDC-based)
- HyperLogLog and Bloom Filter internals and use cases
- Principal: Redis-backed distributed rate limiter at 1M req/s

[E] EASY Category 08 > Redis > Data Structures

Q: Name the core Redis data structures and the primary use case for each. Why does Redis outperform a traditional database for these use cases?

ANSWER:

Redis is an in-memory data structure store. Its speed comes from: (1) all data in RAM, (2) single-threaded command processing (no lock contention), (3) non-blocking I/O via an event loop, and (4) simple data structures with $O(1)$ or $O(\log N)$ operations.

Core data structures:

String: The simplest type. Stores any binary-safe value up to 512MB. Internally uses SDS (Simple Dynamic String). Operations: GET, SET, INCR, INCRBY, APPEND, GETRANGE. Use cases: counters, cached JSON, feature flags, rate limiter counters. At Deepta AI: cache student profile JSON strings with TTL.

List: A doubly-linked list (or ziplist for small lists). $O(1)$ push/pop from both ends. Operations: LPUSH, RPUSH, LPOP, RPOP, LRANGE, LLEN. Use cases: message queues (LPUSH + BRPOP), activity feeds, recent history. Limitation: accessing middle elements is $O(N)$.

Hash: A map of field-value pairs within a key. Like a mini Redis inside a key. Operations: HSET, HGET, HMSET, HMGET, HGETALL, HINCRBY. Use cases: storing an object's fields individually (user profile: HSET user:123 name "Gautham" email "g@deepta.ai"), session data. More memory-efficient than storing multiple String keys for related fields.

Set: An unordered collection of unique strings. Operations: SADD, SREM, SISMEMBER, SMEMBERS, SINTER, SUNION, SDIFF. Use cases: unique visitors, tagging, intersection (users who liked both A and B). At Deepta AI: SADD applied_universities:<student_id> <university_id> to track applications without duplicates.

Sorted Set (ZSet): Like Set but each member has a score. Members ordered by score. Internally a skiplist + hashtable. Operations: ZADD, ZRANGE, ZREVRANGE, ZRANGEBYSCORE, ZRANK, ZINCRBY. $O(\log N)$ for most operations. Use cases: leaderboards, priority queues, rate limiting (sliding window), time-series events. At Deepta AI: rank universities by application volume for dashboard display.

HyperLogLog: Probabilistic data structure for cardinality estimation. Uses ~12KB RAM regardless of dataset size. ~0.81% error rate. Operations: PFADD, PFCOUNT. Use case: count unique website visitors without storing each visitor ID.

Bloom Filter (RedisBloom module): Probabilistic set membership. False positives possible, false negatives impossible. Use case: check if a student has already been sent an email (avoid duplicate sends) without storing all student IDs.

Stream: Append-only log with consumer groups. Similar to Kafka. Operations: XADD, XREAD, XREADGROUP, XACK. Use case: lightweight event streaming within Redis, replace Kafka for simpler workflows.

IMPLEMENTATION CHALLENGE:

For Deepta AI's student platform, implement the following Redis data structure patterns in Go using the go-redis client:

1. String: Cache a university's application count (JSON blob) with 5-minute TTL. Handle cache miss by querying PostgreSQL.
2. Hash: Store a student session as a Hash (session_id → {user_id, university_id, role, created_at}). Set TTL on the hash key.
3. Sorted Set: Maintain a leaderboard of universities by application count. On each new application, ZINCRBY the university's score. Query top 10 universities (ZREVRANGE with scores).
4. Set: Track which notification types have been sent to a student (to avoid duplicates): SADD sent_notifications:<student_id> "acceptance_email". Check with SISMEMBER before sending.
5. HyperLogLog: Count unique student visitors to each university's application portal page. PFADD and PFCOUNT — compare the cardinality estimate vs the actual COUNT DISTINCT from PostgreSQL.

FOLLOW-UP PROBES:

- What is the internal encoding Redis uses for small Hashes vs large Hashes? What thresholds trigger the switch from ziplist to hashtable?
- Why is Redis single-threaded for commands yet still handles 100k+ requests/second? What does 'single-threaded' actually mean in Redis 6+ which introduced I/O threads?
- When would you use a Redis List over a Redis Stream for a queue? What does Stream offer that List doesn't?
- You use SMEMBERS to get all members of a Set with 10 million entries. What problem does this cause and what is the correct alternative?

[E] EASY Category 08 > Redis > Persistence

Q: Explain Redis's persistence options: RDB, AOF, and hybrid mode. What are the durability and performance tradeoffs?

ANSWER:

Redis offers three persistence modes:

RDB (Redis Database Snapshot): Redis forks a child process and writes a point-in-time snapshot of the entire dataset to a .rdb file on disk. The fork uses copy-on-write (CoW) — the parent continues serving requests, the child writes the snapshot without holding locks.

Configuration: save 900 1 (save if 1 key changed in 900s), save 300 10, save 60 10000

- Pros: Compact binary file. Fast restart (load RDB much faster than replaying AOF). Minimal I/O during normal operation. Good for disaster recovery snapshots.
- Cons: Data loss between snapshots. If Redis crashes 30 minutes after the last RDB, you lose 30 minutes of data. Fork can cause latency spikes on large datasets (CoW memory pressure).

AOF (Append-Only File): Redis logs every write command to an AOF file. On restart, Redis replays the AOF to reconstruct state.

fsync policies:

- appendfsync always: fsync after every write command. Maximum durability, slowest (disk I/O per write).

- `appendfsync everysec` (recommended): `fsync` once per second. At most 1 second of data loss. Good balance.
- `appendfsync no`: OS decides when to flush. Fastest, highest data loss risk.

AOF rewrite: AOF grows indefinitely. Redis periodically rewrites it (`BGREWRITEAOF`) — the rewrite creates a new minimal AOF representing current state (e.g., 1000 `INCR` commands → one `SET`).

- Pros: Much lower data loss (1-second window). Human-readable. Can edit to recover from mistakes.
- Cons: Larger file size. Slower restart (must replay all commands). More disk I/O.

RDB+AOF Hybrid (recommended for production): Redis 4.0+. The AOF file starts with an RDB snapshot (for fast load) followed by AOF commands since the snapshot. Best of both: fast restart + minimal data loss.

`appendonly yes + aof-use-rdb-preamble yes`

In Deepta AI's Redis setup: For session management and caching, RDB alone may be acceptable (cache is rebuildable). For distributed locks or Kafka offset tracking in Redis, use AOF `everysec` or hybrid to minimize data loss.

IMPLEMENTATION CHALLENGE:

Configure and test Redis persistence scenarios for Deepta AI:

1. Configure Redis with RDB+AOF hybrid. Start Redis, populate 100,000 session keys.
2. Simulate a crash: `redis-cli DEBUG SLEEP 0` followed by `kill -9` the Redis process.
3. Measure restart time: `time redis-server redis.conf` — compare RDB only vs AOF only vs hybrid.
4. Measure data loss: check how many session keys survived after restart.
5. Implement a Go function that detects Redis reconnection after restart and warms up the cache from PostgreSQL for the top 1000 most-active universities.
6. Configure a Cloud Memorystore (GCP managed Redis) equivalent — what persistence options does it offer vs self-managed Redis?

FOLLOW-UP PROBES:

- What is copy-on-write (CoW) during RDB fork and how can it cause memory usage to double during snapshotting?
- Your Redis instance has 32GB of data. RDB fork takes 2 seconds to complete. During those 2 seconds, what is the worst-case memory overhead?
- What is AOF rewrite and what triggers it? What happens to write commands during the rewrite?
- In Deepta AI's GCP deployment using Cloud Memorystore, which persistence mode is available and what are the limitations vs self-managed Redis?

[E] EASY Category 08 > Redis > Eviction Policies

Q: Explain Redis's eviction policies. When does eviction trigger and how do you choose the right policy?

ANSWER:

Redis eviction triggers when `maxmemory` is set and Redis hits that limit. Redis must either reject new writes or evict existing keys to free memory.

Eviction policies:

`noeviction` (default): Refuse write commands when memory is full. Returns OOM error to client. Good when Redis is a primary data store (you don't want silent data loss). Bad for a cache (you want automatic eviction).

`allkeys-lru`: Evict any key using LRU (Least Recently Used) approximation. Redis doesn't track true LRU — it samples a configurable number of keys (`maxmemory-samples`, default 5) and evicts the least recently used among the sample. Good general-purpose cache policy.

`volatile-lru`: Evict only keys with a TTL set, using LRU. Keys without TTL are never evicted. Useful when you have a mix of persistent data (no TTL) and cached data (with TTL).

`allkeys-lfu`: Evict any key using LFU (Least Frequently Used). Tracks access frequency (not just recency). Better than LRU for skewed access patterns where some keys are accessed very rarely but recently.

`volatile-lfu`: LFU but only keys with TTL.

allkeys-random: Randomly evict any key. Simple, no tracking overhead. Only sensible if all keys have equal access probability (rare).

volatile-random: Randomly evict keys with TTL.

volatile-ttl: Evict keys with the shortest TTL first — the ones about to expire anyway.

Choosing a policy for Deepta AI:

- Pure cache (all data is rebuildable from PostgreSQL): allkeys-lru or allkeys-lfu. LFU better if university application data has hotspots (top 100 universities accessed 1000x more than others).
- Mixed (some persistent keys, some cached): volatile-lru. Set TTL on cached items, leave session tokens without TTL.
- Deepta AI recommendation: Use allkeys-lfu with maxmemory-samples 10 for better approximation quality.

Redis 4.0+ LFU tuning: lfu-log-factor (higher = slower counter growth), lfu-decay-time (minutes before frequency counter halves). Default values work well for most use cases.

IMPLEMENTATION CHALLENGE:

Set up Redis memory pressure testing for Deepta AI's caching layer:

1. Set maxmemory=256mb, maxmemory-policy=allkeys-lfu on a test Redis instance.
2. Write a Go script that: (a) fills Redis with 500,000 university application cache entries (varying access frequencies — 10% of keys accessed 100x more), (b) continues inserting new entries until eviction kicks in, (c) monitors which keys are evicted (use keyspace notifications: config set notify-keyspace-events Ex).
3. Compare hit rate under memory pressure between allkeys-lru and allkeys-lfu for the skewed access pattern.
4. Implement a Redis memory dashboard: INFO memory output parsed into Prometheus metrics (used_memory, maxmemory, mem_fragmentation_ratio).
5. Set up a memory fragmentation alert: if mem_fragmentation_ratio > 1.5, alert and trigger MEMORY PURGE.

FOLLOW-UP PROBES:

- What is memory fragmentation in Redis and what causes mem_fragmentation_ratio > 1.5? How does MEMORY PURGE help?
- LRU and LFU are approximations in Redis. How does the maxmemory-samples parameter affect accuracy vs CPU overhead?
- What happens to Redis Streams or Sorted Sets when a key is partially evicted — can Redis evict part of a key?
- You set maxmemory-policy=volatile-lru but your Redis memory keeps growing. What is the most likely cause?

[E] EASY Category 08 > Redis > Caching Patterns

Q: Explain the four main caching patterns: cache-aside, read-through, write-through, and write-behind. When does each apply?

ANSWER:

Caching patterns define how the application, cache, and database interact:

Cache-Aside (Lazy Loading): Application code manages the cache.

Read: Check cache → miss → read from DB → write to cache → return.

Write: Write to DB → invalidate (or update) cache.

- Pros: Cache only stores what's actually requested. Tolerant of cache failure (fall back to DB). Application has full control.
- Cons: Cache miss is expensive (cold start, thundering herd risk). Data in cache can be stale if invalidation is missed. Developer must implement caching logic everywhere.
- Use case: Most common pattern. Deepta AI student profile caching, university metadata caching.

Read-Through: The cache itself loads from DB on a miss. Application only talks to the cache.

- Pros: Application code is simpler. Cache always has the authoritative read path.

- Cons: First request for any key always misses (cold start). Cache layer must have DB access.
- Use case: CDN caching is a natural read-through. Redis with a sidecar cache layer.

Write-Through: On every write, the application writes to BOTH the cache and DB synchronously.

- Pros: Cache is always in sync with DB. No stale reads after writes.
- Cons: Every write hits both DB and cache — double write latency on write path. Cache may store data that's never read again (waste).
- Use case: Read-heavy data that must be fresh after writes. Leaderboards with frequent reads.

Write-Behind (Write-Back): Application writes to cache only. Cache asynchronously flushes to DB.

- Pros: Write latency is very low (cache speed). Batching writes to DB reduces DB load.
- Cons: Data loss risk if cache fails before flush. Complexity in handling cache failure. DB is temporarily inconsistent.
- Use case: Counters (page view counts, analytics events) where approximate accuracy is acceptable and write throughput is critical.

In Deepti AI:

- Cache-aside: Student profile, university metadata, counselor data (reads >> writes)
- Write-through: Application status updates (students check status frequently immediately after update)
- Write-behind: Page view counters for university profile analytics (high write volume, eventual consistency acceptable)

IMPLEMENTATION CHALLENGE:

Implement all four patterns in Go for Deepti AI's application status system:

1. Cache-aside: Student calls GET /application/:id/status. Implement the miss → DB → cache → return flow using go-redis. Include error handling: if Redis is unavailable, fall back to DB without error propagation.
2. Write-through: When counselor updates application status (PATCH /application/:id/status), write to PostgreSQL and Redis atomically. Use Redis transactions (MULTI/EXEC) to update both the status string and a sorted set of recent status changes.
3. Write-behind: Count application views: when student views their application, INCR view_counter:<app_id> in Redis. A background goroutine flushes counters to PostgreSQL every 60 seconds using GETDEL to atomically read and reset.
4. Implement cache stampede protection for the cache-aside pattern: use a Redis lock (SET NX EX) to ensure only one goroutine fetches from DB on a miss, others wait and retry.

FOLLOW-UP PROBES:

- Cache-aside has a race condition: two requests both miss, both read from DB, both write to cache — one overwrites the other with potentially stale data. How do you prevent this?
- Write-through doubles write latency. How do you implement write-through asynchronously to avoid this, and what consistency guarantee does that provide?
- What is the 'thundering herd' problem specific to caching and how does the 'probabilistic early expiration' technique prevent it?
- In Deepti AI's Kafka pipeline, events are processed asynchronously. How does write-behind caching interact with Kafka consumer-driven DB writes — which is the source of truth?

[E] EASY Category 08 > Redis > Session Store

Q: How do you implement a distributed session store in Redis? What are the design decisions for token structure, TTL, and invalidation?

ANSWER:

A distributed session store allows any application server to validate any session — essential in a horizontally-scaled microservices architecture like Deepti AI's GKE deployment.

Session storage approaches in Redis:

Option 1: Opaque session token → Redis Hash

Generate a random session_id (UUID or crypto-random string). Store session data as a Redis Hash:

HSET session:<session_id> user_id "123" university_id "456" role "admin" created_at "1735000000"

EXPIRE session:<session_id> 3600

Application receives session_id as a cookie or Authorization header. On each request, HGETALL session:<session_id>. If nil, session expired or invalid.

Pros: Instant revocation (DEL the key), variable session data, server-side state.

Cons: Redis round-trip on every request.

Option 2: JWT with Redis blacklist

Use stateless JWTs (no Redis for valid tokens), but maintain a Redis set of revoked token IDs.

On each request: verify JWT signature locally (no Redis), then SISMEMBER revoked_tokens <jti> to check if revoked.

Pros: Reduced Redis load for valid tokens. JWT carries claims (no DB lookup).

Cons: Revocation check still hits Redis. Must manage the blacklist TTL (can't exceed JWT expiry).

Token structure for Deepta AI (refresh/access pattern from your experience):

Access token: Short-lived JWT (15 minutes). Validated locally.

Refresh token: Opaque random string, stored in Redis: SET refresh:<token> <user_id> EX 2592000 (30 days).

On refresh: validate refresh token in Redis, issue new access token, rotate refresh token (delete old, create new) — prevents refresh token reuse.

TTL strategy:

- Session TTL = 24h for web sessions, 30d for "remember me"
- Use EXPIRE, not EXPIREAT — relative TTL avoids clock skew issues
- Sliding expiration: EXPIRE session:<id> 3600 on each request (reset TTL on activity)

Invalidation:

- Single session: DEL session:<session_id>
- All sessions for a user: maintain a secondary index SET user_sessions:<user_id> → set of session_ids. On logout-all: SMEMBERS → DEL each session key → DEL the set.
- Bulk invalidation (security incident): Redis SCAN + DEL pattern (avoid KEYS * in production — blocks).

IMPLEMENTATION CHALLENGE:

Implement Deepta AI's JWT + Redis session system in Go:

1. On login: generate access JWT (15min, contains user_id, university_id, role), generate refresh token (crypto/rand 32 bytes, hex-encoded), store: SET refresh:<token> <user_id_json> EX 2592000. Return both tokens.
2. Auth middleware: verify JWT signature (no Redis), extract claims, attach to context.
3. Refresh endpoint: validate refresh token in Redis with GET, issue new access JWT, rotate refresh token (atomic with MULTI/EXEC: DEL old, SET new), return new pair.
4. Logout: add access token JTI to Redis blacklist with TTL = remaining JWT lifetime. DEL refresh token.
5. Logout-all-devices: SMEMBERS user_sessions:<user_id>, DEL all session keys, SMEMBERS user_refresh_tokens:<user_id>, DEL all refresh tokens.
6. Admin force-invalidation: implement the SCAN-based bulk deletion for a compromised user account.

FOLLOW-UP PROBES:

- Your Redis instance goes down. With opaque sessions in Redis, all users are logged out. With JWT + Redis blacklist, users stay logged in but revocation doesn't work. Which failure mode is worse for Deepta AI?
- What is the security implication of storing sensitive user data in a Redis Hash vs a signed JWT? What if Redis is compromised?
- How does sliding TTL (resetting EXPIRE on every request) create a denial-of-service vector and how do you limit it?
- Deepta AI needs to support 'last active device wins' — logging in on a new device invalidates all previous sessions. How do you implement this with your Redis session design?

[E] EASY Category 08 > Redis > Pub/Sub vs Streams

Q: Compare Redis Pub/Sub and Redis Streams. When would you use each vs Kafka?

ANSWER:

Redis Pub/Sub: A fire-and-forget messaging mechanism. Publishers send messages to channels; subscribers receive them in real time. No persistence — if a subscriber is offline when a message is published, it misses it forever.

PUBLISH channel message → SUBSCRIBE channel (receives in real time)

PSUBSCRIBE pattern* — subscribe to channels matching a glob pattern.

Characteristics:

- At-most-once delivery (fire and forget)
- No message persistence
- No consumer groups, no acknowledgment, no replay
- Very low latency (<1ms) for connected subscribers
- Great for: real-time notifications, live dashboards, broadcasting events that are only relevant now (e.g., "university X's portal is online")

Redis Streams (XADD, XREAD, XREADGROUP):

A persistent, append-only log. Messages are stored in the stream and can be read by multiple consumer groups independently. Each message has a unique auto-generated ID (timestamp-sequence).

Consumer groups:

XREADGROUP GROUP grp1 consumer1 COUNT 10 BLOCK 5000 STREAMS mystream >

- > means "only undelivered messages"
- Multiple consumers in a group share messages (like Kafka consumer groups)
- XACK acknowledges processing — unacknowledged messages can be reclaimed (XPENDING/XCLAIM)

Streams vs Pub/Sub:

- Streams: persistence, consumer groups, ACK, replay — Kafka-like semantics in Redis
- Pub/Sub: ephemeral, real-time broadcast — WebSocket notification pattern

Redis vs Kafka:

- Use Redis Streams when: low message volume (<100k/s), simple consumer groups, already have Redis, don't need multi-datacenter replication
- Use Kafka when: high throughput (1M+/s), long retention (days to forever), complex consumer topologies, multi-datacenter, exactly-once semantics, schema registry

In Deepta AI: Kafka for the core application event pipeline (high volume, needs replay). Redis Pub/Sub for real-time UI notifications to counselors (ephemeral, low volume, no persistence needed). Redis Streams for in-service lightweight queues between Go microservices.

IMPLEMENTATION CHALLENGE:

For Deepta AI, implement a real-time notification system using Redis Pub/Sub for counselor dashboard updates:

1. Publisher (Go): When a new application arrives, publish to channel applications:<university_id>: PUBLISH applications:42 '{"app_id":"...", "student_name":"Ravi", "type":"new_application"}'.

2. Subscriber (Go): Each counselor's WebSocket connection subscribes to their university's channel. Forward messages to the WebSocket client.

3. Handle subscriber reconnection: on reconnect, fetch last 20 applications from PostgreSQL to fill the gap (since Pub/Sub missed messages while disconnected).

4. Compare with Redis Streams implementation: XADD applications:42 * type new_application student_name Ravi. Use XREADGROUP to allow multiple counselors to process the same notification exactly once.

5. Discuss: for Deepta AI's counselor notification requirement, which is more appropriate — Pub/Sub or Stream? Justify.

FOLLOW-UP PROBES:

- What happens to unacknowledged messages in a Redis Stream consumer group? How does XPENDING and XCLAIM work for reclaiming them?

- Redis Pub/Sub has no backpressure — a slow subscriber causes Redis to buffer messages indefinitely. How does this cause memory issues and what is the mitigation?
- How does Redis Streams' consumer group model compare to Kafka's consumer groups in terms of partition equivalence and parallelism?
- Your Redis Pub/Sub notification system needs to fan out to 10,000 concurrent WebSocket connections. What happens at this scale and what is the architecture change?

[M] MID Category 08 > Redis > Sorted Set Internals

Q: Explain the internal implementation of Redis Sorted Sets (skiplist + hashtable). Why this design and what are the performance characteristics?

ANSWER:

Redis Sorted Sets use a dual data structure:

1. Skiplist (for ordered access): A probabilistic alternative to balanced BSTs. Each element in the skiplist is stored at multiple levels — the bottom level has all elements, higher levels are "express lanes" that skip over multiple elements.

Operations:

- Insert: $O(\log N)$ average — randomly assign level height, update forward pointers
- Delete: $O(\log N)$
- ZRANGE (range by rank): $O(\log N + M)$ where M is returned elements
- ZRANGEBYSCORE (range by score): $O(\log N + M)$

Why skiplist over BST: Simpler implementation, similar performance characteristics, and easier to do range operations (forward traversal after finding start node is $O(1)$ per step).

2. Hashtable (for $O(1)$ by-element lookups): Maps element → score. Allows ZSCORE and ZRANK in $O(1)$ and $O(\log N)$ respectively.

The combination gives:

- ZADD: $O(\log N)$ — update both skiplist (ordered) and hashtable (element → score)
- ZRANK: $O(\log N)$ — skiplist traversal
- ZSCORE: $O(1)$ — hashtable lookup
- ZRANGE: $O(\log N + M)$ — skiplist traversal with forward pointers
- ZRANGEBYSCORE: $O(\log N + M)$
- ZCARD: $O(1)$

Encoding optimization: Small sorted sets (<128 members and all members <64 bytes) use ziplist instead of skiplist+hashtable. Ziplist is a compact byte array encoding — 10-100x more memory efficient but $O(N)$ operations. Threshold: `zset-max-ziplist-entries=128`, `zset-max-ziplist-value=64`.

In Deepta AI's application: Using a Sorted Set for "recent applications per university" with `score=created_at timestamp`:

- `ZADD university:42:applications <timestamp> <app_id>` — $O(\log N)$
- `ZREVRANGE university:42:applications 0 9 WITHSCORES` — top 10 most recent in $O(\log N + 10)$
- `ZREMRANGEBYSCORE university:42:applications 0 <cutoff_timestamp>` — prune old entries

For the Vigilant copilot's stock screener: `ZADD stocks:by_pe_ratio <pe_ratio> <ticker>` allows "stocks with P/E between 10 and 20" via `ZRANGEBYSCORE` in $O(\log N + M)$.

IMPLEMENTATION CHALLENGE:

Implement a real-time application scoring system for Deepta AI using Redis Sorted Sets:

1. When a student submits an application, score it based on: academic strength (0-40), application completeness (0-30), lead source quality (0-20), timing bonus (0-10). `ZADD university:<id>:scored_apps <total_score> <app_id>`.

2. University counselors query top candidates: `ZREVRANGE university:<id>:scored_apps 0 19 WITHSCORES`.

3. Implement score updates: when a student uploads a missing document, recalculate completeness score component and `ZADD` to update (`ZADD` updates if element already exists).

4. Implement ZINTERSTORE: find applications that are both in university:42:scored_apps AND university:42:shortlisted (a plain Set). Return the intersection with scores preserved.
5. Benchmark: compare 1M ZADD operations vs 1M HSET operations in Redis — measure throughput and memory usage. When does the ziplist → skiplist encoding switch occur and how does it affect performance?

FOLLOW-UP PROBES:

- Redis uses a probabilistic level assignment for skiplist nodes. What is the expected height distribution and why is this important for worst-case performance?
- When does Redis switch a Sorted Set from ziplist to skiplist encoding? How does this affect memory usage and can you reverse the switch?
- ZRANGEBYSCORE can return millions of results. What happens to Redis's single-threaded event loop during this operation? How do you safely paginate large range queries?
- Implement a rate limiter using a Sorted Set (sliding window algorithm). Walk through the ZADD, ZREMRANGEBYSCORE, ZCARD sequence and explain atomicity requirements.

[M] MID Category 08 > Redis > Cluster & Sentinel

Q: Explain Redis Cluster architecture: hash slots, gossip protocol, MOVED/ASK redirects, and how you handle resharding.

ANSWER:

Redis Cluster distributes data across multiple nodes using hash slots:

Hash slots: Redis Cluster has exactly 16,384 hash slots (0-16383). Each key maps to a slot via: $\text{slot} = \text{CRC16}(\text{key}) \% 16384$. Each master node is responsible for a subset of slots.

Example: 3 masters — Node A: slots 0-5460, Node B: slots 5461-10922, Node C: slots 10923-16383.

Hash tags: To ensure related keys land on the same slot (enabling multi-key operations): use {braces} in key names. E.g., user:{123}:profile and user:{123}:sessions both hash on "123" → same slot → same node.

Without this, multi-key operations (MGET, MSET, transactions) fail across nodes.

Replication in Cluster: Each master has one or more replicas. Replicas are not shards — they serve as hot standbys. On master failure, the cluster elects one replica as the new master (requires $\geq N/2+1$ masters to agree).

Gossip protocol: Nodes exchange cluster state (which nodes are up/down, which slots they own) via a gossip/epidemic protocol. Every 100ms, each node sends PING to a random subset of peers. Failure detection: if a node doesn't respond to PING for cluster-node-timeout (default 15s), it's flagged PFAIL. When a majority of masters flag it PFAIL → FAIL.

MOVED redirect: Client sends MGET foo to Node A, but foo hashes to a slot owned by Node B. Node A responds: -MOVED 3999 127.0.0.1:6380. Client should update its slot map and redirect to Node B. Smart clients (JedisCluster, go-redis Cluster) handle this transparently.

ASK redirect: During resharding, a slot is being migrated from Node A to Node B. If the key hasn't moved yet, Node A returns -ASK <slot> <node_B_addr>. Client sends ASKING to Node B then re-sends the command. Unlike MOVED, ASK is temporary — client should NOT update its slot map permanently.

Resharding: Adding/removing nodes requires moving hash slots. Done with redis-cli --cluster reshard. Keys are migrated one by one (MIGRATE command). Live resharding: both MOVED and ASK coexist during migration. No downtime needed.

Limitations:

- Multi-key operations only work if all keys are in the same slot (use hash tags)
- No cross-slot transactions (MULTI/EXEC)
- Lua scripts must touch keys in the same slot
- SCAN works per-node, need to scan all nodes for cluster-wide scan

IMPLEMENTATION CHALLENGE:

Design the Redis Cluster setup for Deepta AI's caching layer:

1. 3-master + 3-replica cluster on GCP (6 instances). Each master in a different zone. Calculate which hash slots each master owns.

2. Write Go code using go-redis cluster client: ClusterClient with addrs of all 6 nodes. Verify it handles MOVED redirects transparently.
3. Implement hash tag strategy: All session data for university 42 uses key pattern {univ:42}:session:<id> to ensure co-location.
4. Simulate a master failure: stop Node B. Observe failover via cluster-node-timeout (15s). Measure application-side error duration.
5. Resharding exercise: add a 4th master. Use redis-cli --cluster reshard to move 4096 slots to the new node. Monitor ASK redirects in application logs during migration.
6. Implement a cluster-wide SCAN: scan all master nodes in parallel, merge results, deduplicate.

FOLLOW-UP PROBES:

- Why does Redis Cluster use exactly 16,384 hash slots? What is special about this number?
- A Redis Cluster has 3 masters. How many masters need to agree for a replica to be promoted to master? What happens if the cluster loses quorum?
- What is a 'hot slot' in Redis Cluster and how does it differ from a 'hot key'? How do you detect and fix each?
- Your Deepta AI application uses MULTI/EXEC (Redis transaction) across two keys in different slots. What error do you get and how do you redesign to work in a cluster?

[M] MID Category 08 > Redis > Distributed Locking

Q: Explain the Redlock algorithm. What are Martin Kleppmann's criticisms of it and when is it still appropriate to use?

ANSWER:

Distributed locking problem: In a multi-process system, you want only one process to execute a critical section at a time. A Redis lock is typically: SET lock:<resource> <random_value> NX EX <timeout>. NX = only set if not exists. EX = auto-expire.

Single-node Redis lock:

SET lock:process_applications UNIQUE_VALUE NX EX 30

- critical section --

if GET lock:process_applications == UNIQUE_VALUE then DEL lock:process_applications

The UNIQUE_VALUE ensures you only release YOUR lock (not one set by another process). The DEL should be done atomically with the GET check — use a Lua script:

if redis.call("GET", KEYS[1]) == ARGV[1] then return redis.call("DEL", KEYS[1]) else return 0 end

Problem with single-node: If the Redis instance fails, the lock is lost (if no persistence) or unavailable (if the instance is down). A second node might issue the same lock.

Redlock algorithm (antirez, Redis author): Use N independent Redis instances (not a cluster). To acquire a lock:

1. Get current time T1
2. Try to SET lock NX EX on each of N instances sequentially
3. Get current time T2
4. Lock is acquired if: majority of instances (N/2+1) agreed AND (T2-T1) < lock_timeout - clock_drift
5. Actual validity time = lock_timeout - (T2-T1) - clock_drift

To release: run the Lua delete script on all instances.

Martin Kleppmann's criticism (2016 "How to do distributed locking"):

1. Clock assumption: Redlock relies on clocks being roughly synchronized. If the OS clock on a Redis node jumps forward (NTP adjustment, VM pause), a lock can expire prematurely — another process acquires it while the first still "holds" it. Kleppmann says: use fencing tokens (monotonically increasing integer per lock acquisition) that the protected resource validates, not time-based locks.
2. Process pauses: A process holds the lock, gets paused (GC pause, page fault, preemption), lock expires, another process acquires the lock, both now execute the critical section.

3. Conclusion: Redlock provides no stronger guarantee than a single-node lock in the face of GC pauses; and is weaker than ZooKeeper or etcd (which use consensus algorithms and don't rely on timing assumptions).

When Redlock is still appropriate:

- Your critical section has a side effect that is idempotent (safe to re-execute)
- You use fencing tokens alongside the lock
- The cost of occasional double-execution is low (not financial transactions)
- You don't have ZooKeeper/etcd in your stack

For strong distributed locking: Use etcd (with lease-based locks) or ZooKeeper. These use Raft/Paxos which are correct under network partitions.

IMPLEMENTATION CHALLENGE:

Implement a distributed lock for Deepta AI's application processing system where only one worker should process a given application at a time:

1. Implement single-node Redis lock in Go with: acquire (SET NX EX), release (Lua atomic script), automatic TTL extension (background goroutine that EXPIRE-refreshes while work is in progress).
2. Add fencing token: maintain a Redis counter INCR lock:fence:<resource>, return the counter value with the lock. Pass the fence value to the downstream service; the service rejects requests with fence < its last seen fence.
3. Implement Redlock with 3 Redis instances: try to acquire on all 3, measure elapsed time, validate majority + time constraint.
4. Write a chaos test: mid-lock, simulate a Redis node failure and a GC pause (time.Sleep(35s) > lock TTL). Observe whether two workers simultaneously enter the critical section.
5. Compare with an etcd-based lock: use etcd client in Go, acquire a lease-based lock, demonstrate that etcd's Raft consensus prevents the split-brain scenario that Redlock allows.

FOLLOW-UP PROBES:

- What is the 'fencing token' solution to Redlock's timing problem? How does the downstream resource use the token to prevent split-brain?
- A GC pause in your Go application causes it to hold a Redis lock for longer than the TTL. The lock expires, another worker acquires it. How do you prevent double-processing for financial operations like scholarship disbursement?
- Antirez (Redis author) has defended Redlock against Kleppmann's critique. What is his counterargument and who is correct?
- What is the minimum number of Redis nodes needed for Redlock and why? What happens if you use only 2?

[M] MID Category 08 > Redis > Cache Invalidation

Q: What are the main cache invalidation strategies? Why is cache invalidation considered one of the hardest problems in computer science?

ANSWER:

Phil Karlton's famous quote: "There are only two hard things in Computer Science: cache invalidation and naming things."

The difficulty: A cache by definition is a stale copy. The question is: when does staleness matter and how do you detect it?

TTL-based invalidation (expiry): Set a TTL on every key. After expiry, the next read goes to DB. Simple, eventual consistency.

- Pros: Simple, predictable behavior, no invalidation logic needed.
- Cons: Staleness window = TTL duration. A counselor updates application status but students see the old status for up to 5 minutes.

Event-driven invalidation (DEL on write): When data changes in DB, immediately DEL the cache key. Next read fetches fresh data.

- Pros: Near-real-time freshness.

- Cons: Race condition — what if the DEL arrives BEFORE the DB write commits? Or after? In distributed systems, ordering is not guaranteed. What if the DEL call fails (Redis is down)?

Write-through with atomic update: On DB write, simultaneously update the cache key with the new value.

- Pros: Cache always current after a write.
- Cons: Double write. If DB write succeeds but Redis write fails, cache is stale indefinitely.

Cache-Aside with version tags: Each cache key includes a version number. On read, also fetch the current version from DB (fast meta-query). If version mismatch, invalidate and refetch.

- Cons: Still requires a DB lookup for version check — may not be faster than just reading from DB.

CDC-based invalidation (Change Data Capture): Listen to PostgreSQL WAL via Debezium or logical replication. When a row changes, publish the change to Kafka → Redis consumer DEL or SET the cache key.

- Pros: Decoupled, reliable, ordering guaranteed by Kafka.
- Cons: Adds latency (CDC pipeline delay ~100ms-1s). Infrastructure complexity.

The hardest cases:

- Distributed writes: Multiple services can write to the same DB table. Any of them must invalidate the cache. Coordinating this is hard.
- Fan-out invalidation: Changing a university's name must invalidate every application cache that includes university name (potentially millions of keys).
- Computed/aggregated values: A cache of "total applications per university this month" must be invalidated when ANY application in that month changes — extremely hard to track dependencies.

Approach for Deepta AI: Combine TTL (5-minute max staleness acceptable for most data) + event-driven DEL for write-path (immediately invalidate on status change) + CDC for complex aggregated caches.

IMPLEMENTATION CHALLENGE:

Implement a robust cache invalidation system for Deepta AI:

1. Pattern 1 — Event-driven invalidation in Go: On PATCH /applications/:id/status, within the same goroutine: (a) UPDATE PostgreSQL, (b) DEL Redis cache key. Handle Redis failure gracefully (log, don't fail the HTTP request).
2. Pattern 2 — CDC-based invalidation: Set up Debezium to watch the applications table. On row change event in Kafka topic (dbserver.public.applications), a Go consumer: extracts application_id, university_id, DELs affected keys (application:<id>, university:<id>:applications:*, university:<id>:stats).
3. Handle fan-out: When university metadata changes (name, logo), invalidate all cache keys containing that university's data. Implement a tag-based invalidation: maintain a Redis Set invalidation_group:university:<id> → {list of cache keys}. On invalidation: SMEMBERS → pipeline DEL all. Discuss the scalability limit of this pattern.
4. Write a cache coherence test: 100 concurrent goroutines randomly write to PostgreSQL and read from the cache. Assert no reader ever sees state older than 5 seconds.

FOLLOW-UP PROBES:

- Why is DEL-on-write susceptible to a race condition where the cache is repopulated with stale data between the DEL and the DB write completing? How do you fix this with versioned keys?
- What is the 'dog pile effect' (thundering herd) that occurs after a cache invalidation event and how do 'cache stampede prevention' techniques mitigate it?
- How does Cloudflare handle cache purging at scale (billions of cache entries)? What technique allows O(1) invalidation of entire groups of keys?
- You've implemented CDC-based cache invalidation with a Kafka consumer. The consumer is 10 minutes behind due to a lag spike. All cached data is 10 minutes stale. How do you detect this and what is the fallback?

[M] MID Category 08 > Redis > Lua Scripting

Q: Why does Redis support Lua scripting and when is it necessary? Explain EVAL and EVALSHA.

ANSWER:

Redis is single-threaded — each command is atomic. But what if you need to read a value and write based on it atomically? WATCH + MULTI/EXEC (optimistic locking) works but is verbose and can fail/retry. Lua scripting is the cleaner solution.

Why Lua: Redis executes Lua scripts atomically — the entire script runs without interruption from other clients. No other command can interleave between Lua statements.

EVAL script numkeys key [key ...] arg [arg ...]:

```
EVAL "if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0 end" 1 mylock my_value
```

KEYS[1] = mylock (the lock key)

ARGV[1] = my_value (the expected value)

Atomically: GET, compare, DEL if match.

EVALSHA sha1 numkeys ...: Load script with SCRIPT LOAD, get SHA1 hash, then call with EVALSHA sha1 ... instead of sending the full script every time. Saves bandwidth for frequently called scripts.

Redis Lua environment:

- redis.call(cmd, args...) — execute Redis command, raises error on failure
- redis.pcall(cmd, args...) — like call but catches errors, returns them as table
- cjson.encode/decode — JSON support
- tonumber, tostring — type conversion
- redis.log(level, message) — logging

Common Lua use cases:

1. Atomic lock release (GET+compare+DEL)
2. Rate limiter (sliding window: ZADD + ZREMRANGEBYSCORE + ZCARD atomically)
3. Atomic increment-with-cap (INCR but only if current value < max)
4. Conditional SET (SET only if current value matches expected — like CAS)
5. Atomic pipeline with conditional logic

Limitations:

- Scripts can't call blocking commands (BLPOP, etc.)
- Scripts timeout: lua-time-limit (default 5000ms) — if exceeded, Redis blocks other clients until the script finishes or is KILLED
- All keys accessed must be declared in KEYS[] array (cluster routing requirement)
- Redis 7.0+ supports Redis Functions (FUNCTION LOAD, FCALL) — persistent Lua scripts stored in Redis, not re-sent each time

IMPLEMENTATION CHALLENGE:

Implement the following atomic operations for Deepta AI using Redis Lua scripts:

1. Atomic rate limiter (sliding window, per-student): EVAL script that: removes expired entries from a Sorted Set (ZREMRANGEBYSCORE), counts current entries (ZCARD), if count < limit: ZADD current_timestamp, EXPIRE, return 1 (allowed). Else return 0 (blocked).
2. Atomic lock with re-entrance detection: EVAL that: gets current lock holder, if same client: EXPIRE to extend, return 1. If no lock: SET NX EX, return 1. Else: return 0 (blocked by different client).
3. Atomic counter-with-cap: EVAL that: GET current count, if < max_count: INCR and EXPIRE, return new count. Else: return -1 (cap reached). Use for limiting applications per student per day.
4. Load each script with SCRIPT LOAD, store the SHA1 in a Go const, use EVALSHA for production calls.
5. Handle NOSCRIPT error (SHA1 not found after Redis restart): fallback to EVAL, reload the script.

FOLLOW-UP PROBES:

- What happens if a Lua script in Redis runs for longer than lua-time-limit? What is the SCRIPT KILL command and when can't you kill a script?
- Lua scripts in Redis Cluster: all keys accessed in the script must hash to the same slot. How do you enforce this?
- What are Redis Functions (Redis 7+) and how do they differ from EVAL/EVALSHA? What problem do they solve?
- Your Lua rate limiter script runs in 2ms normally. Under extreme load, Redis queues 10,000 EVAL calls. What is the total blocking time and how do you redesign to avoid this?

[M] MID Category 08 > Redis > TTL Strategy

Q: What are the tradeoffs of different TTL strategies in Redis? How do you implement TTL-based hot/cold data tiering?

ANSWER:

TTL (Time To Live) is the expiry time set on a Redis key. After TTL expires, Redis lazily deletes the key (on next access) or actively deletes it (background expiry thread samples 10 keys with TTL per 100ms cycle).

TTL strategies:

Fixed TTL: SET key value EX 3600 — same TTL for all entries of a given type. Simple, predictable.

Tradeoff: All keys of the same type expire at roughly the same time (cache stampede risk if set all at once).

Randomized TTL: TTL = base + random(0, jitter). Spreads expiry across time to prevent mass expiry.

Example: TTL = 3600 + rand(0,300) — expires between 1h and 1h5m. Essential for avoiding thundering herd.

Sliding TTL (touch on access): EXPIRE key <ttl> on every GET. Active keys never expire; inactive keys expire after TTL from last access. Simulates LRU expiry at the application level. Costly: every read requires an EXPIRE command (extra RTT). Use GETEX key EX <ttl> (Redis 6.2+) to GET and reset TTL atomically.

No TTL (persistent keys): Never expire. Rely on eviction policy. Use for truly persistent data (session blacklist, distributed config). Risk: silent data accumulation if you forget to DEL.

Hot/cold tiering: Different TTLs based on data freshness requirements:

- Hot tier (most recently active): 5 minutes TTL. Frequently accessed, small dataset.
- Warm tier: 1 hour TTL. Less frequent access.
- Cold tier: No cache (or very long TTL with write-behind). Rarely accessed.

In Deepta AI's multi-tier caching:

- Current application status (student checking repeatedly): 1 minute TTL, randomized
- University metadata (changes rarely): 24 hour TTL
- Aggregated stats (application counts per month): 30 minute TTL, refresh triggered by new application event
- Static config (lead source list, status codes): No TTL, updated via cache invalidation on admin change

TTL and memory: Keys with TTL are stored in a separate expires dict in Redis. When querying, Redis checks this dict for expiry. This adds ~64 bytes overhead per key with TTL. For millions of keys, this is significant.

OBJECT IDLETIME key: Returns seconds since key was last accessed (requires maxmemory-policy to NOT be allkeys-lru or allkeys-lfu as these disable idle tracking).

IMPLEMENTATION CHALLENGE:

Design and implement a TTL strategy for Deepta AI's Redis caching layer that handles:

- 1M active students with session data
- 100K university application caches (JSON blobs, ~2KB each)
- 50K aggregated metrics (application counts, conversion rates)

1. Calculate total memory: sessions (1M * 200 bytes * 1.5 overhead), app caches (100K * 2KB), metrics (50K * 500 bytes). Propose a maxmemory setting.

2. Assign TTL to each category with jitter. Write a Go wrapper setWithJitter(key, value, baseTTL, jitter).

3. Implement cache warming on service startup: a Go function that pre-populates the top 1000 universities' application caches from PostgreSQL using parallel goroutines (bounded by a semaphore of 50).

4. Implement GETEX for sliding TTL on active student sessions.

5. Build a TTL health monitor: scan for keys with TTL < 60 seconds and alert if more than 10% of session keys are about to expire simultaneously (stampede risk).

FOLLOW-UP PROBES:

- What is the 'thundering herd' / 'cache stampede' problem and how does TTL jitter solve it? What is the probabilistic early expiration technique as an alternative?

- Redis's lazy expiry means a key with an expired TTL still occupies memory until accessed. How does the active expiry background process work and at what rate does it sample?
- What is the OBJECT ENCODING and OBJECT IDLETIME command and why can IDLETIME not be used with LRU eviction policy?
- Your Redis keys have TTL=3600 but you also have maxmemory-policy=allkeys-lru. A key is evicted by LRU before its TTL expires. Is this correct behavior? What does it imply about your maxmemory setting?

[A] **ADVANCED** Category 08 > Redis > Cache Stampede

Q: Explain cache stampede (thundering herd). Implement three different prevention mechanisms and compare their tradeoffs.

ANSWER:

Cache stampede: A cached value expires. Multiple concurrent requests simultaneously miss the cache and each independently issues a DB query to repopulate it. If the DB query takes 500ms and there are 1,000 concurrent users, the DB receives 1,000 simultaneous queries — overloading it, potentially causing cascading failures.

Worst case scenario:

- University application portal with 10,000 students hitting "check status" during admission announcement
- Cache key status:university:42 expires
- All 10,000 simultaneously query PostgreSQL for the same data
- PostgreSQL connection pool exhausted → timeout errors → retry loops → worse overload

Prevention Mechanism 1 — Mutex/Lock:

On cache miss, acquire a Redis distributed lock (SET NX EX). Only the lock holder fetches from DB and repopulates cache. Others wait (polling) or receive a stale value.

- Pro: Simple, deterministic
- Con: Lock holder latency affects all waiters. Deadlock risk if lock holder crashes. Polling adds Redis load.

Prevention Mechanism 2 — Probabilistic Early Expiration (XFetch):

Key insight: Don't wait for TTL to expire. Randomly decide to recompute BEFORE expiry, proportional to how close to expiry and how expensive the computation is.

Formula: recompute if: $\text{current_time} - \beta * \log(\text{random}()) > \text{expiry_time}$

Where β is the cost of recomputation (higher β = recompute earlier).

Random: when current_time is far from expiry, random() must be very close to 0 to trigger early recompute.

As expiry approaches, trigger probability increases.

- Pro: No locking, no coordination, probabilistic spreading of recomputation
- Con: Slightly complex math, tuning β

Prevention Mechanism 3 — Stale-While-Revalidate:

Serve stale cached value immediately. In background, kick off exactly one DB fetch and update cache.

Dual TTL: store both the value AND an expiry timestamp. Check: if expired → return stale immediately, trigger background refresh, update cache. Others who hit the stale key see the old value but don't trigger additional fetches.

- Pro: Zero latency for users (always serve from cache, even stale). No thundering herd.
- Con: Stale data served during revalidation period. Complex coordination to ensure only one background refresh.

TTL jitter (prevention, not cure): Adds randomness to expiry time so all keys of the same type don't expire simultaneously — reduces the scale of stampede but doesn't eliminate it.

IMPLEMENTATION CHALLENGE:

Implement all three prevention mechanisms for Deepta AI's university application status cache:

1. Mutex lock mechanism in Go: On cache miss, attempt Redis SETNX lock:status:<univ_id> worker_id EX 10. If acquired: fetch from DB, SET cache, DEL lock. If not acquired: wait 50ms and retry up to 10 times. If still no cache after 10 retries: fetch from DB directly (circuit breaker fallback).
2. Probabilistic early expiration (XFetch): Store cache value with embedded expiry_timestamp. On each cache read, compute: if time.Now().UnixNano() - beta*math.Log(rand.Float64()) > expiry — then recompute synchronously. Tune $\beta=1.0$ and observe that recomputation begins ~30 seconds before a 5-minute TTL expires.
3. Stale-while-revalidate: Store {value, soft_ttl, hard_ttl} as JSON. On read: if within soft_ttl → return fresh. If past soft_ttl but within hard_ttl → return stale + trigger background goroutine (via channel, bounded to 1 concurrent refresh per key) to recompute. If past hard_ttl → block and recompute synchronously.
4. Load test with k6: simulate 5,000 concurrent requests at the moment cache key expires. Compare DB query count for each mechanism. Target: DB receives ≤ 5 queries regardless of concurrency.

FOLLOW-UP PROBES:

- The mutex lock mechanism has a failure mode: the lock holder crashes after acquiring the lock but before setting the cache. The key expires after 10s. All waiters then flood DB. How do you handle this?
- Stale-while-revalidate serves stale data. For Deepta AI's application status, a student's application was just accepted. Under stale-while-revalidate, they see 'pending' for up to 30 seconds. Is this acceptable? How do you handle it?
- At what traffic scale does TTL jitter alone become insufficient to prevent stampede?
- XFetch requires random number generation per cache read. At 100,000 reads/second, what is the CPU overhead of this check and how do you optimize it?

[A] ADVANCED Category 08 > Redis > HyperLogLog & Bloom Filter

Q: Explain HyperLogLog and Bloom Filter internals. When are probabilistic data structures the correct engineering choice?

ANSWER:

Both are probabilistic data structures — they trade perfect accuracy for dramatic memory reduction.

HyperLogLog (HLL): Estimates cardinality (number of unique elements) with ~0.81% standard error using only ~12KB of memory, regardless of input size.

Concept: Hash each incoming element. Look at the longest run of leading zeros in the hash. A longer run of zeros means more elements have been seen (probabilistic). HLL maintains 16,384 registers, each tracking the maximum leading-zeros run length for a subgroup. The harmonic mean of these registers gives the cardinality estimate.

Redis operations: PFADD key element [element...], PFCOUNT key [key...], PFMERGE destkey [sourcekey...]

PFCOUNT is $O(1)$. PFADD is $O(1)$. PFMERGE is $O(N)$ where N is number of registers.

When to use HLL: You need to count unique users, unique IPs, unique events — where exact count isn't required and the dataset is too large to store all unique values. E.g., Deepta AI counting unique students who viewed a university profile page — 0.81% error on a count of 1M students is $\pm 8,100$ — completely acceptable. Storing 1M UUIDs would cost ~38MB per university.

Bloom Filter: A bit array + multiple hash functions. Tests set membership.

- BFADD key element: hash element with k hash functions, set those k bits to 1
- BFMEMBERSHIP key element: hash element with k hash functions, check if ALL those bits are 1

Properties:

- False positives possible: All k bits are 1 but element was never added (set by other elements)
- False negatives impossible: If any bit is 0, the element was definitely NOT added
- Memory: $O(m)$ where m is number of bits (independent of number of elements after initial sizing)
- Error rate vs memory: configurable. BF.RESERVE key error_rate initial_capacity. 1% error at 1M items ≈ 9 MB.

When to use Bloom Filter: You want to avoid expensive lookups for definitely-absent elements.

Example: Deepta AI lead deduplication — before writing a new lead from Google Ads to PostgreSQL, check the Bloom filter. If BF EXISTS returns false, definitely new → insert. If true (possibly seen before) → do the PostgreSQL lookup to confirm. Reduces DB reads for duplicates by ~90%.

Other use cases: Email duplicate-send prevention, URL seen-before checking (crawler), cache-miss optimization (check Bloom filter before hitting DB — if filter says "no", skip DB).

IMPLEMENTATION CHALLENGE:

Implement probabilistic data structures for Deepta AI's analytics platform:

1. HyperLogLog for unique student reach: PFADD portal_views:univ:<id>:<date> <student_id> on each page view. PFCOUNT portal_views:univ:<id>:<date> for daily unique visitors. PFMERGE portal_views:univ:<id>:monthly ... for monthly aggregation across daily HLLs. Compare memory: HLL (12KB) vs a Redis Set ($N * 50$ bytes per UUID).
2. Bloom filter for lead deduplication: At Deepta AI, webhook leads from Google Ads arrive repeatedly for the same phone number. BF.RESERVE leads_seen 0.001 5000000 (0.1% error rate, 5M capacity). On each new lead: if BF EXISTS leads_seen <phone> == 0 → insert to DB + BF.ADD leads_seen <phone>. Else → check DB for exact match. Measure reduction in DB writes.
3. Counting Bloom Filter: Discuss how a CBF enables deletion (regular BF doesn't support deletion). When would you need this for Deepta AI (e.g., a student withdraws an application — remove from 'applied' bloom filter)?
4. Measure accuracy: Insert 1M student IDs into HLL, compare PFCOUNT vs actual COUNT DISTINCT in PostgreSQL. Repeat 10 times with different IDs and plot the error distribution.

FOLLOW-UP PROBES:

- Can you merge two HyperLogLogs from different time periods to get the unique count for the combined period? What is the PFMERGE operation doing internally?
- A Bloom filter's false positive rate increases as more elements are added beyond the initial capacity. How do you detect when a Bloom filter is 'full' and what are your options?
- Redis's built-in HyperLogLog has a fixed 12KB size. What is the tradeoff between register count (16,384) and accuracy?
- Your Bloom filter for lead deduplication shows a 5% false positive rate (expected 0.1%). What are the possible causes and how do you fix it without dropping the filter?

[P] PRINCIPAL Category 08 > Redis > Principal Design

Q: Design a Redis-backed distributed rate limiter that handles 1 million requests per second across 100 microservices, with per-user, per-endpoint, and global limits, supporting sliding window semantics.

ANSWER:

This is a Principal-level design problem requiring correctness, performance, and operational excellence.

REQUIREMENTS DECOMPOSITION:

- 1M req/s total, 100 services, ~10k req/s per service on average
- Per-user rate limit: e.g., 1000 req/min per user_id
- Per-endpoint rate limit: e.g., 100 req/min on POST /applications
- Global rate limit: e.g., 50,000 req/min total across all services
- Sliding window (not fixed window — no burst at window boundary)
- Latency: rate limit check must add <5ms p99 to request path

SLIDING WINDOW IMPLEMENTATION:

Use Redis Sorted Set per limit key. Score = timestamp (microseconds). Member = unique request ID.

Algorithm per request:

1. ZREMRANGEBYSCORE key 0 (now - window_ms) — prune expired entries
2. ZCARD key — count current requests in window
3. If count < limit: ZADD key now request_id; EXPIRE key window_sec*2; return ALLOW
4. Else: return DENY

Atomicity: Wrap in Lua script (steps 1-3 must be atomic).

SCALE TO 1M RPS:

Challenge: 1M Lua script invocations/second on a single Redis node is impossible (Redis handles ~100k ops/s for complex commands).

Solution — Redis Cluster with consistent hash routing:

- 16 Redis shards, each handling ~62k req/s (manageable)
- Route per-user limits to a specific shard based on $\text{hash}(\text{user_id}) \% 16$
- Per-endpoint limits similarly sharded
- Global limits: CANNOT be sharded (need aggregation) → use a token bucket with periodic sync

GLOBAL RATE LIMITING DESIGN:

Exact global limit at 1M RPS is impractical with Redis — the coordination overhead exceeds the limit check latency budget.

Solutions:

1. Approximate global with local quotas: Distribute global budget across services. Global 50k/min → each of 100 services gets 500/min local budget. Services request budget refills from a central coordinator every second. Coordinator is the bottleneck but only receives 100 requests/second (one per service).
2. Token bucket with async sync: Each service maintains a local token bucket in memory. Every 100ms, sync consumed tokens to Redis (`INCRBY global_consumed <local_count>`). Accept approximate counting (~100ms window of error).
3. Sliding window with sampling: Only check Redis for 1% of requests. Scale the count by 100. Acceptable for rate limiting where exact precision isn't critical.

ARCHITECTURE:

- Rate limiter as a shared library (Go) imported by each microservice
- Each service: local in-memory token bucket (handles 90% of decisions), backed by Redis Sorted Set (handles residual, enforces hard limits)
- Redis Cluster: 6 nodes (3 masters + 3 replicas), each master in different GCP zone
- Fallback: if Redis is unavailable, degrade to local token bucket only (accept false permits under Redis outage)

OPERATIONAL:

- Metrics: hit rate, allow rate, Redis RTT via Prometheus
- Dynamic limits: store limits in Redis Hash `rate_limits:<service>:<endpoint>`, update without deployment
- Burst allowance: sliding window naturally allows bursts within the window — add a "burst cap" check using ZCARD for last 1-second sub-window

IMPLEMENTATION CHALLENGE:

Implement the rate limiter system:

1. Lua script for sliding window: takes key, window_ms, limit as args. Returns {allowed: bool, current_count: int, retry_after_ms: int}.
2. Go RateLimiter struct: maintains a pool of Redis connections (go-redis Cluster), key naming convention `rate:<user_id>:<endpoint>`, calls EVALSHA, handles NOSCRIPT fallback.
3. Go middleware for Gin/Echo: wrap each endpoint with rate limit check. Return 429 Too Many Requests with Retry-After header on denial.
4. Local token bucket fallback: in-memory token bucket (refill goroutine every second), used as first line of defense. Only call Redis if local bucket is exhausted.
5. Load test with k6: ramp to 10,000 concurrent users each sending 100 req/s. Verify: (a) rate limit is enforced per user, (b) Redis latency stays <2ms p99, (c) overall throughput stays at 1M req/s.
6. Dynamic limit configuration: `rate_limits` hash in Redis, watched by each service via SUBSCRIBE `config_updates` channel. On update, reload limit config without restart.

FOLLOW-UP PROBES:

- Your sliding window Lua script runs at 100k req/s on a single Redis shard. At 200k req/s, latency spikes to 50ms. You can't add more shards for per-user keys (user IDs are not uniformly hashable). What do you do?

- A service using your rate limiter is mis-configured with a bug that sends 10M requests in 10 seconds to one endpoint. Your rate limit is 1000/min. How does your system respond and what is the blast radius on Redis?
- Your rate limiter uses Redis Sorted Sets. Redis memory for rate limit data is growing unboundedly — ZREMRANGEBYSCORE runs but eviction isn't keeping up. What is happening and how do you fix it?
- Regulatory requirement: the rate limit system must produce an audit log of every DENY decision for compliance. At 1M req/s with 1% deny rate, that's 10k events/second to log. Design the audit pipeline without adding latency to the request path.

CATEGORY 09

KAFKA & EVENT STREAMING

Brokers · Producers · Consumers · Exactly-Once · KRaft · Schema Registry

14 questions · [E]=6 [M]=5 [A]=2 [P]=1

TOPICS COVERED IN THIS CATEGORY:

- Kafka architecture: brokers, topics, partitions (append-only log, offsets, segments)
- Replication: ISR, leader election, min.insync.replicas, acks=all
- Producer internals: batching, linger.ms, compression, idempotent producer, transactional API
- Consumer internals: poll loop, fetch.min.bytes, max.poll.interval.ms
- Consumer groups: partition assignment, rebalancing — eager vs cooperative sticky
- Offset management: auto-commit vs manual, at-most-once / at-least-once / exactly-once semantics
- KRaft mode (replacing ZooKeeper): Raft consensus, metadata log
- Log compaction, tombstone records, compacted topics
- Dead letter queue pattern, backpressure handling
- Hot partition problem and solutions (salting, composite keys)
- Schema Registry: Avro/Protobuf/JSON Schema, compatibility modes
- Monitoring: consumer lag, __consumer_offsets, Prometheus integration
- Deepti AI Kafka pub/sub pipeline design and exactly-once to PostgreSQL
- GDPR deletion in immutable Kafka logs: crypto-shredding and outbox pattern

[E] EASY Category 09 > Kafka > Architecture Basics

Q: Explain Kafka's core architecture: brokers, topics, partitions, and offsets. How does this differ from a traditional message queue?

ANSWER:

Kafka is a distributed, append-only commit log designed for high-throughput, fault-tolerant event streaming. Core concepts:

Broker: A Kafka server process. A Kafka cluster contains multiple brokers (typically 3+). Each broker stores a subset of the data. Brokers are identified by a numeric ID. One broker acts as the cluster controller (manages partition leadership, broker registration).

Topic: A named logical channel for messages. Producers write to topics; consumers read from topics. A topic is conceptually similar to a database table or a folder.

Partition: Topics are divided into partitions. Each partition is an ordered, immutable sequence of records (the append-only log). Partitions are the unit of parallelism — more partitions = more consumers can read concurrently. Each partition is stored on exactly one broker (the leader) with replicas on other brokers.

Offset: Each message within a partition has a unique, monotonically increasing integer offset starting from 0. Offsets are per-partition (partition 0 has offsets 0..N, partition 1 has offsets 0..M — they're independent).

Consumers track their position by storing offsets.

Segments: Each partition is further divided into segment files on disk (default 1GB or 1 week). Old segments can be deleted or compacted based on retention policy.

How Kafka differs from traditional MQ (e.g., RabbitMQ):

- **Log retention:** Traditional MQ deletes messages after delivery. Kafka retains messages for a configurable period (default 7 days). Multiple consumers can re-read the same messages at different offsets.
- **Consumer pull model:** Consumers pull messages at their own pace. Traditional MQs push to consumers.
- **Ordering:** Kafka guarantees ordering within a partition, not across partitions. Traditional queues typically guarantee FIFO.

- Consumer groups: Multiple consumers in a group share partitions (each partition goes to exactly one consumer in the group). Traditional MQ typically uses competing consumers on a shared queue.
- Replay: Consumers can reset offsets to re-process historical data. Traditional MQ: once consumed and acked, message is gone.
- Scale: Kafka is designed for millions of messages/second. Traditional MQs typically peak at thousands/second.

In Deepta AI: Kafka serves as the central nervous system for the student application platform — application_submitted events flow through Kafka to trigger async processing (document validation, email notifications, lead attribution) without coupling services.

IMPLEMENTATION CHALLENGE:

Set up a local Kafka environment (or use Confluent Cloud) for Deepta AI:

1. Create a topic 'student.applications' with 6 partitions and replication factor 3.
2. Write a Go Kafka producer (using confluent-kafka-go or franz-go) that publishes ApplicationSubmittedEvent with: application_id, student_id, university_id, timestamp, lead_source. Use student_id as the partition key to ensure all events for one student go to the same partition.
3. Write a Go consumer group (group.id="application-processor") that reads from 'student.applications' and logs each event with its partition and offset.
4. Start 3 instances of the consumer — observe partition assignment (2 partitions per consumer).
5. Stop one consumer instance — observe Kafka trigger a consumer group rebalance and reassign its partitions to the remaining 2 consumers.
6. Use kafka-consumer-groups.sh to inspect consumer lag.

FOLLOW-UP PROBES:

- Why is ordering only guaranteed within a partition and not across partitions? What design decision do you make at Deepta AI to preserve per-student event ordering?
- What determines which partition a message goes to? Compare: no key (round-robin), explicit key (murmur2 hash), custom partitioner.
- What is the Kafka controller broker and what happens when it fails?
- Your topic has 6 partitions but you start a consumer group with 8 consumers. What happens to the extra 2 consumers?

[E] EASY Category 09 > Kafka > Producer Internals

Q: Walk through Kafka producer internals: the record accumulator, batching, linger.ms, compression, and the acks setting.

ANSWER:

Kafka producer internals are designed for maximum throughput with configurable durability.

Record Accumulator: When you call producer.send(record), the record goes into an in-memory buffer called the Record Accumulator, organized by (topic, partition). Records for the same partition accumulate in a batch before being sent.

Batching: The sender thread reads from the accumulator and sends batches to brokers. A batch is sent when either:

- batch.size (default 16KB) is reached — the batch is full
- linger.ms (default 0ms) elapses — wait this long before sending even if batch isn't full

linger.ms=0: Send immediately with whatever is in the accumulator at this moment (low latency, small batches, lower throughput).

linger.ms=5: Wait up to 5ms to accumulate more records. At high throughput, batches fill quickly anyway. At low throughput, this adds 5ms latency but better amortizes the per-request overhead.

Recommendation: linger.ms=5-20ms for high-throughput systems.

Compression: Set compression.type=snappy|lz4|gzip|zstd. Compression happens per batch (before sending to broker). Larger batches compress better (more redundancy to exploit). LZ4 is fast and efficient. Snappy is balanced. GZIP best compression but slow. ZSTD (Kafka 2.1+) best overall.

acks: Controls how many broker acknowledgments the producer waits for before considering a send complete.

acks=0: Fire and forget. No acknowledgment. Fastest, no durability guarantee.

acks=1: Leader broker acknowledges after writing to its log. If leader fails before replicating, data is lost.

acks=all (or -1): All in-sync replicas (ISR) acknowledge. Highest durability. Combined with min.insync.replicas=2 ensures at least 2 brokers have the data before ack.

Idempotent producer (enable.idempotence=true): Prevents duplicate writes on producer retry. Kafka assigns each producer a PID (producer ID) and tracks sequence numbers per partition. If a retry sends a duplicate sequence, the broker discards it. Automatically forces acks=all and retries > 0.

Transactional API: For exactly-once across multiple partitions/topics — wraps multiple sends and consumer offset commits in an atomic transaction. Used in Kafka Streams.

For Deepta AI's Kafka pipeline: Use acks=all, enable.idempotence=true, linger.ms=10, compression.type=snappy. This gives durability + throughput at the cost of slightly higher latency.

IMPLEMENTATION CHALLENGE:

Implement and benchmark different Kafka producer configurations for Deepta AI's application event pipeline:

1. Baseline producer: acks=1, no compression, linger.ms=0. Send 100,000 ApplicationSubmittedEvents and measure throughput (messages/sec) and latency (p50, p95, p99).
2. Optimized producer: acks=all, compression=snappy, linger.ms=10, batch.size=65536. Measure same metrics.
3. Idempotent producer: enable.idempotence=true. Simulate a retry (send the same message twice with same sequence). Verify only one message arrives at the consumer.
4. Produce with custom partitioner: route applications by lead_source (google_ads → partition 0, facebook → partition 1, ivr → partition 2, others → partition 3+). Verify partition distribution.
5. Monitor producer metrics via JMX or producer stats: record-send-rate, request-latency-avg, batch-size-avg, compression-rate-avg.

FOLLOW-UP PROBES:

- What happens when the producer's send buffer (buffer.memory) is full? What does max.block.ms control?
- With acks=all and min.insync.replicas=2, you have 3 brokers and 1 goes down. Can the producer still send? What if 2 go down?
- What is the difference between idempotent producer and transactional producer? Can you use transactions without idempotence?
- Your Deepta AI producer sends application events but the consumer reports duplicate events during a Kafka broker restart. What producer configurations prevent this?

[E] EASY Category 09 > Kafka > Consumer Internals

Q: Explain Kafka consumer internals: the poll loop, fetch configuration, and consumer group rebalancing.

ANSWER:

Kafka consumers use a pull model — they actively poll brokers for new records.

Poll loop: The core consumer execution pattern:

```
while (true) {
  records = consumer.poll(timeout=Duration.ofMillis(1000))
  for record in records: process(record)
  consumer.commitSync() // or commitAsync
}
```

poll() does multiple things:

1. Fetches records from the broker for assigned partitions
2. Sends heartbeats to the group coordinator (proves consumer is alive)
3. Triggers rebalance if group state has changed

`fetch.min.bytes` (default 1): Minimum data to fetch from broker per request. If less is available, broker waits up to `fetch.max.wait.ms` (default 500ms) before responding. Increase for higher throughput at cost of latency.

`fetch.max.bytes` (default 50MB): Maximum data per fetch response.

`max.poll.records` (default 500): Maximum records returned per `poll()`.

`max.poll.interval.ms` (default 5 minutes): Maximum time between consecutive `poll()` calls. If a consumer doesn't call `poll()` within this interval, the group coordinator considers it dead and triggers rebalance. Critical: if your message processing takes longer than this, increase it or reduce `max.poll.records`.

`session.timeout.ms` (default 45s): How long without a heartbeat before a consumer is considered dead (heartbeats are sent automatically in the background by the consumer client).

Consumer Group: Multiple consumers with the same `group.id` form a group. Kafka assigns each partition to exactly one consumer in the group. This is partition-based load sharing.

Offset management:

`auto-commit` (`enable.auto.commit=true`): Automatically commits the last polled offset every `auto.commit.interval.ms` (5s). Risk: if the consumer crashes after poll but before processing, committed offsets advance and messages are lost (at-most-once).

Manual commit: `commitSync()` after processing — at-least-once. `commitAsync()` — non-blocking but ack order not guaranteed.

In Deepta AI's Go Kafka consumers: Use manual offset commit after PostgreSQL write to ensure exactly-once processing semantics (combined with idempotent DB writes).

IMPLEMENTATION CHALLENGE:

Implement a robust Kafka consumer for Deepta AI's application processing pipeline in Go:

1. Consumer setup: `group.id="application-processor"`, `max.poll.records=100`, `max.poll.interval.ms=300000` (5min), `enable.auto.commit=false`.
2. Poll loop: fetch batch, process each `ApplicationSubmittedEvent` (validate, write to PostgreSQL), commit offsets manually after each batch succeeds.
3. Handle processing failures: if PostgreSQL write fails, log the error and DO NOT commit — Kafka will redeliver the batch on next poll.
4. Implement graceful shutdown: on `SIGTERM`, stop the poll loop, commit pending offsets, close the consumer cleanly.
5. Simulate `max.poll.interval.ms` breach: add a 6-minute sleep in the processing loop. Observe the rebalance error and the duplicate deliveries.
6. Implement a dead letter queue (DLQ): after 3 processing failures for a message, write to a `dead_letter` Kafka topic instead of blocking the main consumer.

FOLLOW-UP PROBES:

- What is the difference between `session.timeout.ms` and `max.poll.interval.ms`? Can you set `max.poll.interval.ms < session.timeout.ms`?
- You use `commitAsync()` in your consumer for performance. What happens if a later offset is committed before an earlier one fails? What does `commitAsync()` guarantee?
- What is the 'phantom rebalance' problem in Kafka consumers and how does cooperative sticky rebalancing (introduced in Kafka 2.4) solve it?
- Your Deepta AI consumer processes 100 messages in a batch. The 73rd message fails. With manual commit, you DON'T commit and the whole batch re-delivers. How do you avoid re-processing the first 72 messages?

[E] EASY Category 09 > Kafka > Offset Management

Q: Explain Kafka offset management. What is the difference between at-most-once, at-least-once, and exactly-once semantics?

ANSWER:

Kafka's delivery guarantees are determined by how producers send and consumers commit offsets:

At-most-once (fire and forget): Messages may be lost but never duplicated.

Producer: acks=0 (no acknowledgment) or no retry on failure.

Consumer: commit offset BEFORE processing. If processing crashes after commit, message is lost.

Use case: Metrics, analytics events where occasional data loss is acceptable.

At-least-once (the default, practical choice): Messages may be delivered multiple times but never lost.

Producer: acks=all + retries=MAX_INT + idempotent producer.

Consumer: commit offset AFTER successful processing. If consumer crashes after processing but before commit, message re-delivers on restart → process again.

Your system must be idempotent: "receiving the same message twice should produce the same result." In

Deepta AI: check if application_id already exists in PostgreSQL before inserting.

Exactly-once (the holy grail): Each message is processed exactly once end-to-end.

Two levels:

1. Exactly-once within Kafka (Kafka transactions): Producer wraps multiple sends in a transaction. Either all succeed or none do. Prevents duplicates on producer retry.

2. Exactly-once to external system (consumer → PostgreSQL): Kafka alone can't guarantee this because the external write (PostgreSQL INSERT) is outside Kafka's control. The pattern:

- Write to PostgreSQL + commit Kafka offset in the SAME database transaction (store offset in a PostgreSQL table alongside your data)
- Or use the Kafka Streams transactional API (if your sink is also a Kafka topic)

The offset commit + DB write pattern for true exactly-once:

BEGIN PostgreSQL transaction

INSERT INTO applications (...) VALUES (...)

INSERT INTO kafka_offsets (topic, partition, offset) VALUES (...) ON CONFLICT UPDATE

COMMIT

Only commit Kafka offset when PostgreSQL transaction succeeds.

On consumer restart: read the last committed offset from the kafka_offsets table — resume exactly where you left off.

In Deepta AI's pipeline: Use at-least-once with idempotent consumer (check for existing application_id before insert). True exactly-once is complex — implement only if business logic demands it (e.g., financial disbursements).

IMPLEMENTATION CHALLENGE:

Implement exactly-once delivery for Deepta AI's application processing consumer:

1. Create a kafka_offsets table in PostgreSQL: (topic TEXT, partition INT, consumer_group TEXT, offset BIGINT, PRIMARY KEY (topic, partition, consumer_group)).
2. Consumer startup: read committed offsets from kafka_offsets for this consumer group. Use consumer.Seek(partition, offset+1) to resume from the correct position (don't use Kafka's internal offset store).
3. Per-message processing: within a single PostgreSQL transaction: INSERT INTO applications (ignore if duplicate application_id), UPDATE kafka_offsets SET offset=<current_offset>.
4. NO Kafka offset commit — offsets are entirely managed in PostgreSQL.
5. Simulate crash: kill the consumer after 50 messages without completing. Restart — verify it resumes from exactly message 51.
6. Compare this with Kafka's transactional API (enable.idempotence=true + transactional.id): show how Kafka transactions handle exactly-once for Kafka-to-Kafka scenarios.

FOLLOW-UP PROBES:

- The 'at-least-once + idempotent consumer' pattern is simpler than true exactly-once. Why does idempotency give you effectively the same correctness guarantee?
- What is the 'two generals problem' and why does it make exactly-once delivery fundamentally impossible across a network partition?
- Kafka's transactional API uses a 2PC-like protocol with a Transaction Coordinator. What is the Transaction Coordinator and what happens if it fails mid-transaction?
- Your Deepta AI exactly-once consumer is processing 1,000 messages/second. Each message requires a PostgreSQL transaction (with offset update). At what throughput does PostgreSQL become the bottleneck and how do you batch?

[E] EASY Category 09 > Kafka > Replication

Q: Explain Kafka's replication model: ISR, leader election, min.insync.replicas, and what acks=all guarantees.

ANSWER:

Kafka replication ensures fault tolerance: each partition has one leader and N-1 followers (replicas).

Leader: All reads and writes go through the leader. The leader tracks which replicas are "in sync."

In-Sync Replicas (ISR): The set of replicas that are caught up with the leader. A replica is in the ISR if it has fetched all messages from the leader within `replica.lag.time.max.ms` (default 30s). If a replica falls behind (slow network, GC pause, disk I/O), it's removed from the ISR.

Leader Election: When a leader fails, the controller (a broker) elects a new leader from the ISR. Only ISR members can be elected — ensuring the new leader has all committed messages.

`unclean.leader.election.enable=false` (default): A leader is only elected from ISR. If ISR is empty (all replicas are behind or down), the partition is offline. Data availability sacrificed for data correctness.

`unclean.leader.election.enable=true`: Any replica can become leader (even if not in ISR). Partition stays available but committed messages may be lost.

`min.insync.replicas` (default 1): Minimum number of ISR replicas that must acknowledge a write before the broker considers it committed (when combined with `acks=all`).

- `min.insync.replicas=1`: `acks=all` means just the leader acks. Same as `acks=1` essentially.
- `min.insync.replicas=2` (recommended): With replication factor 3, at least 2 brokers must ack. Can tolerate 1 broker failure without data loss.

`acks=all` + `min.insync.replicas=2` + `replication.factor=3`: The production-safe configuration. A write is durable against single broker failure. If ISR shrinks to 1 (two brokers down), producers get

`NotEnoughReplicasException` — correct behavior: refuse new writes rather than risk data loss.

Deepta AI's Kafka cluster (GCP): Topic configuration: `replication.factor=3` (3 brokers across 3 GCP zones), `min.insync.replicas=2`. Producer: `acks=all`. This ensures no data loss for application events even if one entire GCP zone goes down.

IMPLEMENTATION CHALLENGE:

Set up a 3-broker Kafka cluster and test replication behavior for Deepta AI:

1. Create topic 'student.applications' with `replication.factor=3`, `min.insync.replicas=2`.
2. Use `kafka-topics.sh --describe` to observe ISR for each partition.
3. Produce 10,000 events with `acks=all`.
4. Simulate broker failure: kill broker 2 (not the leader). Observe: ISR shrinks from [1,2,3] to [1,3]. Producers continue (ISR=2 still meets `min.insync.replicas=2`). Consumer continues.
5. Kill broker 3: ISR shrinks to [1]. Producer receives `NotEnoughReplicasException`. Show this in code and implement retry with backoff.
6. Bring broker 2 back online: observe replica catch-up and ISR expanding back.
7. Monitor replication lag: `kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions` JMX metric — alert if > 0 in production.

FOLLOW-UP PROBES:

- What is the difference between replication factor and `min.insync.replicas`? Can `min.insync.replicas` exceed replication factor?
- What is 'preferred replica election' and why does Kafka use it? What is the LeaderCount imbalance?
- If a broker falls out of ISR and then rejoins, how does it catch up? What is the 'follower fetch' mechanism?
- You have 5 brokers and `replication.factor=3`. Broker 0 goes down. Which partitions are affected? Write the formula for determining impact.

[E] EASY Category 09 > Kafka > Log Compaction

Q: Explain Kafka log compaction and tombstone records. When do you use a compacted topic?**ANSWER:**

Standard Kafka retention: Delete messages older than retention.ms (default 7 days) or larger than retention.bytes. Good for event streams where old events are irrelevant.

Log compaction: Instead of time-based deletion, Kafka retains only the most recent message per key. Older messages with the same key are cleaned up by the log cleaner.

How compaction works:

- Each partition has a "clean" section (already compacted) and a "dirty" section (not yet compacted)
- The log cleaner scans the dirty section, builds a map of key → latest offset, then rewrites removing old versions while keeping the latest
- Compaction is non-blocking — reads and writes continue during compaction

Tombstone records: To delete a key from a compacted topic, produce a message with the key and null value (tombstone). The compaction process retains tombstones for delete.retention.ms (default 24h) then discards them. After the tombstone is cleaned up, the key no longer exists in the log.

When to use compacted topics:

1. Change Data Capture (CDC): Each event represents the latest state of an entity. Consumer reading from offset 0 gets current state of all entities (key=entity_id, value=latest_state). Consumers joining late can bootstrap from the compacted state.
2. Configuration/reference data: Application configuration, lookup tables. Producers update values; compacted topic holds current config. Consumers always have latest values after reading to end.
3. Event sourcing snapshots: Store the latest snapshot of an aggregate root. Old snapshots can be compacted away.
4. User profile updates: key=user_id, value=user_profile_json. Compacted log = current state of all profiles.

Deepta AI use case: Create a compacted topic 'student.profiles.changelog' with cleanup.policy=compact.

Whenever a student updates their profile, produce to this topic with key=student_id, value=updated_profile_json. New microservices can bootstrap their local cache by consuming this topic from the beginning — they'll receive the latest profile for each student.

Contrast with delete policy: cleanup.policy=delete,compact — keep both time-based deletion AND compaction.

IMPLEMENTATION CHALLENGE:

Implement a compacted topic pattern for Deepta AI's university metadata:

1. Create topic 'university.metadata' with cleanup.policy=compact, min.cleanable.dirty.ratio=0.1 (compact aggressively).
2. Producer: publish university metadata updates (name, country, department list) with key=university_id.
3. Update a university 5 times (simulate name change, adding departments).
4. After compaction runs (wait or trigger with kafka-log-dirs.sh), consume from offset 0 — verify only the latest message per university_id is returned.
5. Publish a tombstone (null value) for a closed university. After delete.retention.ms, verify the key is gone.
6. Implement a 'changelog consumer' that starts from offset 0, builds an in-memory map of university_id → latest_metadata, then switches to consuming new updates in real time. This is the Kafka Streams KTable pattern — build it from scratch.

FOLLOW-UP PROBES:

- What is the min.cleanable.dirty.ratio and how does it affect when compaction runs? What are the risks of setting it too low?
- After a compacted topic is consumed from offset 0, are the offsets still monotonically increasing? Can there be gaps?
- What is the difference between cleanup.policy=compact and cleanup.policy=delete,compact? When would you use the combination?
- Your compacted topic grows to 500GB but compaction is not reducing it. What are the possible causes (hint: consider active producers and dirty ratio)?

[M] MID Category 09 > Kafka > Consumer Group Rebalancing

Q: Explain Kafka consumer group rebalancing: eager vs cooperative sticky protocols. What are the production implications?

ANSWER:

When a consumer joins or leaves a group, or a new partition is added, Kafka must redistribute partition assignments — this is a rebalance.

Classic (Eager) Rebalance:

1. Group coordinator (a broker) receives a join request (new consumer or heartbeat timeout)
2. Coordinator sends JoinGroup response to all consumers: "everyone stop and revoke ALL partitions"
3. ALL consumers stop processing and revoke their current partition assignments
4. Consumers re-send JoinGroup requests
5. The elected group leader (first consumer to join) computes new assignments
6. SyncGroup: leader sends assignments to coordinator; coordinator distributes to all consumers
7. Consumers resume with new assignments

Problem: "Stop-the-world" rebalance. All consumers stop processing for the entire duration (can be seconds). All uncommitted messages from revoked partitions must be re-read. Under high throughput, this causes consumer lag spikes and latency jitter.

Cooperative Sticky Rebalance (Kafka 2.4+, Java client; Kafka 3.0+, librdkafka):

1. First rebalance round: consumers revoke only the partitions that need to move (based on the incremental rebalance protocol). They KEEP their currently-owned partitions and continue processing.
2. Second rebalance round: consumers that received new partitions claim them.

Net effect: Only the partitions actually being moved are paused. Other partitions continue processing uninterrupted.

Sticky assignment: The `partition.assignment.strategy=CooperativeStickyAssignor` minimizes partition movement between rebalances. A consumer that previously owned partition 3 will keep partition 3 if possible, reducing the number of partitions that change hands.

Incremental Cooperative Rebalance: Combined with sticky — only the minimum set of partitions moves, and processing continues on stable partitions.

Deepta AI implications:

- Use `partition.assignment.strategy=CooperativeStickyAssignor` in production
- In Go (franz-go or confluent-kafka-go with librdkafka): set `partition_assignment_strategy=cooperative-sticky`
- Monitor rebalance events: `consumer_rebalance_total` metric in Prometheus
- Set `session.timeout.ms` appropriately: too short → spurious rebalances on GC pauses; too long → slow failure detection

Rebalance triggers to minimize:

- Static group membership (`group.instance.id`): Assign a stable instance ID to each consumer. On restart, consumer rejoins with the same ID and reclaims its previous partitions WITHOUT triggering a full rebalance. Critical for pod restarts in Kubernetes.

IMPLEMENTATION CHALLENGE:

Implement consumer group management for Deepta AI's processing pipeline:

1. Set up 3 consumers with `CooperativeStickyAssignor`. Observe initial assignment.
2. Add a 4th consumer — observe ONLY the rebalanced partitions stop processing (vs eager rebalance which stops all).
3. Remove a consumer (kill process) — observe failover. Compare restart time with static membership (`group.instance.id=worker-{pod_name}`) vs dynamic membership.
4. Implement a rebalance listener in Go: `OnPartitionsRevoked` (commit pending offsets), `OnPartitionsAssigned` (reset state for new partitions). This is critical for at-least-once correctness.
5. Simulate a GC pause (`time.Sleep` longer than `session.timeout.ms`). Observe the spurious rebalance. Fix by increasing `session.timeout.ms` or using static membership.
6. In your Kubernetes deployment for Deepta AI, set `group.instance.id` to the pod name via downward API. Show how pod restarts no longer trigger full rebalances.

FOLLOW-UP PROBES:

- What is the 'incremental rebalance' protocol and how does it differ from cooperative rebalance?
- Static group membership (group.instance.id) prevents rebalances on restart. What is the tradeoff? What happens if a pod with a static ID is permanently dead?
- What is the group coordinator and how does Kafka route all consumers in a group to the same coordinator broker?
- Your Deepta AI Kafka consumers use eager rebalancing and each rebalance takes 30 seconds. During a deployment (rolling restart of 10 consumer pods), how long is the total disruption window? How does cooperative sticky reduce this?

[M] MID Category 09 > Kafka > Hot Partition & Backpressure

Q: Explain the hot partition problem in Kafka. How do you detect and fix it? What backpressure strategies does Kafka support?

ANSWER:

Hot partition: A partition that receives a disproportionate amount of traffic compared to others, causing that broker to be overwhelmed while others are underutilized.

Causes:

1. Skewed partition key: If you partition by university_id and 80% of applications come from 2 universities, those 2 keys always hash to the same 2 partitions. The brokers hosting those partitions are overloaded.
2. Large message size: One partition receives large messages (e.g., student documents 5MB each) while others receive small messages.
3. Uneven partition assignment: The cluster controller places multiple hot partitions on the same broker.

Detection:

- Monitor: BytesInPerSec per partition — a healthy topic has uniform distribution
- kafka.server:type=BrokerTopicMetrics,name=BytesInPerSec,topic=<>,partition=<>
- Consumer lag: the consumer assigned to the hot partition accumulates lag while others are idle
- Broker CPU/disk I/O: the hot broker shows high utilization

Solutions:

1. Salting (key randomization): Add a suffix to the partition key: key = university_id + ":" + rand(0, N). Messages for the same university spread across N partitions. Tradeoff: ordering per university is lost across the salt range.
2. Composite key: university_id:timestamp_minute — spreads load over time while maintaining ordering within a minute.
3. Custom partitioner: Implement a weighted partitioner that detects hot keys and routes them to multiple partitions. Complex to maintain.
4. Increase partition count: More partitions = more even distribution if the key space is large enough. Doesn't help if only 2 keys exist.
5. Message size: Move large payloads out of Kafka. Store documents in GCS/S3, send only a reference (URL) in the Kafka message.

Backpressure in Kafka:

Kafka itself doesn't have a native backpressure mechanism — producers can always write faster than consumers process. Backpressure must be implemented in the consumer or upstream.

Consumer-side backpressure strategies:

- Reduce max.poll.records: fetch fewer messages per poll, giving more processing time per batch
- Pause partition: consumer.pause(partitions) — stops fetching from that partition while processing. Resume when caught up: consumer.resume(partitions)
- Rate limit: token bucket on the consumer processing side
- Back off and retry: exponential backoff on processing failure

Producer-side: buffer.memory and max.block.ms — if producer's buffer fills up, send() blocks. This naturally propagates backpressure to the calling code.

IMPLEMENTATION CHALLENGE:

Fix the hot partition problem in Deepta AI's application submission pipeline:

1. Reproduce the problem: create a topic with 6 partitions, send 100,000 events all with key=university:42 (a mega university). Use kafka-log-dirs.sh to show partition size imbalance.
2. Implement salted partition key in Go: key = fmt.Sprintf("%s:%d", universityID, rand.Intn(6)). Re-distribute — observe even partition utilization.
3. Implement a consumer-side circuit breaker: if processing latency for a batch > 5 seconds, call consumer.Pause(assignedPartitions), sleep 10 seconds, call consumer.Resume(assignedPartitions). Prevents unbounded lag accumulation.
4. Implement consumer lag monitoring: every 30 seconds, fetch partition offsets (OffsetFetch) and latest partition offsets (ListOffsets), compute lag = latest - committed. Alert if lag > 10,000 messages.
5. Design a document upload pattern: instead of including the 5MB PDF in the Kafka message, upload to GCS, send message with gcs_url field. Show Kafka message size drops from 5MB to <1KB.

FOLLOW-UP PROBES:

- Salting a partition key solves hot partitions but breaks ordering guarantees. For Deepta AI, what events MUST maintain ordering and what events can tolerate salting?
- What is the difference between consumer pause/resume and simply increasing max.poll.interval.ms as a backpressure mechanism?
- Your Kafka topic has 6 partitions and your consumer group has 6 consumers. You add salting with N=3, creating effectively 18 logical partitions for university:42. How does the consumer group handle more logical partitions than physical partitions?
- At what consumer lag threshold do you trigger a scale-out (add more consumer pod replicas) vs a scale-up (increase processing speed per consumer)?

[M] MID Category 09 > Kafka > KRaft Mode

Q: What is KRaft mode in Kafka and why does it replace ZooKeeper? What are the operational implications?

ANSWER:

Before KRaft: Kafka relied on Apache ZooKeeper for:

- Cluster metadata storage (broker registration, topic configs, partition leaders)
- Controller election (which broker is the active controller)
- Consumer group coordination (before Kafka 0.9, now handled by group coordinator broker)

Problems with ZooKeeper dependency:

1. Operational complexity: Two separate systems to deploy, monitor, tune, and scale.
2. Metadata bottleneck: ZooKeeper handles all metadata changes; at large scale (millions of partitions), controller operations were slow.
3. Controller failover latency: On controller failure, new controller must reload all metadata from ZooKeeper — seconds of unavailability.
4. Write amplification: Every partition change: ZooKeeper write → ZooKeeper watch notification → controller processes → broker updates.

KRaft (Kafka Raft) — Kafka 3.3+ (production-ready), Kafka 4.0+ (ZooKeeper removed):

Kafka uses its own Raft consensus protocol for metadata management. A set of brokers (or dedicated controller nodes) form a quorum and store metadata in an internal Kafka topic (__cluster_metadata).

Benefits:

- Single system: No ZooKeeper to deploy/manage.
- Faster metadata operations: metadata stored in Kafka log (optimized for sequential write). Controller startup reloads from local log, not ZooKeeper network round-trips.
- Higher partition scale: KRaft supports millions of partitions (ZooKeeper struggled beyond 200k).
- Faster failover: New controller starts serving immediately (reads from local log). Failover from seconds → milliseconds.

KRaft architecture:

- Controller quorum: 3 or 5 broker nodes designated as controllers (can be separate from data nodes).
- Active controller: One controller is the leader (elected via Raft). Handles all metadata writes.

- Metadata log: All cluster metadata events appended to `__cluster_metadata` partition.
- Brokers: Register with controllers, receive metadata updates via fetch from `__cluster_metadata`.

Migration from ZooKeeper to KRaft: Kafka 3.x supports a migration mode — run ZooKeeper and KRaft simultaneously during transition, then cut over. Tools: `kafka-metadata-migration.sh`.

For Deepta AI on GCP: Confluent Cloud and GCP Pub/Sub (as Kafka alternative) already abstract infrastructure. If running self-managed Kafka on GKE, use KRaft mode from the start for simpler operations.

IMPLEMENTATION CHALLENGE:

Set up a KRaft-mode Kafka cluster for Deepta AI:

1. Configure a 3-node KRaft cluster: 1 combined controller+broker, 2 pure brokers. Write `server.properties` for KRaft mode (`process.roles=broker,controller`, `controller.quorum.voters=1@broker1:9093,2@broker2:9093,3@broker3:9093`).
2. Generate cluster UUID: `kafka-storage.sh random-uuid`. Format storage: `kafka-storage.sh format -t <uuid> -c server.properties`.
3. Start all 3 nodes and verify cluster is active: `kafka-metadata-quorum.sh --bootstrap-server localhost:9092 describe --status`.
4. Create a topic, produce 1000 messages, kill the active controller broker. Measure failover time (how quickly can you produce again?). Compare with ZooKeeper-mode failover time.
5. Observe `__cluster_metadata` topic: `kafka-dump-log.sh --files __cluster_metadata/`. Identify records for: `BrokerRegistration`, `TopicRecord`, `PartitionRecord`.
6. Discuss: In Deepta AI's GKE deployment, how would you run a 3-controller KRaft cluster with separate data brokers using `StatefulSets`?

FOLLOW-UP PROBES:

- What is the Raft consensus algorithm at a high level? How does KRaft use it for leader election among Kafka controllers?
- In KRaft, what happens if the active controller crashes and a new controller is elected? What is the maximum staleness of the new controller's view of the cluster?
- What is the `__cluster_metadata` topic and why can't you delete it? What happens if it becomes corrupted?
- Kafka KRaft has a quorum of 3 controllers. A network partition splits the cluster into `[controller1]` | `[controller2, controller3]`. Which side continues serving? Why?

[M] MID Category 09 > Kafka > Schema Registry

Q: What is the Schema Registry and why is it critical for production Kafka? Compare Avro, Protobuf, and JSON schema.

ANSWER:

Schema Registry: A centralized repository for message schemas. Producers register schemas; the Schema Registry assigns a schema ID. Messages sent to Kafka include only the schema ID (4 bytes), not the full schema. Consumers look up the schema by ID to deserialize.

Why Schema Registry is critical:

1. Schema evolution: Producer and consumers can evolve schemas independently. Old consumers can still read new messages (if backward compatible). New consumers can read old messages (if forward compatible).
2. Contract enforcement: Prevents a bad producer from sending malformed data that breaks all consumers.
3. Storage efficiency: Instead of embedding schema in every message (wasteful), only a 4-byte schema ID is included.
4. Discoverability: Developers can browse schemas to understand the data model.

Schema compatibility modes:

- BACKWARD: New schema can read data written with old schema (default). Consumers upgrade first, producers second.
- FORWARD: Old schema can read data written with new schema. Producers upgrade first, consumers second.

- FULL: Both backward and forward compatible.
- NONE: No compatibility check.

Avro: Binary serialization format, schema defined in JSON. Compact binary encoding (no field names in the binary — schema required to decode). Schema evolution built-in (default values for new fields). Best for high-throughput pipelines. Confluent Schema Registry was originally designed for Avro.

Protobuf: Google's binary serialization. Field numbers instead of names (very compact). Strong typing, code generation in multiple languages. Schema evolution via field number preservation. Better cross-language support than Avro. Supported by Confluent Schema Registry since Kafka 2.6.

JSON Schema: Schema for JSON messages. Human-readable, easy debugging. Much larger message size than Avro/Protobuf. Lower throughput. Use when human readability during development outweighs efficiency.

For Deepta AI:

- Core high-throughput pipeline (application events): Protobuf — cross-language (Go producers, Python ML consumers), compact binary, strong typing.
- Internal Go-to-Go services: Could use Avro for Schema Registry integration.
- Webhook ingestion (external third-party events): JSON Schema — third parties can't use Avro/Protobuf.

IMPLEMENTATION CHALLENGE:

Implement Schema Registry integration for Deepta AI's application event pipeline:

1. Define a Protobuf schema for ApplicationSubmittedEvent: (string application_id, string student_id, string university_id, int64 timestamp_ms, string lead_source, map<string, string> metadata).
2. Register the schema with Confluent Schema Registry via REST API: POST /subjects/student.applications-value/versions.
3. Go producer: use the confluent-kafka-go Schema Registry client to serialize with schema ID prefix.
4. Evolve the schema: add an optional field string counselor_id. Register v2. Verify backward compatibility check passes. Attempt to add a required field (breaking change) — verify the registry rejects it.
5. Go consumer: deserialize by reading schema ID from message prefix, fetching schema from registry, deserializing.
6. Demonstrate cross-version compatibility: v1 producer message consumed by v2 consumer (counselor_id is absent → default empty string). v2 producer message consumed by v1 consumer (counselor_id ignored gracefully).

FOLLOW-UP PROBES:

- What is the 'magic byte' in the Confluent wire format? What are the 5 bytes prepended to every Schema Registry-serialized message?
- What is a schema registry subject and what are the two naming strategies (TopicNameStrategy vs RecordNameStrategy)? When does RecordNameStrategy help?
- Your Deepta AI pipeline has 10 topics, each with their own schema. You need to add a trace_id field to ALL schemas for distributed tracing. How do you manage this migration across 10 schemas?
- The Schema Registry goes down. Your Kafka producers cache schemas locally. What happens to producers and consumers during the outage?

[M] MID Category 09 > Kafka > Deepta AI Pipeline

Q: Walk through Deepta AI's Kafka pub/sub pipeline. How would you design it for exactly-once, multi-consumer, and observability?

ANSWER:

This question leverages Gautham's actual Deepta AI experience. Let's design the full pipeline:

Event taxonomy for Deepta AI's student application platform:

- application.submitted — student submits application
- application.document_uploaded — student uploads a required document
- application.status_changed — counselor updates application status

- lead.received — webhook from Google Ads/Facebook/IVR
- notification.requested — any service requests a notification be sent
- notification.sent — notification delivery confirmed

Topic design:

- 1 topic per event type (clean separation, independent retention policies)
- application.* topics: 12 partitions (parallel processing), retention=7 days, replication=3
- lead.received: 6 partitions, retention=30 days (lead history matters)
- notification.*: 6 partitions, retention=24h (fire-and-forget pattern)

Consumer groups per pipeline stage:

- application.submitted → "application-validator" (validates completeness)
- application.submitted → "lead-attributor" (marks lead as converted)
- application.submitted → "notification-dispatcher" (sends confirmation email)
- application.status_changed → "status-notifier" (emails student on status change)
- lead.received → "lead-deduplicator" (checks Redis Bloom filter, deduplicates, writes to PostgreSQL)

Partition key design:

- application.submitted: key=student_id — all events for a student in same partition (ordering)
- lead.received: key=phone_number or email — deduplication requires same-partition processing
- application.status_changed: key=application_id — ordering per application

Exactly-once for status changes (financial-grade operations):

- Kafka transactions: transactional.id=status-processor-{pod_id}
- BEGIN transaction; produce to application.status_changed; commit offset from upstream; END transaction
- If producer restarts, incomplete transaction is aborted on startup

Observability:

- Consumer lag monitoring: custom Prometheus exporter that queries Kafka AdminClient for lag per group/topic/partition
- End-to-end latency: each message includes produce_timestamp; consumer records (consume_timestamp - produce_timestamp)
- Dead letter queue: failed messages after 3 retries sent to *.dlq topic with error context
- Kafka UI (Conduktor/Redpanda Console) for human inspection

IMPLEMENTATION CHALLENGE:

Implement the complete Deepta AI Kafka pipeline for application processing in Go:

1. Shared event types in Go struct: ApplicationSubmittedEvent, ApplicationStatusChangedEvent with JSON tags and Kafka key methods.
2. Producer service: GoLang struct with confluent-kafka-go producer, publishes to correct topic with correct key, includes tracing headers (trace_id, span_id as Kafka headers).
3. Application validator consumer: reads application.submitted, validates completeness (all required documents present via PostgreSQL check), publishes application.validation_result with status=valid/invalid.
4. Status notifier consumer: reads application.status_changed, sends email notification (mock), uses idempotency key in Redis to avoid duplicate sends.
5. Dead letter queue: after 3 failures, produce to application.submitted.dlq with original message + error_reason + attempt_count in headers.
6. Prometheus metrics: kafka_consumer_lag{group, topic, partition}, kafka_messages_processed_total{topic, status}, kafka_end_to_end_latency_ms{topic}.

FOLLOW-UP PROBES:

- You have 3 consumer groups all consuming application.submitted. A new consumer group 'audit-logger' joins. From where does it start consuming (earliest, latest)?
- Your application.submitted consumer group has 12 consumers for 12 partitions. You add a 13th partition. What happens before and after the rebalance?
- A counselor updates an application status, triggering application.status_changed. The status-notifier consumer processes it and sends an email. The consumer crashes before committing the offset. The email is sent again on redelivery. How do you prevent the duplicate email?

- Deepta AI needs to comply with GDPR: delete all data for a student. Their application events are in Kafka with 7-day retention. How do you handle event data that must be deleted on request but exists in an immutable log?

[A] ADVANCED Category 09 > Kafka > Exactly-Once with PostgreSQL

Q: Implement a Kafka consumer that guarantees exactly-once semantics when writing to PostgreSQL. Handle all failure scenarios.

ANSWER:

True exactly-once semantics between Kafka and an external system like PostgreSQL requires careful design. Kafka's transactional API handles Kafka-to-Kafka exactly-once. For Kafka-to-PostgreSQL, you must manage idempotency at the application layer.

The core problem: Kafka offset commit and PostgreSQL write are two separate operations. Any combination of orderings can fail:

- Write to PG succeeds, offset commit fails → re-read and duplicate write
- Write to PG fails, offset commit succeeds → message lost
- Both fail → safe, will retry

Strategy 1: Offset stored in PostgreSQL (the pattern from Q4):

Store Kafka offset in the same PostgreSQL transaction as the business data. On consumer restart, seek to the offset from PostgreSQL, not from Kafka.

Failure scenarios handled:

- PG write fails, offset not updated → on retry, same message, idempotent insert handles duplicate
- PG write succeeds, offset updated → on retry (shouldn't happen but if it does), idempotent insert ignores duplicate
- Process crashes mid-transaction → PG rolls back both writes → on restart, re-reads same message

Idempotency implementation for each operation type:

1. INSERT: Include event_id (Kafka message key + offset) as a UNIQUE key in the destination table.

INSERT INTO applications (... , kafka_event_id) VALUES (... , \$event_id) ON CONFLICT (kafka_event_id) DO NOTHING

2. UPDATE: Update is naturally idempotent if you update to an absolute value (not relative). SET status=\$new_status is idempotent. SET count = count + 1 is NOT idempotent.

3. Financial operations: Use a ledger pattern — never update balances, always insert credit/debit transactions with unique event_id. Balance = SUM of transactions. Re-inserting a transaction with the same event_id → ON CONFLICT DO NOTHING.

Strategy 2: Outbox pattern (for Kafka-initiated writes):

Don't write to Kafka from the consumer. Instead, write to a PostgreSQL outbox table in the same transaction as the business data. A separate CDC process reads the outbox and publishes to Kafka. This inverts the dependency — Kafka is downstream of PostgreSQL, not the source of truth.

Strategy 3: Saga pattern with compensation: For complex multi-step operations, use a saga. If a step fails, execute compensating transactions. This doesn't require exactly-once but achieves eventual consistency.

Failure scenario matrix for Deepta AI's application processing:

- application.submitted received, application already exists in DB: ON CONFLICT DO NOTHING on application_id + kafka_event_id
- application.submitted received, notification already sent (duplicate delivery): check Redis idempotency key SETNX notification:sent:<event_id> 1 EX 3600 before sending
- consumer killed after PG write, before offset stored: on restart, re-reads event, PG write is idempotent

IMPLEMENTATION CHALLENGE:

Build the exactly-once processor for Deepta AI's application submission pipeline:

1. Schema: applications table (id UUID PK, student_id UUID, university_id UUID, kafka_event_id TEXT UNIQUE, status TEXT, ...) + kafka_offsets table (topic TEXT, partition INT, consumer_group TEXT, offset BIGINT, PK(topic,partition,consumer_group)).

2. Go consumer with exactly-once guarantee:

- On startup: `SELECT offset FROM kafka_offsets WHERE topic=$1 AND partition=$2 AND consumer_group=$3`. Seek to offset+1.

- Per message processing:

`BEGIN`

`INSERT INTO applications (... , kafka_event_id) VALUES (... , $event_id) ON CONFLICT (kafka_event_id) DO NOTHING`

`INSERT INTO kafka_offsets (...) VALUES (...) ON CONFLICT (...) DO UPDATE SET offset=$offset`

`COMMIT`

- No Kafka offset commit — fully managed in PostgreSQL.

3. Chaos test: run 10,000 messages through. Randomly kill the consumer at random points. Restart. Verify: exactly 10,000 unique application rows in PostgreSQL. Zero duplicates. Zero gaps.

4. Benchmark: exactly-once (PG transaction per message) vs at-least-once (batch commit every 100 messages). Compare throughput. At what batch size does the exactly-once overhead become acceptable?

5. Handle the PostgreSQL transaction failure: if the application table has a constraint violation (non-idempotency related), the transaction rolls back. Write this error to the DLQ topic and advance the offset (skip this message after manual review).

FOLLOW-UP PROBES:

- The exactly-once pattern requires a PostgreSQL transaction per Kafka message. At 10,000 messages/second, this is 10,000 TPS on PostgreSQL. At what point does this become unscalable and how do you batch while maintaining exactly-once?
- What is the outbox pattern and how does it guarantee at-least-once delivery from PostgreSQL to Kafka? How does Debezium implement the CDC read of the outbox?
- Your exactly-once consumer has been down for 12 hours. Kafka topic has 7-day retention. On restart, it must replay 12 hours of backlog. How do you handle this while the live system continues writing new events?
- The Saga pattern for distributed transactions uses compensating transactions instead of exactly-once. For Deepta AI's scholarship disbursement (Kafka message → debit student account → credit university account), design the saga with compensation steps.

[A] ADVANCED Category 09 > Kafka > Monitoring & Consumer Lag

Q: How do you monitor a production Kafka cluster? What metrics matter most and how do you detect and respond to consumer lag?

ANSWER:

Production Kafka monitoring requires three layers: broker health, producer health, and consumer health.

BROKER METRICS (via JMX or Prometheus JMX exporter):

Critical:

- `UnderReplicatedPartitions`: Number of partitions with fewer replicas than configured. Should be 0. Any non-zero value = degraded durability.
- `ActiveControllerCount`: Should be exactly 1 across the cluster. 0 = no controller (cluster issue). >1 = split-brain.
- `OfflinePartitionsCount`: Partitions with no leader. Any non-zero = data unavailable for those partitions.
- `RequestHandlerAvgIdlePercent`: Broker thread pool utilization. If < 30%, broker is at capacity.
- `NetworkProcessorAvgIdlePercent`: Network thread saturation.
- `BytesInPerSec` / `BytesOutPerSec`: Throughput per broker and per topic. Detect hot brokers.
- `LogFlushRate` / `LogFlushTime`: Disk I/O. High values indicate disk bottleneck.
- `ISRShrinks` / `ISRExpands`: Frequency of ISR changes. High shrink rate indicates follower lag.

PRODUCER METRICS:

- `record-error-rate`: Non-zero = producer dropping messages.
- `request-latency-avg`: Round-trip time to broker.

- batch-size-avg: If consistently below batch.size, your batching is not effective.
- compression-rate-avg: Compression effectiveness.

CONSUMER METRICS — Consumer Lag (most critical):

Consumer lag = Latest offset (end of partition) – Consumer's committed offset.

Lag = 0 → consumer is real-time.

Lag > threshold → consumer is falling behind → alert.

How to measure:

kafka-consumer-groups.sh --bootstrap-server broker:9092 --group my-group --describe

Programmatically: AdminClient.listConsumerGroupOffsets() vs AdminClient.listOffsets(LATEST)

Lag interpretation:

- Sudden spike in lag: Consumer processing slowed (DB contention, external API latency)
- Steadily growing lag: Consumer throughput < producer throughput → need more consumer instances
- Lag on only some partitions: Hot partition issue or a specific consumer instance is slow

Alert thresholds for Deepta AI:

- WARNING: lag > 10,000 messages per partition for > 5 minutes
- CRITICAL: lag > 100,000 messages or lag growth rate > 1,000 messages/minute with no downward trend

CONSUMER LAG RESPONSE:

1. Growing lag → scale out: add consumer instances (up to partition count)
2. Processing bottleneck → profile the consumer code, check DB query times
3. Backlog after outage → temporarily increase max.poll.records, reduce processing complexity
4. Structural: topic may need more partitions for permanent throughput increase

IMPLEMENTATION CHALLENGE:

Build a complete Kafka observability stack for Deepta AI:

1. Prometheus JMX exporter: deploy as a sidecar on each Kafka broker pod. Expose UnderReplicatedPartitions, ActiveControllerCount, BytesInPerSec per topic.
2. Consumer lag exporter in Go: every 30 seconds, use AdminClient to compute lag for all Deepta AI consumer groups. Emit Prometheus gauge: kafka_consumer_lag{group, topic, partition}.
3. Grafana dashboard: panels for UnderReplicatedPartitions, lag per consumer group (stacked by partition), BytesInPerSec by topic, end-to-end message latency histogram.
4. Alertmanager rules:
 - CRITICAL: kafka_controller_active_count != 1
 - WARNING: kafka_under_replicated_partitions > 0 for 2 minutes
 - WARNING: kafka_consumer_lag{group="application-processor"} > 50000
 - CRITICAL: kafka_consumer_lag{group="application-processor"} > 500000
5. Auto-scaling consumer response: When lag > 50,000, trigger a Kubernetes HPA scale-out. When lag < 1,000, scale back in. Write the custom HPA metric adapter that reads from your lag Prometheus metric.
6. Alerting runbook: for each alert, write the exact investigation steps (which kafka-consumer-groups.sh command, which Grafana panel, which Go log grep).

FOLLOW-UP PROBES:

- Consumer lag is 0 but messages are taking 30 seconds end-to-end from producer to consumer processing completion. What is happening and how do you measure end-to-end latency?
- Your consumer lag exporter shows lag = 0 but users report delayed application status updates. What other metrics do you check?
- What is the __consumer_offsets topic? What happens if it becomes full or corrupted?
- Kafka's kafka-consumer-groups.sh shows 'UNKNOWN' for committed offset on a new consumer group. What does this mean and what is the auto.offset.reset setting doing?

Q: Design a Kafka-based event streaming platform for Deepta AI that processes 10 million application events per day, guarantees exactly-once delivery to PostgreSQL, supports replay, and handles GDPR deletion requests for student data.

ANSWER:

This is a Principal-level design combining Kafka architecture, exactly-once semantics, GDPR compliance, and operational excellence.

SCALE CALCULATIONS:

10M events/day = 116 events/second average. Peak (admission deadlines, 10x): 1,160 events/second.

With 1KB average message size: 1.16 MB/s average, 11.6 MB/s peak.

TOPIC DESIGN:

application.events (raw events, all types): 24 partitions (handles 500 msg/s per partition with headroom).

retention.ms=604800000 (7 days). replication.factor=3, min.insync.replicas=2.

application.events.processed (confirmed processed events): 24 partitions, compacted. Used for downstream consumers that need current state.

application.events.dlq (dead letter): 6 partitions, retention=30 days for manual inspection.

gdpr.deletion.requests: 6 partitions, retention=infinite (compliance record), compacted.

EXACTLY-ONCE TO POSTGRES:

Use the outbox pattern for application writes:

1. Application API writes to PostgreSQL (applications table + outbox table) in one transaction.
2. Debezium CDC reads outbox table changes → publishes to application.events Kafka topic.
3. Downstream consumers process application.events → write results to their databases with kafka_event_id deduplication.

This inverts the Kafka dependency: Kafka is never the primary source of truth. If Kafka is down, the API still works (outbox accumulates). Debezium catches up when Kafka recovers.

GDPR DELETION DESIGN:

Kafka's immutable log makes GDPR deletion fundamentally challenging. Options:

1. Crypto-shredding: Encrypt PII fields with a per-student key stored in a key management service (GCP KMS). On GDPR deletion: delete the encryption key. All messages with that student's PII become unreadable (ciphertext without key). Kafka log physically remains but is meaningless. This is accepted GDPR practice.
2. Tombstone + reprocessing: Publish a tombstone for the student_id in a compacted topic. Downstream systems that built state from Kafka events must reprocess with tombstone awareness to delete the state. Kafka's raw log is still retained (use short retention + crypto-shredding for raw events).
3. PII-free Kafka messages: Never put raw PII in Kafka. Messages contain only IDs (student_id, application_id). PII lives in PostgreSQL (which supports DELETE). Kafka is PII-free by design — no GDPR conflict.

Recommendation for Deepta AI: Design 3 (PII-free Kafka) + crypto-shredding for any metadata that could be PII. The Kafka topic contains only UUIDs and event metadata. PII (name, email, passport number) lives only in PostgreSQL which you can DELETE.

REPLAY ARCHITECTURE:

Consumers can reset offsets to replay historical events:

- Replay queue: a separate consumer group specifically for replay operations
- Replay coordinator: manages which offset range to replay, monitors replay completion
- Idempotent consumers: replay is safe because all consumers handle duplicates (ON CONFLICT DO NOTHING pattern)

OPERATIONAL EXCELLENCE:

- Schema Registry with compatibility enforcement (FULL compatibility for core events)
- Consumer lag alerting per group/topic/partition
- Distributed tracing: every message carries trace_id as a Kafka header
- Chaos engineering: monthly broker kill tests to validate failover
- Capacity planning: monitor BytesInPerSec trend, project partition count increases

IMPLEMENTATION CHALLENGE:

Implement the complete platform:

1. Outbox pattern in Go: `ApplicationService.SubmitApplication()` opens a PostgreSQL transaction, inserts application + outbox row (`id`, `aggregate_type`, `aggregate_id`, `event_type`, `payload_json`, `created_at`, `processed=false`), commits.
2. Debezium connector config: watches `public.outbox` table, uses transform to extract `aggregate_type` as Kafka topic routing key. Produces to `application.events.{aggregate_type}`.
3. Crypto-shredding key service: on each `student_id`, create a data encryption key (DEK) in GCP KMS. Encrypt sensitive fields in the outbox `payload_json`. Store `key_id` reference in message. On GDPR deletion: destroy DEK in KMS. Old messages become opaque.
4. Consumer with idempotency: reads `application.events.application`, processes with `ON CONFLICT (kafka_event_id) DO NOTHING`, writes to `kafka_offsets` in same PG transaction.
5. Replay service: `AdminClient.AlterConsumerGroupOffsets()` to reset a consumer group to a specific timestamp offset (`OffsetForTimes`). Trigger replay for a specific university's data to rebuild a derived view.
6. GDPR deletion workflow: receive `gdpr.deletion.requests` event → delete PII from all PostgreSQL tables → destroy encryption keys for that `student_id` in KMS → write `gdpr.deletion.confirmed` event with timestamp (audit trail).

FOLLOW-UP PROBES:

- The outbox pattern adds a step (write to outbox → CDC → Kafka) vs writing directly to Kafka. What is the added latency and is it acceptable for Deepta AI's use case?
- Your Debezium connector falls behind by 2 hours (slot lag). The outbox table accumulates 10M unprocessed rows. What is the impact on PostgreSQL and how do you recover?
- Crypto-shredding destroys the encryption key on GDPR deletion. But your Kafka topic retention is 7 days. A consumer replaying 3-day-old messages encounters the student's encrypted data. What does the consumer see and how should it handle it?
- You need to replay 30 days of events for a new consumer group (audit-logger). The Kafka topic has 7-day retention. The events older than 7 days are gone. How do you provide the missing history?

CATEGORY 10

DEVOPS, KUBERNETES & CLOUD

Docker · GKE Architecture · CI/CD · Observability · GCP · Zero-Downtime

13 questions · [E]=6 [M]=5 [A]=1 [P]=1

TOPICS COVERED IN THIS CATEGORY:

- Docker internals: namespaces, cgroups, Union filesystem/OverlayFS, image layers
- Multi-stage builds and Dockerfile best practices for Go microservices
- Kubernetes control plane: API server, etcd, scheduler, controller manager
- Data plane: kubelet, kube-proxy, CNI; pod lifecycle, QoS classes
- Deployments vs StatefulSets vs DaemonSets
- Services: ClusterIP, NodePort, LoadBalancer, Ingress
- ConfigMaps/Secrets, resource limits/requests, HPA/VPA/KEDA
- Rolling updates vs blue-green vs canary; PodDisruptionBudgets
- Health probes: liveness, readiness, startup
- GKE-specific: Workload Identity, node pools, cluster autoscaler, Autopilot vs Standard
- CI/CD: GitHub Actions patterns, GitOps with ArgoCD, Workload Identity Federation
- Observability: RED method, USE method, Prometheus, OpenTelemetry tracing, structured logs
- Zero-downtime cluster migration for stateful Go services
- Principal: 99.95% SLO production infrastructure for Deepta AI on GCP with DPDP compliance

[E] EASY Category 10 > Docker > Internals

Q: Explain how Docker works under the hood: namespaces, cgroups, and Union filesystem. What is a container vs a VM?

ANSWER:

Docker containers are not VMs — they are processes running on the host kernel with strong isolation mechanisms.

Linux Namespaces: Isolate what a process can SEE.

- PID namespace: Container process sees its own PID tree (its PID 1 is actually a higher PID on the host)
- Network namespace: Container has its own network interfaces, routing table, iptables rules
- Mount namespace: Container sees its own filesystem mount tree (its root is the container image)
- UTS namespace: Container has its own hostname and domain name
- IPC namespace: Container has its own inter-process communication (shared memory, semaphores)
- User namespace: Container can have its own user/group ID mapping (allows running as UID 0 inside but a non-root UID on the host)

cgroups (Control Groups): Limit what a process can USE (resources).

- cpu: Limit CPU shares or set hard quotas (cpu.cfs_quota_us)
- memory: Limit RAM usage. Container OOM-killed if it exceeds limit.
- blkio: Limit disk I/O bandwidth
- net_cls / net_prio: Network traffic classification

Union Filesystem (OverlayFS): Docker images are composed of read-only layers stacked together. Each Dockerfile instruction (RUN, COPY, ADD) creates a new layer. The container gets a thin writable layer on top.

- Lower layers (image layers): Read-only, shared across all containers using the same image
- Upper layer (container layer): Writable, unique per container. Deleted on container stop.

- Copy-on-write: When a container writes to a file from a lower layer, it's first copied to the upper layer (the "write" goes to the copy).

Container vs VM:

- VM: Full guest OS, hypervisor, hardware virtualization. ~100-500ms startup. 100s of MB overhead per VM.
- Container: Shares host kernel, uses namespaces/cgroups. ~50ms startup. MB-level overhead.
- Security: VMs stronger isolation (separate kernel). Containers share kernel — a kernel exploit can potentially escape the container.
- Use case: VMs for multi-tenant cloud compute (untrusted workloads). Containers for microservices (trusted application code).

In Deepta AI's GKE: Go microservices run as containers on GKE worker nodes. The node is a VM (GCE instance), providing a trusted boundary. Multiple containers share the node's kernel.

IMPLEMENTATION CHALLENGE:

Run a Docker internals experiment:

1. Start an nginx container: `docker run -d --name web nginx`
2. Find the host PID for the nginx process: `docker inspect web --format '{{.State.Pid}}'`. Then: `ls -la /proc/<pid>/ns/` — observe the namespace symlinks.
3. Enter the network namespace without Docker: `nsenter -t <pid> -n ip addr` — observe the container's network interfaces from the host.
4. Observe cgroups: `cat /sys/fs/cgroup/memory/docker/<container_id>/memory.limit_in_bytes`.
5. Docker build a Deepta AI Go service image. Use `docker history` to inspect layers. Show which layers are reused when you only change application code vs when you change dependencies.
6. Demonstrate OverlayFS: `docker inspect web --format '{{.GraphDriver}}'`. Mount the container's overlay filesystem and show the layer structure.

FOLLOW-UP PROBES:

- What is the difference between COPY and ADD in a Dockerfile? When does ADD have hidden behavior?
- What is a Docker layer cache and how does the order of Dockerfile instructions affect build time? Why should COPY package.json come before COPY . . in a Node.js image?
- What is the security risk of running a container as root? How do USER instruction and user namespaces mitigate this?
- Your Deepta AI Go service Docker image is 1.2GB. How do you reduce it to under 20MB using multi-stage builds and a distroless base image?

[E] EASY Category 10 > Docker > Multi-Stage Builds

Q: What are multi-stage Docker builds and why are they essential for Go microservices? Write an optimized Dockerfile for a Go service.

ANSWER:

Multi-stage builds allow a Dockerfile to use multiple FROM instructions, each defining a separate build stage. The final image only contains what you explicitly copy from earlier stages — build tools, source code, and intermediate artifacts are left behind.

Why critical for Go:

- Go compiles to a single statically linked binary. The build requires the Go toolchain (~600MB) but the runtime needs only the binary itself.
- Without multi-stage: image = Go toolchain + source + binary = 600MB+
- With multi-stage: image = binary + minimal base (or nothing) = <20MB

Anatomy of an optimized Go Dockerfile:

Stage 1 (builder):

FROM golang:1.22-alpine AS builder

WORKDIR /app


```

COPY go.mod go.sum ./
RUN go mod download          # Layer cache: only re-runs if go.mod/go.sum change
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags="-w -s" -o server ./cmd/server
Stage 2 (final):
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/server /server
COPY --from=builder /app/configs /configs
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]

```

Key optimizations:

- Copy go.mod/go.sum first, RUN go mod download before COPY . — dependency layer cached unless deps change
- CGO_ENABLED=0: disable C bindings → fully static binary → works in scratch/distroless
- GOOS=linux GOARCH=amd64: explicit cross-compilation
- ldflags="-w -s": strip debug symbols and DWARF info → 30-40% smaller binary
- distroless/static: minimal base with no shell, package manager, or OS utilities — attack surface nearly zero
- USER nonroot: never run as root in production
- .dockerignore: exclude .git, vendor, *.md, test files from build context

For Deepta AI's Go microservices on GKE: Each service gets a <20MB image. GKE image pull time drops from 30s (1.2GB) to <2s. Cold start time (pod startup) improves. Security scanning finds zero OS vulnerabilities in distroless.

IMPLEMENTATION CHALLENGE:

Create an optimized Dockerfile for Deepta AI's student application Go microservice:

1. Application code: main.go, handlers/, kafka/, postgres/, redis/ packages. Has external deps: confluent-kafka-go (requires CGO), go-redis, pgx.
2. Handle the CGO complication: confluent-kafka-go requires CGO_ENABLED=1 and librdkafka. Use golang:1.22-bullseye as builder, install librdkafka-dev. Use debian:bullseye-slim as final (not distroless — needs libc for CGO).
3. Build and compare image sizes: naive (FROM golang:1.22) vs multi-stage (from debian:bookworm-slim) vs replacing confluent-kafka-go with franz-go (pure Go, CGO_ENABLED=0 → distroless possible).
4. Add a .dockerignore that excludes: .git, *.md, *_test.go, testdata/, vendor/, .env files.
5. Add a health check: HEALTHCHECK --interval=30s --timeout=5s CMD ["/server", "health"] or curl http://localhost:8080/health.
6. Show layer sizes with docker image history and calculate total final image size.

FOLLOW-UP PROBES:

- Your Go service uses CGO (for librdkafka). Can you use a scratch base image? What are your options for minimizing the final image?
- What is .dockerignore and what happens to build context size and build time if you forget it in a large repo?
- How do you pass environment-specific configuration (database URLs, API keys) into a container at runtime without baking them into the image?
- Your Deepta AI CI pipeline builds the Docker image on every push. Build time is 8 minutes because go mod download re-runs every time. How do you fix this with layer caching in GitHub Actions?

[E] EASY Category 10 > Kubernetes > Architecture

Q: Explain Kubernetes architecture: control plane components and data plane (node) components. What does each component do?

ANSWER:

Kubernetes is a container orchestration system. It consists of a control plane (cluster brain) and data plane (worker nodes where containers run).

CONTROL PLANE (runs on master nodes):

kube-apiserver: The single entry point for all cluster operations. All kubectl commands, controller queries, and kubelet registrations go through the API server. Validates and persists objects to etcd. Horizontally scalable.

etcd: A distributed key-value store (uses Raft consensus). Stores all cluster state: pod specs, service definitions, config maps, secrets, node registrations. The only stateful component of the control plane. Loss of etcd = loss of cluster state. Typically 3 or 5 nodes for HA.

kube-scheduler: Watches for unscheduled pods (pods with no nodeName assigned). Selects the best node for each pod based on: resource requests/limits, node selectors, affinity/anti-affinity rules, taints/tolerations. Does NOT schedule the pod — marks it assigned, and kubelet does the actual work.

kube-controller-manager: Runs multiple controller loops:

- Deployment controller: ensures desired replica count matches actual
- ReplicaSet controller: creates/deletes pods to match replicas
- Node controller: marks nodes as unhealthy if heartbeats stop
- Service Account controller: creates default service accounts
- Job controller, CronJob controller, etc.

cloud-controller-manager: GCP/AWS/Azure-specific controllers (load balancer provisioning, node lifecycle integration with cloud APIs). In GKE: handles GCP Load Balancer creation for Services, node autoscaling.

DATA PLANE (worker nodes):

kubelet: The "node agent" running on every worker node. Watches the API server for pods assigned to its node. Pulls container images, creates containers via CRI (Container Runtime Interface — e.g., containerd). Reports pod status back to API server. Runs health checks (liveness/readiness probes).

kube-proxy: Maintains network rules on each node (iptables or ipvs rules). Implements Service abstraction — routes traffic from ClusterIP/NodePort to the correct pod endpoints. Watches API server for Service and Endpoints changes.

Container Runtime: Actually runs containers. GKE uses containerd (Docker was deprecated as Kubernetes runtime in 1.24). Implements CRI spec.

CNI (Container Network Interface) plugin: Sets up pod networking. Gives each pod a unique IP. Implements network policies. GKE uses Calico or Dataplane V2 (based on Cilium/eBPF).

IMPLEMENTATION CHALLENGE:

Explore Kubernetes architecture on a GKE cluster for Deepta AI:

1. `kubectl get nodes --` describe a node and identify: Allocatable CPU/Memory, Container Runtime Version, OS Image.
2. `kubectl get pods -n kube-system --` identify kube-proxy, CNI plugin, kube-dns pods.
3. `kubectl get componentstatuses --` verify scheduler, controller-manager, etcd health.
4. Trace a pod deployment: `kubectl apply` a Deployment. Use `kubectl get events --sort-by=.lastTimestamp` to see: scheduler assigned node, kubelet pulled image, container started.
5. Examine etcd: `kubectl exec` into etcd pod, use `etcdctl get /registry/pods/<namespace>/<pod-name>` to see raw pod spec in etcd.
6. `kubectl describe node <worker-node> --` observe Conditions (Ready, MemoryPressure, DiskPressure, PIDPressure), Capacity vs Allocatable, allocated resources.

FOLLOW-UP PROBES:

- What happens when the kube-apiserver goes down? Can existing pods continue running? Can you schedule new pods?
- What is etcd quorum and why do you need an odd number of etcd nodes (3 or 5, never 4)?
- The kube-scheduler selects a node but the pod stays in Pending. What are all the possible reasons?
- What is the difference between kube-proxy using iptables mode vs ipvs mode? When would you choose ipvs for Deepta AI's GKE cluster?

[E] EASY Category 10 > Kubernetes > Pod Lifecycle

Q: Explain the Kubernetes pod lifecycle, resource requests vs limits, and the three types of health probes.

ANSWER:

Pod Lifecycle Phases:

Pending: Pod accepted by the cluster, but containers not yet running. Could be waiting for: node scheduling, image pull, volume mounting.

Running: At least one container is running or starting/restarting.

Succeeded: All containers exited with code 0 (batch jobs).

Failed: At least one container exited non-zero or was OOM-killed.

Unknown: Node communication lost.

Container States: Waiting, Running, Terminated.

Resource Requests vs Limits:

requests: Minimum resources guaranteed to the container. Kubernetes scheduler uses this to find a node with sufficient available capacity. A pod's request must be $\leq$ node's allocatable.

limits: Maximum resources the container can use. If CPU exceeds limit $\rightarrow$ throttled (not killed). If memory exceeds limit $\rightarrow$ OOM-killed (SIGKILL).

QoS Classes (determined by requests/limits):

- Guaranteed: requests == limits for all containers. Highest priority, last to be evicted.
- Burstable: requests < limits or only one set. Middle priority.
- BestEffort: No requests or limits set. First to be evicted under node pressure.

Deepta AI's Go microservices: Set requests and limits. Production recommendation: requests=limits (Guaranteed QoS) for predictable performance. This prevents noisy neighbor issues in a multi-tenant GKE cluster.

Health Probes:

Liveness probe: "Is this container alive or does it need to be killed and restarted?"

Failure action: kubelet restarts the container (respects restartPolicy).

Use for: detecting deadlocks, infinite loops, corrupted state that can't recover without restart.

CAUTION: Don't probe dependencies (DB, Redis) in liveness — if DB is down, you don't want to restart all pods.

Readiness probe: "Is this container ready to receive traffic?"

Failure action: kubelet removes the pod from Service Endpoints (no traffic, pod not killed).

Use for: warming up application (loading caches), waiting for DB connection to establish.

Start a pod with readiness failing, add to Service only when truly ready.

Startup probe: "Has the container started successfully?"

Failure action: If startup probe fails within $\text{failureThreshold} * \text{periodSeconds}$, container is killed.

Use for: slow-starting containers (JVM warm-up, large config loading). Disables liveness probe until startup probe succeeds.

Prevents premature liveness kills during initialization.

Deepta AI Go services: Implement /health/live (liveness — checks if process is responsive), /health/ready (readiness — checks DB, Redis, Kafka connectivity), and use startupProbe only if service has >10s initialization.

IMPLEMENTATION CHALLENGE:

Configure production-ready health probes and resource settings for Deepta AI's application service in Kubernetes:

1. Write a Kubernetes Deployment manifest with:

- resources: requests: {cpu: 200m, memory: 256Mi}, limits: {cpu: 1000m, memory: 512Mi}
- livenessProbe: HTTP GET /health/live, initialDelaySeconds=5, periodSeconds=10, failureThreshold=3
- readinessProbe: HTTP GET /health/ready, initialDelaySeconds=10, periodSeconds=5, failureThreshold=3, successThreshold=1

- startupProbe: HTTP GET /health/live, failureThreshold=30, periodSeconds=2 (60 second window for startup)
- 2. Implement the Go health endpoints:
 - /health/live: return 200 if server is running
 - /health/ready: ping PostgreSQL (SELECT 1), ping Redis (PING), check Kafka consumer is active. Return 503 if any dependency is down.
- 3. Simulate liveness failure: add an endpoint that causes the health check to return 500 after 60 seconds. Observe pod restart.
- 4. Simulate readiness failure: take down Redis. Observe: pod marked not ready, removed from Service endpoints, no traffic. Restore Redis: pod becomes ready again without restart.
- 5. Test OOM kill: allocate memory beyond limits in Go. Observe OOMKilled in pod status. Check kubectl describe pod for OOM events.

FOLLOW-UP PROBES:

- What happens if you set memory limit lower than the JVM/Go runtime's base memory? What is the symptom?
- Liveness probe is checking the database connection. The database goes down. All pods fail liveness and restart simultaneously. What problem does this cause and what is the correct design?
- What is a Burstable pod and what happens during node memory pressure — which pods are evicted first?
- Your Deepta AI pod has readiness probe checking Redis. During a Redis failover (30 seconds), all pods go not-ready and traffic is dropped. How do you design readiness to be more resilient?

[E] **EASY** Category 10 > Kubernetes > Services & Networking

Q: Explain Kubernetes Service types: ClusterIP, NodePort, LoadBalancer. How does kube-proxy implement ClusterIP using iptables?

ANSWER:

Kubernetes Services provide a stable network endpoint for a group of pods (selected by label selector). Pods come and go (IP changes) but the Service IP is stable.

ClusterIP (default): Assigns a virtual IP (the ClusterIP) that is only reachable from within the cluster. kube-proxy creates iptables rules on every node that DNAT traffic to the ClusterIP to one of the backing pods (randomly selected with equal probability by default, or IPVS for load balancing).

Example: Service my-service ClusterIP=10.96.0.1 port=80. Pod A sends request to 10.96.0.1:80. iptables on Pod A's node matches this → DNAT to one of: 10.244.0.5:8080, 10.244.1.3:8080 (actual pod IPs).

How iptables implements this:

1. PREROUTING chain: match ClusterIP:port → jump to service chain
2. Service chain: probabilistically select a pod endpoint (using iptables statistics module)
3. DNAT: rewrite destination IP:port to the selected pod's IP:port
4. Return path: conntrack handles response packet destination rewriting

NodePort: Extends ClusterIP by opening a port (30000-32767) on EVERY node in the cluster. External traffic can reach the service via <any-node-ip>:<nodeport>. kube-proxy adds rules to accept this port on the node's host network and DNAT to a pod.

Use case: Simple external access without a cloud load balancer. Not recommended for production (security, no load balancing across nodes).

LoadBalancer: Creates a cloud load balancer (GCP Load Balancer for GKE) that routes to NodePort on all nodes. The cloud-controller-manager provisions the GCP LB and updates the Service with the external IP.

Use case: Production external traffic ingress. Each Service gets its own LB (can be expensive for many services → use Ingress instead).

Ingress: An HTTP(S) reverse proxy managed by Kubernetes. One Ingress controller (e.g., nginx ingress, GKE Ingress) handles routing for multiple Services based on host/path rules. One GCP LB for all services. Much more cost-effective.

For Deepta AI: Internal Go microservices use ClusterIP. External API (student portal, university dashboard) uses Ingress backed by one GCP Load Balancer with TLS termination and path routing.

IMPLEMENTATION CHALLENGE:

Set up networking for Deepta AI's microservices on GKE:

1. Deploy: application-service (ClusterIP, port 8080), student-portal (LoadBalancer, port 443), api-gateway (NodePort for testing).
2. Configure an Ingress: route /api/applications → application-service, /api/students → student-service, / → student-portal. TLS via cert-manager (Let's Encrypt).
3. Verify ClusterIP iptables rules: kubectl exec into a pod, run iptables -t nat -L KUBE-SERVICES to observe service chains.
4. Network policy: deny all ingress to application-service by default. Allow ingress only from api-gateway namespace. Write the NetworkPolicy YAML.
5. Test service discovery: from within a pod, curl `http://application-service.default.svc.cluster.local:8080/health`. Show how DNS resolves to ClusterIP.
6. Trace a request through the networking stack: external request → GCP LB → NodePort → iptables DNAT → pod.

FOLLOW-UP PROBES:

- What is Kubernetes DNS and how does kube-dns (CoreDNS) resolve service names? What is the full DNS name for a Service?
- What is the difference between Ingress and a Gateway API (Kubernetes 1.21+)? Why is Gateway API considered the successor to Ingress?
- kube-proxy in iptables mode has O(N) performance for N services. Why? How does ipvs mode solve this?
- You have 200 Services in Deepta AI's cluster. A pod is making DNS queries at 100 req/s for service discovery. What is the load on CoreDNS and how do you reduce it with DNS caching?

[E] EASY Category 10 > Kubernetes > Deployments & Updates

Q: Compare rolling update, blue-green, and canary deployment strategies. How do you implement zero-downtime deployments in Kubernetes?

ANSWER:

Rolling update (Kubernetes default): Kubernetes replaces pods gradually. At any point, both old and new versions run simultaneously.

Configuration: `strategy.rollingUpdate.maxSurge=1` (max extra pods during update), `maxUnavailable=0` (always maintain desired replicas).

Process: Create 1 new pod, wait for it to be Ready, delete 1 old pod. Repeat.

Zero-downtime requirement: readiness probe must be configured so traffic only routes to pods that are Ready. Without readiness probe, traffic can reach not-yet-ready new pods.

Pros: Simple, built-in. Gradual rollout. Automatic rollback with `kubectl rollout undo`.

Cons: Old and new versions coexist — API must be backward compatible (consumers handle both response shapes). Canary-like (uncontrolled portion of traffic on new version). If new version is bad, N pods get bad before rollback triggers.

Blue-Green: Two identical environments (blue=current, green=new). Deploy to green, test, then instantly switch traffic by updating the Service selector.

With Kubernetes: Two Deployments (app-blue, app-green). Service selects app: app, version: blue. To switch: `kubectl patch service to version: green`.

Pros: Instant cutover. Easy rollback (patch back to blue). No version coexistence.

Cons: 2x resource cost during deployment. Cold-start (green is new, no warm caches).

Canary: Route a small percentage of production traffic to the new version for validation before full rollout.

Kubernetes implementation options:

1. Two Deployments (app-stable: 9 replicas, app-canary: 1 replica), same Service selector. ~10% of traffic goes to canary (based on replica ratio). Coarse-grained.

2. Ingress-based canary (nginx ingress annotation: `nginx.ingress.kubernetes.io/canary-weight: "10"`).

Precise percentage routing.

3. Service mesh (Istio/Linkerd): VirtualService with weight-based routing. Most precise, requires mesh.

GitOps canary with Flagger/Argo Rollouts:

- Argo Rollouts: CRD that extends Deployments with canary/blue-green steps. Define: "send 10% traffic, wait 5min, if error rate < 1% advance to 25%, etc."
- Automated rollback: if Prometheus metrics breach thresholds during canary, auto-rollback.

For Deepta AI's GKE: Use rolling update for most services. Use Argo Rollouts canary for the student-facing application API (critical path where bad deployments hurt students during admission season).

IMPLEMENTATION CHALLENGE:

Implement zero-downtime deployment for Deepta AI's application service:

1. Configure rolling update: `maxSurge=1`, `maxUnavailable=0`, `minReadySeconds=10` (pod must be ready for 10s before being considered stable).
2. Deploy v1. Trigger rolling update to v2. Use `kubectl rollout status` to monitor. Simultaneously run a load test (k6 or hey) — verify zero dropped requests during rollout.
3. Simulate a bad deployment: v2 returns 500s. Configure a readiness probe that fails if error rate > 10% (check an internal metric endpoint). Observe rolling update stall (`maxUnavailable=0` prevents removing good pods when new pods aren't ready).
4. Implement blue-green with a bash script: (a) deploy app-green, (b) wait for Ready, (c) patch Service selector, (d) delete app-blue after 60 seconds.
5. (Bonus) Install Argo Rollouts: define a Rollout with canary: steps: `[[setWeight: 10], {pause: {duration: 2m}}, {setWeight: 50}, {pause: {duration: 5m}}, {setWeight: 100}]`. Monitor analysis metrics.

FOLLOW-UP PROBES:

- What is the `minReadySeconds` field in a Deployment and why is it important for rolling updates?
- During a rolling update, your Service has 50% traffic on v1 and 50% on v2. A v2 bug causes 500 errors for POST /applications/submit but v1 handles them correctly. How do you detect this quickly and rollback?
- What does `kubectl rollout undo` do exactly? What version does it roll back to?
- Your Deepta AI application service has a database schema migration in v2 that v1 cannot read. Can you do a rolling update? What must be true about the migration?

[M] MID Category 10 > Kubernetes > StatefulSets & Storage

Q: When do you use StatefulSet vs Deployment? Explain persistent storage in Kubernetes: PV, PVC, and StorageClass.

ANSWER:

Deployment (stateless): All pods are identical and interchangeable. Pods can be replaced, rescheduled, or scaled without concern for state. Each pod gets a random name (app-7d4f8c9-xk2v1). If a pod dies, a new one with a different name takes its place.

StatefulSet (stateful): Pods have:

1. Stable network identity: pod names are deterministic (kafka-0, kafka-1, kafka-2) and stable across restarts
2. Stable storage: each pod has its own PVC that persists across pod restarts and rescheduling
3. Ordered deployment and scaling: pods are created/deleted in order (0 before 1, 1 before 2)
4. Ordered rolling updates: updates proceed from the highest ordinal down (2 → 1 → 0)

Use StatefulSet for: databases (PostgreSQL, MySQL), distributed systems (Kafka, ZooKeeper, etcd, Redis Cluster), any workload where pod identity or persistent storage matters.

PersistentVolume (PV): A piece of storage in the cluster, provisioned by an admin or dynamically. Lifecycle independent of any pod. Has: capacity, accessModes (ReadWriteOnce, ReadOnlyMany, ReadWriteMany), reclaim policy (Retain, Delete, Recycle).

PersistentVolumeClaim (PVC): A user's request for storage. Specifies: storage size, access mode, StorageClass. Kubernetes binds the PVC to a suitable PV (matching capacity, accessMode, StorageClass).

StorageClass: Defines the "class" of storage — the provisioner, parameters, reclaim policy. Enables dynamic provisioning (PV created automatically when PVC is created).

GKE StorageClasses:

- standard (default): GCP Persistent Disk, HDD
- standard-rwo (ReadWriteOnce): Balanced PD (SSD)
- premium-rwo: SSD PD
- filestore: NFS-like ReadWriteMany

VolumeClaimTemplates in StatefulSet: Each StatefulSet pod gets its own PVC from the template. kafka-0 gets data-kafka-0, kafka-1 gets data-kafka-1. On pod restart, the same PVC (and thus the same data) is reattached.

For Deepta AI on GKE:

- Go microservices: Deployments (stateless). No PVCs needed — state in Redis/PostgreSQL.
- Kafka brokers (self-managed): StatefulSets with premium-rwo StorageClass (SSD, ReadWriteOnce — Kafka needs fast disk I/O).
- PostgreSQL (Patroni): StatefulSets with premium-rwo PVCs.

IMPLEMENTATION CHALLENGE:

Deploy Kafka as a StatefulSet on GKE for Deepta AI:

1. Write the Kafka StatefulSet manifest: 3 replicas, each with a 100Gi premium-rwo PVC via volumeClaimTemplates.
2. Configure stable hostnames: Kafka requires stable DNS (kafka-0.kafka-headless.default.svc.cluster.local). Create a headless Service (clusterIP: None) to enable per-pod DNS.
3. Observe StatefulSet ordering: apply and watch `kubectl get pods --watch` — verify kafka-0 reaches Running before kafka-1 starts.
4. Test stable storage: write 100,000 messages to Kafka. Delete pod kafka-1. Observe new kafka-1 pod reattaches to same PVC (data-kafka-1) and resumes from correct state.
5. Scale StatefulSet from 3 to 5: observe pods created in order (kafka-3 then kafka-4).
6. Simulate volume expansion: increase PVC from 100Gi to 200Gi using GKE's `allowVolumeExpansion=true` StorageClass. Show online volume expansion without pod restart.

FOLLOW-UP PROBES:

- What happens to PVCs when a StatefulSet is deleted? How does this differ from reclaim policy=Delete vs Retain?
- Your StatefulSet has 3 Kafka pods. You update the image. In what order are pods updated and can you control this?
- What is a headless Service and why does Kafka require one? How is the DNS different from a regular Service?
- A Kafka broker pod (kafka-1) is OOMKilled repeatedly. The PVC has valid data. How do you debug and resolve without losing the data?

[M] MID Category 10 > Kubernetes > HPA & Autoscaling

Q: Explain Horizontal Pod Autoscaler (HPA). How do you configure custom metrics autoscaling for a Kafka-consumer workload?

ANSWER:

HPA (Horizontal Pod Autoscaler): Automatically scales the number of pod replicas based on observed metrics. Every 15 seconds (by default), the HPA controller queries the Metrics Server (or custom metrics adapter) and adjusts replicas.

Standard metrics:

- CPU: `targetCPUUtilizationPercentage=70` → scale out when average CPU > 70% of requests.
- Memory: `targetAverageValue=400Mi`.

Algorithm: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} * (\text{currentMetricValue} / \text{desiredMetricValue}))$

E.g.: 3 replicas, current CPU=90%, target=70% → $\text{desiredReplicas} = \text{ceil}(3 * 90/70) = \text{ceil}(3.86) = 4$.

HPA constraints:

- minReplicas, maxReplicas: bounds on scaling
- scaleDown stabilizationWindowSeconds (default 300s): wait 5 minutes before scaling down to prevent flapping
- scaleUp policies: can limit rate of scale-up

Metrics Server: In-cluster component that collects resource usage (CPU, memory) from kubelet. Required for HPA with default metrics. Lightweight, does NOT provide custom metrics.

Custom Metrics API: For scaling on non-resource metrics (Kafka consumer lag, HTTP request rate, queue depth). Requires a custom metrics adapter that bridges your monitoring system (Prometheus) to the Kubernetes Metrics API.

Kafka consumer autoscaling pattern:

- Metric: consumer lag (messages in queue / consumers = messages per consumer). Scale when lag > threshold.
- KEDA (Kubernetes Event Driven Autoscaling): The standard tool. KEDA provides a ScaledObject CRD:

ScaledObject:

triggers:

- type: kafka

metadata:

bootstrapServers: kafka:9092

consumerGroup: application-processor

topic: student.applications

lagThreshold: "50000" # scale when lag > 50k per partition

- KEDA scales the Deployment from 0 → N based on lag. Scales to 0 when no messages (cost saving).

For Deepta AI: HPA on CPU for the API gateway (traffic-driven). KEDA on Kafka consumer lag for application-processor workers (workload-driven). Set maxReplicas to partition count (no point having more consumers than partitions).

IMPLEMENTATION CHALLENGE:

Set up autoscaling for Deepta AI's services on GKE:

1. API Gateway HPA: apiVersion: autoscaling/v2, targetCPUUtilizationPercentage=60, minReplicas=2, maxReplicas=20. Deploy and load test with k6 (ramp 0 → 10,000 RPS). Observe HPA scaling events: kubectl describe hpa api-gateway.
2. Install KEDA on GKE. Create a ScaledObject for application-processor consumer group watching student.applications topic, lagThreshold=10000, minReplicaCount=1, maxReplicaCount=12 (= partition count).
3. Simulate lag spike: produce 1M messages rapidly. Observe KEDA scale out from 1 to 12 consumer pods. Monitor lag reduce to 0. Observe scale-back after stabilization window.
4. Configure scale-to-zero: set minReplicaCount=0 for a non-critical consumer. Observe it scales to 0 after all messages consumed. Observe cold start when new messages arrive.
5. Combine VPA (Vertical Pod Autoscaler) recommendation mode: install VPA for the application-processor. Run 24 hours, check VPA's recommended resource requests. Update the Deployment accordingly.

FOLLOW-UP PROBES:

- What is the difference between HPA and VPA? Can you use both simultaneously?
- HPA scale-down has a default 5-minute stabilization window. Why? What happens to in-flight Kafka messages when a consumer pod is scaled down?
- KEDA scales your Kafka consumer to 12 pods (= partition count). You add 12 more Kafka partitions. What happens to KEDA's scaling behavior?
- Your HPA is configured for CPU but your Go microservice is I/O-bound (waiting on PostgreSQL queries). CPU stays at 20% but response times are 3 seconds. HPA won't scale. What do you do?

[M] MID Category 10 > Kubernetes > GKE-Specific

Q: What GKE-specific features does Deepta AI's platform use? Explain Workload Identity, node pools, and GKE Autopilot vs Standard.

ANSWER:

GKE (Google Kubernetes Engine) adds GCP-specific features on top of vanilla Kubernetes:

Workload Identity (critical for security): Allows Kubernetes Service Accounts (KSA) to impersonate GCP Service Accounts (GSA) without managing service account key files.

How it works:

1. Create a GCP Service Account (GSA) with required IAM roles (e.g., Cloud SQL Client, Pub/Sub Publisher)
2. Create a Kubernetes Service Account (KSA) in your namespace
3. Bind them: `gcloud iam service-accounts add-iam-policy-binding gsa@project.iam.gserviceaccount.com --member "serviceAccount:project.svc.id.goog[namespace/ksa]" --role roles/iam.workloadIdentityUser`
4. Annotate KSA: `kubectl annotate serviceaccount ksa iam.gke.io/gcp-service-account=gsa@project.iam.gserviceaccount.com`
5. Pods using the KSA automatically get GCP credentials via the metadata server — no JSON key files, no secrets to rotate

For Deepta AI: application-service KSA gets Cloud SQL Client role (connects to Cloud SQL without password). kafka-producer KSA gets Pub/Sub Publisher role. No secret management complexity.

Node Pools: Groups of nodes with identical configuration (machine type, disk size, labels, taints). Allows heterogeneous clusters:

- Default pool: n2-standard-4, preemptible, for application pods (cheap, stateless)
- Memory-optimized pool: m2-ultramem-208, for Redis/PostgreSQL pods (high RAM, stable)
- GPU pool: n1-standard-4 + nvidia-tesla-t4, for ML inference (Vigilant AI copilot)
- System pool: e2-standard-4, non-preemptible, for kube-system pods (reliable)

Node auto-provisioning (NAP): GKE automatically creates new node pools when pods can't be scheduled due to resource constraints (e.g., GPU pod needs GPU node that doesn't exist → NAP creates a GPU node pool).

Cluster Autoscaler: Scales the number of nodes in a pool. Adds nodes when pods are Pending (insufficient resources). Removes nodes when underutilized (< 50% for configurable duration). Works per node pool.

GKE Autopilot vs Standard:

- Standard: Full control over node configuration, machine types, OS. You manage node pools, cluster autoscaler. Billed by node.
- Autopilot: Google manages all node infrastructure. You only specify pod resource requests. Billed by pod (not node). Automatic security hardening, auto-upgrade, auto-repair. Limited customization. Recommended for teams who don't want to manage Kubernetes infrastructure.

For Deepta AI: Start with Autopilot for simplicity. Migrate to Standard if you need: GPU nodes, specific machine types for Kafka StatefulSets, advanced networking configurations.

IMPLEMENTATION CHALLENGE:

Implement Workload Identity for Deepta AI's Cloud SQL connection:

1. Create GSA: `gcloud iam service-accounts create deepta-app-sa`. Grant Cloud SQL Client role.
2. Create KSA: `kubectl create serviceaccount deepta-app-ksa -n production`.
3. Bind: `gcloud iam service-accounts add-iam-policy-binding`. Annotate KSA.
4. Update Deployment: `spec.serviceAccountName: deepta-app-ksa`.
5. Go application: remove password from connection string. Use Cloud SQL Auth Proxy sidecar OR Cloud SQL connector (`cloudsql.googleapis.com/go`). Verify connection works without explicit credentials.
6. Set up node pools for Deepta AI:
 - app-pool: n2-standard-4, autoscale 2-20, preemptible, label app-tier=application
 - data-pool: n2-highmem-8, autoscale 1-5, non-preemptible, label app-tier=data
7. Add node affinity to Kafka StatefulSet: preferably schedule on data-pool. Add tolerations for preemptible: avoid kafka on preemptible nodes (data loss risk).

FOLLOW-UP PROBES:

- What are the security risks of using Kubernetes Secrets to store GCP Service Account JSON keys vs using Workload Identity?
- What is a preemptible/spot VM in GKE and what happens to pods when the node is preempted? Which of Deepta AI's workloads should NOT run on preemptible nodes?
- What is GKE's node auto-repair and how does it interact with PodDisruptionBudgets?
- Your Deepta AI GKE cluster costs \$15,000/month. The FinOps team asks you to reduce costs by 40%. What are the highest-impact changes (node rightsizing, preemptible VMs, Autopilot migration, idle workload scaling)?

[M] MID Category 10 > DevOps > CI/CD & GitHub Actions

Q: Design a production-grade CI/CD pipeline for Deepta AI's Go microservices on GKE using GitHub Actions.

ANSWER:

A production CI/CD pipeline for microservices must: validate code quality, ensure test coverage, build secure images, deploy safely, and provide observability.

Pipeline stages:

1. PR Validation (runs on every PR):

- Lint: golangci-lint (includes staticcheck, errcheck, govet, gofmt)
- Unit tests: go test ./... -race -coverprofile=coverage.out; fail if coverage < 80%
- Security scan: govulncheck (Go vulnerability scanner), trivy (dependency scan)
- Build: go build (fail fast on compilation errors)
- Integration tests: spin up Docker Compose (PostgreSQL, Redis, Kafka) and run integration test suite

Time target: < 5 minutes

2. Merge to main (triggers deployment):

- All PR checks pass
- Build Docker image with multi-stage build
- Scan image: trivy image --exit-code 1 --severity HIGH,CRITICAL (fail on critical vulns)
- Tag image: commit SHA (immutable), :latest (mutable)
- Push to GCP Artifact Registry (not DockerHub — private, GKE has direct access with Workload Identity)

3. Deploy to staging:

- Update Kubernetes manifest (image tag) in a GitOps repo (ArgoCD watches it)
- OR: kubectl set image deployment/app-service app=gcr.io/project/app:sha
- Run smoke tests against staging URL
- If pass: auto-promote to production

4. Deploy to production:

- Canary: 10% traffic to new version (Argo Rollouts or Ingress canary)
- Monitor: error rate, p99 latency (Prometheus/Cloud Monitoring) for 5 minutes
- Auto-promote if metrics within SLO
- Auto-rollback if SLO breach

5. Rollback:

- Automatic: ArgoCD detects health check failures → sync to previous revision
- Manual: git revert → push → pipeline triggers new deployment of old image
- Emergency: kubectl rollout undo deployment/app-service (immediate)

GitHub Actions specifics:

- Caching: actions/cache for Go modules, Docker layer cache
- Secrets: GCP Service Account key stored as GitHub secret (or use Workload Identity Federation from GitHub Actions → GCP — the recommended approach)
- Matrix builds: test across Go 1.21, 1.22 simultaneously
- Parallelism: unit tests and lint run in parallel

- Environments: staging and production environments with required approvals for production

IMPLEMENTATION CHALLENGE:

Write the complete GitHub Actions workflow for Deepta AI:

1. `.github/workflows/ci.yml`: triggered on PR.
jobs: lint (golangci-lint), test (go test -race, upload coverage to Codecov), build (docker build, trivy scan).
Use actions/cache for \$GOPATH/pkg/mod.
2. `.github/workflows/cd.yml`: triggered on push to main.
jobs: build-push (docker build + push to Artifact Registry using Workload Identity Federation), deploy-staging (kubectl set image, run smoke tests), deploy-production (Argo Rollouts canary, wait for analysis).
3. Workload Identity Federation from GitHub Actions: configure GCP Workload Identity Pool to trust GitHub's OIDC token. No JSON key stored in GitHub Secrets.
4. Implement a database migration step: before deploying, run golang-migrate up in a Job. Ensure migrations are backward-compatible (both old and new app versions can run against the migrated schema).
5. Add deployment notification: on success/failure, post to Slack via webhook (actions/webhook step).
Include: service name, version, deploy time, who triggered.

FOLLOW-UP PROBES:

- What is Workload Identity Federation for GitHub Actions and why is it preferred over storing GCP service account JSON keys as GitHub Secrets?
- Your Go microservice CI takes 12 minutes. How do you reduce it to under 3 minutes? List specific optimizations.
- What is a GitOps workflow (ArgoCD vs Flux) vs a push-based CI/CD (direct kubectl apply in pipeline)? What are the tradeoffs?
- Your database migration (golang-migrate up) runs during deployment and is not backward-compatible. How do you prevent downtime during the migration window?

[M] MID Category 10 > DevOps > Observability

Q: Explain the three pillars of observability: metrics, logs, and traces. What are the RED and USE methods?

ANSWER:

Observability is the ability to understand a system's internal state from its external outputs. The three pillars:

1. METRICS: Numeric measurements over time. Aggregated, efficient to store and query.

Types:

- Counter: monotonically increasing (`http_requests_total`, `kafka_messages_produced_total`)
- Gauge: point-in-time value (`memory_usage_bytes`, `goroutines_active`)
- Histogram: distribution of values with configurable buckets (`http_request_duration_seconds`)
- Summary: pre-computed percentiles (less flexible, client-side)

Stack: Prometheus (collection, storage, alerting) + Grafana (visualization).

Prometheus scrapes /metrics endpoints from pods. Service discovery via Kubernetes pod annotations.

2. LOGS: Timestamped, structured records of discrete events. High cardinality, high detail.

Best practice: Structured logs (JSON) with consistent fields: timestamp, level, service, trace_id, span_id, message, error.

Go: zerolog or zap for structured logging. Log to stdout (12-factor app principle) → collected by Fluentbit DaemonSet → shipped to Cloud Logging (GCP) or Loki.

Deepta AI practice: Every log line includes trace_id and span_id — enables correlation between logs and traces.

3. TRACES: Distributed tracing captures the journey of a request across multiple services.

OpenTelemetry: the standard. Instrumented with spans (unit of work). Spans have: traceID, spanID, parentSpanID, service name, operation name, timestamps, attributes.

Stack: OpenTelemetry SDK in Go → collector → Jaeger or Tempo (storage) → Grafana (visualization).

RED Method (for services — primarily useful for request-based services):

- Rate: requests per second (is the service being called?)
- Errors: error rate (is the service returning errors?)
- Duration: latency distribution (is the service slow?)

Three metrics for any service → first dashboard to build.

USE Method (for resources — primarily useful for infrastructure/hardware):

- Utilization: average time the resource is busy (CPU %)
- Saturation: extra work queued (CPU run queue length, disk I/O wait)
- Errors: error events (disk errors, network drops)

Applied to: CPU, memory, disks, network interfaces, Kafka consumer lag (as a queue saturation metric).

For Deepta AI: Implement RED metrics on every Go HTTP handler (middleware), USE metrics on Kafka consumers (lag = saturation), OpenTelemetry traces across the application → Kafka → consumer pipeline.

IMPLEMENTATION CHALLENGE:

Build the observability stack for Deepta AI:

1. Prometheus metrics in Go:

- HTTP middleware: record `http_requests_total` (method, path, status), `http_request_duration_seconds` (histogram with buckets [0.01, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5]).
- Kafka consumer metrics: `kafka_consumer_lag{topic, partition}`, `kafka_messages_processed_total{topic, status}`.
- Business metrics: `applications_submitted_total{university, lead_source}`, `applications_processed_total{status}`.

2. OpenTelemetry tracing:

- Instrument the request flow: HTTP handler → Kafka produce → Kafka consume → PostgreSQL write.
- Each step = a span. Propagate trace context via HTTP headers (W3C Trace Context) and Kafka headers.
- Export to Tempo (via OTLP). Visualize trace in Grafana: see the full request path from API call to database write.

3. Structured logging with zap:

```
logger.Info("application.submitted", zap.String("app_id", appId), zap.String("trace_id", traceID),
zap.Duration("db_write_latency", dbLatency))
```

4. Grafana dashboards: RED dashboard for each service, USE dashboard for cluster nodes, Kafka consumer lag dashboard, business metrics (applications per day per university).

5. Alert rules: HTTP error rate > 1% for 5min, p99 latency > 2s for 5min, Kafka consumer lag > 50k for 10min, node memory utilization > 80%.

FOLLOW-UP PROBES:

- What is the difference between metrics (Prometheus) and logs (Loki) for debugging a slow query? When do you reach for each?
- How does distributed tracing handle async operations (like a Kafka producer/consumer)? How does the trace context propagate across the async boundary?
- Your Deepta AI service has 1000 pods generating 10GB of logs per day. Cloud Logging costs are high. How do you reduce log volume without losing critical information?
- What is exemplar-based observability? How do Prometheus histograms with exemplars enable jumping from a metric data point directly to the trace that caused it?

[A] **ADVANCED** Category 10 > Kubernetes > Zero-Downtime Migration

Q: Design a zero-downtime migration of Deepta AI's stateful Go service from one GKE cluster to a new cluster. What are all the failure points?

ANSWER:

Zero-downtime migration of a stateful service is one of the hardest DevOps challenges. The service has:

- Stateful: sessions in Redis, data in PostgreSQL
- Kafka consumers: must not miss or duplicate messages

- WebSocket connections: long-lived, can't be interrupted
- External dependencies: Google Ads webhooks, university integrations hitting the service's IP

PHASE 1 — PREPARATION (weeks before):

1. Data layer decoupling: Ensure Redis and PostgreSQL are NOT on the old cluster. They must be external managed services (Cloud SQL, Cloud Memorystore) accessible from both clusters. If they're cluster-internal, migrate them first.
2. Stateless application code: Review all in-process state. Local caches must be backed by Redis. Configuration must come from ConfigMaps/Secrets (not baked into container). All state must be externalizable.
3. New cluster setup: GKE cluster with identical configuration, node pools, network policies. Same VPC for access to Cloud SQL/Memorystore.
4. Traffic routing: GCP Load Balancer with global anycast IP (Global HTTP(S) LB). Both clusters behind the same LB. Traffic weight controlled by backend service weights.

PHASE 2 — PARALLEL OPERATION:

5. Deploy services to new cluster: deploy all microservices, verify health in isolation (no traffic yet). Test: can they connect to Cloud SQL, Redis, Kafka?
6. Shadow traffic (optional): route 0% of actual traffic, but mirror a copy (using GCP Traffic Mirroring) to new cluster to observe behavior under realistic load.
7. Kafka consumer migration: Start consumers on NEW cluster ALSO in the SAME consumer group. They will take over some partitions (rebalance). Old cluster consumers handle remaining partitions. Both are active — this is fine for Kafka's partition-based parallel processing.
8. Shift traffic: update GCP LB backend weights. 10% new cluster, wait 15 minutes, observe error rates. If healthy: 25%, 50%, 75%, 100%. Automated rollback: if error rate spikes > threshold, immediately reweight back to 100% old.

PHASE 3 — CUTOVER:

9. WebSocket connections: long-lived connections on old cluster drain naturally (users who navigate away). Force drain with a preStop hook: sleep 60 (gives in-flight requests time to complete). Set `terminationGracePeriodSeconds=120`.
10. Webhook IP change: Update DNS or GCP LB IP. Change can't be instant (DNS propagation). Run dual-listen on both IPs during TTL window. Coordinate with partners (Google Ads, Facebook) that hit your webhook URL.

FAILURE POINTS:

- Cloud SQL connection exhaustion: both clusters try to connect simultaneously → double connection count → PgBouncer is essential
- Kafka rebalance during migration: consumer group rebalances with mixed old/new consumers → use cooperative sticky to minimize disruption
- DNS TTL: old IPs may still receive traffic for TTL duration → old cluster must stay running
- Session state: Redis sessions are shared (correct) but if Redis is unavailable during migration → all sessions lost
- Kafka consumer group offset divergence: if both clusters process different partitions simultaneously and one commits incorrect offsets

IMPLEMENTATION CHALLENGE:

Execute the migration plan for Deepta AI:

1. Infrastructure validation: connectivity test from new GKE cluster → Cloud SQL (Cloud SQL Auth Proxy), Redis (Memorystore VPC peering), Kafka (broker addresses). Automate with a connectivity-check Job that tests all dependencies and fails if any are unreachable.
2. Traffic splitting script: a Go program that uses GCP Compute API to update backend service weights. Input: percentage (0-100). Validates current cluster health before shifting.
3. Canary migration: deploy new cluster with 0% traffic. Shift to 5%. Monitor for 10 minutes. If `error_rate < 0.1%` AND `p99 < 500ms`: proceed. Else: rollback. Implement this as a GitHub Actions workflow with Prometheus query validation.
4. WebSocket drain: in the Go service, on SIGTERM: stop accepting new WebSocket connections, set a flag that returns 503 for new requests, let existing connections complete naturally for up to 60 seconds. Test by holding a WebSocket connection open during pod shutdown — verify it receives a clean close frame.

5. Rollback procedure: document the exact commands to shift 100% traffic back to old cluster in under 60 seconds. Practice this as a runbook drill.

FOLLOW-UP PROBES:

- What is a PodDisruptionBudget and how does it protect Deepta AI during the migration? Set maxUnavailable=1 for a 5-replica Deployment during voluntary disruptions.
- During the migration, both clusters run PostgreSQL migrations on startup. What happens if both run the migration simultaneously?
- Your WebSocket connections are 30 minutes old on average. Setting terminationGracePeriodSeconds=1800 (30 min) makes deployments painfully slow. How do you balance connection draining vs deployment speed?
- The old GKE cluster is on Kubernetes 1.26 and the new is 1.29. Are there API version differences that could break your existing manifests? How do you audit for this?

[P] PRINCIPAL Category 10 > DevOps > Principal Design

Q: Design the complete production infrastructure for Deepta AI's platform targeting 99.95% uptime, 100ms p99 latency for student-facing APIs, and compliance with India's DPDP Act (data residency in india-south1).

ANSWER:

This is a Principal-level infrastructure design problem. Here is the complete reference architecture:

ARCHITECTURE OVERVIEW:

Multi-zone GKE Standard cluster in asia-south1 (Mumbai) with 3 zones. All PII data must stay in asia-south1. No multi-region replication of PII.

COMPUTE LAYER:

GKE Standard cluster in asia-south1:

- System node pool: e2-standard-4, 3 nodes (1 per zone), non-preemptible
- Application node pool: n2-standard-8, autoscale 3-30 nodes, preemptible (cost savings for stateless)
- Data node pool: n2-highmem-16, autoscale 2-6, non-preemptible (Redis, Kafka)
- Total baseline: ~15 nodes, scales to 50+ during peak admission season

DATA LAYER (all in asia-south1):

- Cloud SQL PostgreSQL: HA with synchronous standby in different zone. Automated backups to GCS asia-south1. PITR enabled. Private IP only (no public endpoint).
- Memystore Redis: Standard tier (HA with failover replica in different zone). Private Service Connect.
- Kafka on GKE: 3-broker StatefulSet across 3 zones, SSD PersistentDisks, replication.factor=3, min.insync.replicas=2.
- Artifact Registry: asia-south1 for container images.
- GCS: asia-south1 bucket for student documents (encrypted with Cloud KMS CMEK).

NETWORKING:

- VPC: custom VPC, subnets in asia-south1-a/b/c
- Private Google Access: pods access Google services without internet
- Cloud Armor: DDoS protection, WAF rules on external load balancer
- Cloud NAT: outbound internet for pods (no external IPs on nodes)
- Private Service Connect: Cloud SQL, Memystore accessed via private endpoints
- Ingress: GKE Ingress (L7 Global HTTP(S) LB with managed SSL cert, HTTPS only)

HIGH AVAILABILITY (99.95% = 4.38 hours/year downtime):

- All components spread across 3 zones
- PodDisruptionBudgets: min 2 replicas always available during disruptions
- HPA: min 3 replicas for student-facing services (1 per zone)
- Anti-affinity: pods spread across zones (podAntiAffinity preferredDuringScheduling spread across zones)

- Patroni/Cloud SQL HA: <60s RTO for database failover
- Kafka: ISR across all 3 zones, leader election <10s

100ms p99 LATENCY:

- API tier: Go services, in-memory LRU cache for hot data, Redis for shared cache
- Target: DB query < 10ms (index optimization), Redis < 2ms, Kafka produce < 5ms, application logic < 20ms
- Horizontal scaling: HPA triggers scale-out before latency degrades (CPU threshold 60%)
- CDN: Cloud CDN for static assets (student portal frontend)

DPDP ACT COMPLIANCE (India):

- Data residency: all data in asia-south1, no cross-region replication
- Consent management: store consent records in Cloud SQL, immutable audit trail
- Data retention: automated deletion jobs after consent withdrawal (DPDP Article 8)
- Encryption: Cloud KMS CMEK for Cloud SQL, GCS, Memorystore (BYOK for premium compliance)
- Access logging: Cloud Audit Logs for all data access (Cloud SQL, GCS)
- Data processor agreements: with all third-party integrations (Google Ads, Facebook)

MONITORING & ON-CALL:

- Cloud Monitoring: uptime checks every 30s from multiple GCP regions
- Alert routing: PagerDuty for P0/P1, Slack for P2/P3
- Runbooks: for each alert, automated diagnosis (Cloud Functions run diagnostic queries on alarm)
- Incident response: 15-min SLA for P0 response, 1-hour for P1

IMPLEMENTATION CHALLENGE:

Implement key pieces of this architecture:

1. Terraform modules: GKE cluster (with all node pools, Workload Identity), Cloud SQL HA (private IP, read replica), Memorystore (HA tier), VPC with firewall rules, Cloud Armor policy.
2. GKE Autopilot evaluation: compare monthly cost of Standard (current design) vs Autopilot (billing by pod resource requests). For Deepta AI's workload profile, calculate break-even point.
3. SLO definition and burn rate alerts: define SLOs (availability 99.95%, p99 latency <100ms). Implement multi-window burn rate alerts: fast burn (1-hour window, 14x budget consumption) → page immediately. Slow burn (6-hour window, 5x budget) → ticket.
4. Chaos engineering: use Chaos Mesh (Kubernetes chaos engineering tool). Monthly test: (a) kill all pods in zone asia-south1-a, verify service continues from b+c, (b) network partition between app pods and Cloud SQL, verify circuit breaker activates, (c) Kafka broker 0 failure, verify consumer rebalance and message processing continues.
5. DPDP compliance audit: automated daily job that queries Cloud Audit Logs for unauthorized data access. Send report to compliance team. Flag any access patterns that don't match expected service accounts.

FOLLOW-UP PROBES:

- Your 99.95% SLO allows 4.38 hours of downtime per year. A Cloud SQL failover takes 60 seconds (happens quarterly for maintenance). How much of your error budget does each failover consume?
- A student reports that their application data is visible to another student (data leak). Walk through the incident response: immediate containment, root cause investigation, and post-incident remediation.
- India's DPDP Act requires data deletion within 30 days of consent withdrawal. A student withdraws consent. Their data is in: PostgreSQL, Redis, Kafka (7-day retention), GCS (3-year retention), Artifact Registry (container env vars with student data — hypothetically). How do you handle each?
- Your team wants to deploy a new feature that processes student documents using a third-party ML API hosted outside India. What DPDP compliance requirements must you satisfy before this can go live?

QUESTION COUNT SUMMARY

Category 07 — Databases & SQL: 24 questions [E]=6 [M]=11 [A]=5 [P]=2

Category 08 — Caching & Redis: 15 questions [E]=6 [M]=6 [A]=2 [P]=1

Category 09 — Kafka & Event Streaming: 14 questions [E]=6 [M]=5 [A]=2 [P]=1

Category 10 — DevOps, Kubernetes & Cloud: 13 questions [E]=6 [M]=5 [A]=1 [P]=1

TOTAL: 66 questions across 4 categories

SESSION COVERAGE

- Session 1: JavaScript, TypeScript, React, Frontend Systems
- Session 2: Go Concurrency, Python Async, REST/GraphQL/WebSocket APIs, Auth & Security
- **Session 3 (this document): Databases & SQL, Redis, Kafka, DevOps & Kubernetes**
- Session 4: AI/LLM Engineering, RAG, Agents, MCP, Transformers, Embeddings
- Session 5: System Design, Behavioral, DSA Hard, Leadership & Ownership